

CraftCircle - Aplicație web dedicată promovării, vânzării și schimbului de idei între pasionații de produse handmade

Candidat: Oxani Corina

Coordonator: Șl. Dr. Ing. Răzvan Cioargă

Sesiunea: Iulie 2025

REZUMAT

„CraftCircle” este o aplicație web creată pentru a sprijini pasionații de produse handmade în promovarea, vânzarea și descoperirea creațiilor într-un mediu digital prietenos și interactiv. Platforma le oferă utilizatorilor posibilitatea de a-și crea propriul magazin virtual, de a publica postări cu lucrările lor și de a comunica direct printr-un sistem de mesagerie privată.

Mai mult decât un simplu marketplace, „CraftCircle” își propune să încurajeze conexiunile autentice între membri. Prin sistemul de follow, aprecieri, comentarii și notificări în timp real, utilizatorii pot rămâne conectați, pot descoperi alți creatori, adăuga produse în favorite sau în coșul de cumpărături și plasa comenzi cu opțiuni diverse de plată și livrare.

Aplicația a fost dezvoltată cu React pentru partea de interfață, folosind Node.js și Express pentru backend. Stilizarea a fost realizată cu module CSS, utilizând fonturi personalizate, animații și decoruri tematice, pentru a oferi o experiență vizuală coerentă și atractivă. Baza de date este gestionată cu PostgreSQL, iar imaginile sunt stocate în Cloudinary. Funcționalitățile în timp real sunt susținute de Socket.IO, iar trimiterea automată a emailurilor este realizată cu ajutorul Nodemailer.

CUPRINS

REZUMAT	3
CUPRINS.....	5
LISTĂ DE FIGURI.....	9
LISTĂ DE TABELE	11
LISTĂ DE SECVENȚE DE COD.....	13
1. INTRODUCERE	17
1.1 CONTEXTUL PROBLEMEI	17
1.2 SCOPUL APLICAȚIEI.....	17
1.3 CONTRIBUȚII PERSONALE	18
1.4 ANALIZA DOMENIULUI STATE-OF-THE-ART	19
1.4.1 Etsy	20
1.4.2 Amazon Handmade	20
1.4.3 Breslo.....	20
2. TEHNOLOGII UTILIZATE ȘI BAZA TEORETICĂ	23
2.1 FRONTEND.....	23
2.1.1 React.....	23
2.1.2 CSS Modules	23
2.1.3 Figma	24
2.1.4 Servicii API externe.....	24
2.2 BACKEND	24
2.2.1 Node.js și Express	24
2.2.2 Cloudinary.....	25
2.2.3 Passport.js	26
2.2.4 Socket.IO	26
2.2.5 Stripe.....	26
2.2.6 Nodemailer.....	27
2.3 BAZA DE DATE.....	27
2.3.1 PostgreSQL	27
2.3.2 pgAdmin.....	28
2.3.3 Postman.....	28
2.4 CONTROL VERSIUNI	28
2.4.1 Git	28

3. PROIECTAREA SISTEMULUI	31
3.1 PROIECTAREA GENERALĂ A APLICAȚIEI	31
3.1.1 Frontend-ul aplicației (React)	31
3.1.2 Backend-ul aplicației (Node.js + Express).....	31
3.1.3 Baza de date (PostgreSQL)	32
3.1.4 Servicii externe	32
3.2 DIAGRAMA UML A CAZURILOR DE UTILIZARE.....	33
3.3 NAVIGAREA ÎN APLICAȚIE	35
3.3.1 Înregistrare și autentificare	35
3.3.2 Redirecționare după logare	36
3.3.3 Bara de navigare.....	36
3.4 PROIECTAREA BAZEI DE DATE	39
4. IMPLEMETAREA FUNCȚIONALITĂȚILOR	41
4.1 MECANISME DE AUTENTIFICARE ȘI GESTIONARE CONT	43
4.1.1 Înregistrare prin formular	43
4.1.2 Înregistrare și autentificare socială.....	45
4.1.3 Completarea profilului	47
4.1.4 Integrarea în contextul global	47
4.1.5 Autentificare standard (administratori și utilizatori).....	48
4.1.6 Resetarea parolei.....	50
4.1.7 Editarea datelor personale și schimbarea parolei	51
4.2 FUNCȚIONALITĂȚI SOCIALE	53
4.2.1 Gestionarea postărilor: adăugare, editare și ștergerea	53
4.2.2 Aprecierea, comentarea și raportarea postărilor	57
4.2.3 Interacțiuni între utilizatori: urmărire și raportare	60
4.2.4 Pagina Discover – explorare de postări și produse cu filtre dinamice ..	62
4.2.5 Sistemul de notificări (like, follow, comentarii).....	65
4.2.6 Funcționalități comerciale.....	67
4.2.7 Funcționalități de comunicare	76
4.2.8 Funcționalități administrative	81
4.3 TABELUL RUTELOR BACKEND	85
5. UTILIZAREA APLICAȚIEI DE CĂTRE UTILIZATORI	101
5.1 ÎNREGISTRARE ȘI AUTENTIFICARE	101

5.1.1	Înregistrare prin formular	101
5.1.1	Înregistrare și autentificare cu Google / Facebook	101
5.1.2	Completare date profil	102
5.1.3	Autentificarea prin formular	102
5.1.4	Resetarea parolei	103
5.2	PAGINA DE PROFIL	104
5.2.1	Vizualizarea profilului personal și al altor utilizatori	104
5.2.2	Pagina Followers	105
5.2.3	Secțiunea Following	106
5.2.4	Pagina Shop personal	107
5.3	PAGINA DE APRECIERI	108
5.4	PAGINA DE MESAJE	109
5.5	PAGINA DE FAVORITE	110
5.6	PAGINA COȘ DE CUMPĂRĂTURI	111
5.7	PAGINA COMENZI	112
5.8	MENIUL DIN BARA DE NAVIGAȚIE	113
6.	UTILIZAREA APLICAȚIEI DE CĂTRE ADMINISTRATOR	115
6.1	AUTENTIFICARE ADMIN	115
6.2	PAGINA DE PROFIL	115
6.3	GESTIONAREA CATEGORIILOR	115
6.4	MODERAREA UTILIZATORILOR	116
6.5	MODERAREA POSTĂRILOR	117
6.6	MODERAREA PRODUSELOR	117
6.7	MANAGEMENTUL COMENZILOR	118
6.8	STATISTICI GENERALE	119
6.9	MENIUL DIN BARA DE NAVIGARE	120
7.	TESTARE ȘI VALIDARE	121
7.1	TESTARE FUNCȚIONALĂ (BLACK BOX)	121
7.2	TESTARE STRUCTURALĂ (WHITE BOX)	121
7.3	LIMITĂRI ȘI DIRECȚII VIITOARE	122
8.	CONCLUZII	123
	BIBLIOGRAFIE	127

LISTĂ DE FIGURI

Figura 3.1 Arhitectura Aplicației	33
Figura 3.2 Diagrama UML a cazurilor de utilizare pentru utilizator – Aplicația „CraftCircle”	34
Figura 3.3 Diagrama UML a cazurilor de utilizare pentru administrator	35
Figura 3.4 Bara de navigare pentru utilizatorul aplicației „CraftCircle”	37
Figura 3.5 Bara de navigare pentru administratorul aplicației „CraftCircle”	37
Figura 3.6 Diagrama de navigare a utilizatorului normal.....	38
Figura 3.7 Diagrama de navigare a utilizatorului administrator	38
Figura 3.8 Diagrama relațională a bazei de date „CraftCircle” generată în pgAdmin (include toate entitățile, relațiile și atributele esențiale pentru funcționarea completă a aplicației) ..	40
Figura 4.1 Interfața de înregistrare	45
Figura 4.2 Interfața de adăugare informații adiționale.....	47
Figura 4.3 Interfața de login și modalul de resetare a parolei	51
Figura 5.1 Interfața Sign up	101
Figura 5.2 Opțiuni de înregistrare sau autentificare cu Google și Facebook din pagin Sign up.....	101
Figura 5.3 Opțiuni de înregistrare sau autentificare cu Google și Facebook din pagin Log in	102
Figura 5.4 Interfața de completare a datelor	102
Figura 5.5 Intefată Log In.....	103
Figura 5.6 Modale Reset Password	103
Figura 5.7 Pagina de profil - utilizator logat.....	104
Figura 5.8 Interfața profilului unui alt utilizator (vizualizat de către utilizatorul logat).....	105
Figura 5.9 Schimbarea pozei de profil	105
Figura 5.10 Formular de creare a unei postări.....	105
Figura 5.11 Modal comentarii aferente unei postări.....	105
Figura 5.12 Modal raportare utilizator	105
Figura 5.13 Pagina Followers: afișarea urmăritorilor	106
Figura 5.14 Pagina Following: afișarea utilizatorilor urmăriți.....	106
Figura 5.15 Formular de adăugare produs în pagina de profil personal	107
Figura 5.16 Carduri de produs vizibile pentru alți utilizatori, cu opțiuni de raportare, adăugare în coș și favorite	107
Figura 5.17 Carduri de produs proprii, cu opțiuni de editare, ștergere și actualizare a stocului	107
Figura 5.18 Interfață pagina de Aprecieri.....	108
Figura 5.19 Notificare apreciere.....	108
Figura 5.20 Interfață pagină mesagerie	109
Figura 5.21 Mesaje interlocutor	110
Figura 5.22 Căutarea unui utilizator și începerea unei conversații.....	110
Figura 5.23 Interfața paginii de favorite.....	111
Figura 5.24 Interfața paginii „Shopping Cart”	112

Figura 5.25 Formular comandă plată ramburs	112
Figura 5.26 Formular comandă plată prin card	112
Figura 5.27 Interfață pagina comenzi	113
Figura 5.28 Formular schimbare parolă	114
Figura 5.29 Formular editare date personale	114
Figura 5.30 Modal de contact	114
Figura 5.31 Modal confirmare ștergere cont	114
Figura 5.32 Modal confirmare ieșire din cont	114
Figura 6.1 Interfață pagina profil administrator	115
Figura 6.2 Interfață pagina „Manage Categories”	116
Figura 6.3 Interfață pagina „Moderate Users”	116
Figura 6.4 Interfață pagin „Moderate posts”	117
Figura 6.5 Interfață pagina „Moderate Products”	118
Figura 6.6 Interfață pagin „Admin Order Management”	119
Figura 6.7 Statistici despre utilizatori în panoul de administrare	119
Figura 6.8 Lista utilizatorilor afișată în modalul de administrare	120
Figura 8.1 Interfețele pentru autentificare și înregistrare realizate în Figma	125

LISTĂ DE TABELE

Tabel 1.1 Compararea principalelor funcționalități între cele mai apreciate trei aplicații (Etsy, Amazon Handmade și Breslo) și „CraftCircle”. Funcționalitățile disponibile sunt notate cu X, iar cele indisponibile cu un –	20
Tabel 4.1 Lista completă a endpointurilor backend RESTful	86

LISTĂ DE SECVENȚE DE COD

Secvență de cod 4.1 Punctul de intrare în aplicația React (index.js)	41
Secvență de cod 4.2 Funcția ScrollToTop() și definirea rutelor (App.js)	41
Secvență de cod 4.3 Fișierul .env din frontend și integrarea cheilor Stripe și OpenRouteService	42
Secvență de cod 4.4 Configurarea completă a serverului backend (server.js)	43
Secvență de cod 4.5 Exemplu de câmp obligatoriu în formularul de înregistrare	43
Secvență de cod 4.6 Verificarea potrivirii parolelor	44
Secvență de cod 4.7 Validarea complexității parolei	44
Secvență de cod 4.8 Apelul către backend și salvarea datelor în localStorage	44
Secvență de cod 4.9 Înregistrarea unui nou utilizator în baza de date	45
Secvență de cod 4.10 Verificarea existenței utilizatorului și înregistrarea automată prin Facebook Oauth	46
Secvență de cod 4.11 Redirecționarea utilizatorului după autentificare socială	46
Secvență de cod 4.12 Validarea vârstei minime de 16 ani pentru înregistrare	47
Secvență de cod 4.13 Emiterea automată a evenimentului join după login	48
Secvență de cod 4.14 Inițializarea stării formularului de autentificare	48
Secvență de cod 4.15 Actualizarea dinamică a valorilor din formular	48
Secvență de cod 4.16 Determinarea tipului de utilizator și trimiterea cererii de autentificare	49
Secvență de cod 4.17 Backend autentificare admin și user normal	49
Secvență de cod 4.18 Redirecționarea către pagina de profil în cazul unei autentificări reușite	50
Secvență de cod 4.19 Generarea unui cod de securitate și trimiterea prin email	50
Secvență de cod 4.20 Preluarea dinamică a țărilor și orașelor	51
Secvență de cod 4.21 Trimiterea datelor modificate către backend	52
Secvență de cod 4.22 Backend - rutele pentru schimbarea pozei de profil administrator + utilizator	52
Secvență de cod 4.23 Accesarea contextului utilizatorului	53
Secvență de cod 4.24 Trimiterea cererii de actualizare a parolei către backend	53
Secvență de cod 4.25 Renderează formularul de postare doar pentru utilizatorul autentificat	53
Secvență de cod 4.26 Inițializarea stării pentru fișierele postării și previzualizarea acestora	54
Secvență de cod 4.27 Gestionarea fișierelor încărcate și generarea previzualizărilor	54
Secvență de cod 4.28 Eliminarea fișierelor încărcate din formularul de postare	55
Secvență de cod 4.29 Construirea și trimiterea datelor postării către backend	55
Secvență de cod 4.30 Endpoint backend pentru încărcarea postării și fișierelor media în baza de date	56
Secvență de cod 4.31 Formular de editare a conținutului unei postări	56
Secvență de cod 4.32 Funcția de ștergere a unei postări	56
Secvență de cod 4.33 Backend, ruta de ștergere a unei postări	57

Secvență de cod 4.34 Obținerea stării de like.....	57
Secvență de cod 4.35 Comutarea stării de like la click.....	58
Secvență de cod 4.36 Afișarea comentariilor și redirecționarea către profilul autorului	58
Secvență de cod 4.37 Trimiterea unui comentariu nou și actualizarea interfeței	59
Secvență de cod 4.38 Afișarea condiționată a meniului postării în funcție de autorul postării	59
Secvență de cod 4.39 Raportarea unei postări de către utilizator.....	59
Secvență de cod 4.40 Endpoint backend pentru raportarea unei postări	60
Secvență de cod 4.41 Verificarea relației de urmărire dintre utilizatori	60
Secvență de cod 4.42 Adăugarea și eliminarea relațiilor de urmărire (follow/unfollow)	61
Secvență de cod 4.43 Endpoint backend pentru raportarea unui utilizator	62
Secvență de cod 4.44 Salvarea căutărilor utilizatorului în istoricul postărilor	63
Secvență de cod 4.45 Construirea condițiilor de filtrare SQL în funcție de căutare, categorie, follow și sugestii personalizate.....	63
Secvență de cod 4.46 Construirea interogării SQL finale, executarea ei și atașarea media la postări	64
Secvență de cod 4.47 Excluderea postărilor proprii din feedul utilizatorului	64
Secvență de cod 4.48 Definirea filtrelor disponibile pentru postări și produse	65
Secvență de cod 4.49 Generarea notificărilor de apreciere (follow, like, comment)	65
Secvență de cod 4.50 Gestionarea notificărilor de apreciere și emiterea în timp real (backend).....	66
Secvență de cod 4.51 Recepționarea notificărilor de apreciere în timp real (frontend).....	66
Secvență de cod 4.52 Afișarea notificării vizuale pentru aprecieri primite	67
Secvență de cod 4.53 Afișarea și marcarea notificărilor de apreciere ca văzute	67
Secvență de cod 4.54 Formular pentru adăugarea unui produs – preț, descriere și imagini	67
Secvență de cod 4.55 Trimiterea datelor unui produs nou către backend și actualizarea produselor afișate	68
Secvență de cod 4.56 Editarea detaliilor unui produs și a imaginilor asociate	69
Secvență de cod 4.57 Actualizarea unui produs și gestionarea imaginilor atașate (backend)	69
Secvență de cod 4.58 Trimiterea modificărilor produsului și afișarea feedback-ului vizual	70
Secvență de cod 4.59 Meniu contextual pentru produse: editare, ștergere și raportare	70
Secvență de cod 4.60 Ștergerea unui produs din shop și actualizarea interfeței.....	70
Secvență de cod 4.61 Obținerea produselor și a numărului de articole din coș și favorite	71
Secvență de cod 4.62 Adăugarea unui produs în coș sau la favorite și afișarea notificărilor	72
Secvență de cod 4.63 Ștergerea produselor din coș și din favorite	72
Secvență de cod 4.64 Actualizarea cantității unui produs din coș	73
Secvență de cod 4.65 Calculul costului de transport pe baza distanței dintre cumpărător și vânzător	73
Secvență de cod 4.66 Confirmarea plății cu cardul prin Stripe	74

Secvență de cod 4.67 Inserarea unei comenzi în baza de date cu valorile complete ale adresei și RETURNING*	74
Secvență de cod 4.68 Golirea coșului după plasarea comenzii și notificarea componentelor frontend.....	75
Secvență de cod 4.69 Încărcarea comenzilor în pagina de moderare pentru administratori	75
Secvență de cod 4.70 Actualizarea statusului unei comenzi din interfața de administrare	76
Secvență de cod 4.71 Trimiterea unui mesaj nou în conversațiile utilizatorilor	76
Secvență de cod 4.72 Trimiterea unui mesaj nou în conversațiile utilizatorilor	77
Secvență de cod 4.73 Recepția mesajelor noi în timp real și marcarea ca citite	77
Secvență de cod 4.74 Gestionarea mesajelor necitite și a notificărilor de tip preview	78
Secvență de cod 4.75 Inițierea ștergerii mesajelor – pentru mine sau pentru toți.....	78
Secvență de cod 4.76 Finalizarea ștergerii mesajelor: doar pentru utilizator sau global...	79
Secvență de cod 4.77 Ștergerea mesajelor din conversații: logică per utilizator și eliminare definitivă.....	79
Secvență de cod 4.78 Căutarea utilizatorilor pentru mesagerie și asocierea conversațiilor existente	80
Secvență de cod 4.79 Endpoint pentru căutarea utilizatorilor în sistemul de mesagerie ...	80
Secvență de cod 4.80 Obținerea categoriilor cu numărul de postări și produse asociate ..	81
Secvență de cod 4.81 Afișarea categoriilor și activarea butonului de ștergere condiționată	82
Secvență de cod 4.82 Ștergerea unei postări de către administrator și notificarea utilizatorului.....	83
Secvență de cod 4.83 Interfață de filtrare pentru pagina de moderare a postărilor	83
Secvență de cod 4.84 Obținerea raportărilor asociate unui utilizator (moderare conturi) ..	84
Secvență de cod 4.85 Obținerea progresului statistic pentru interfața de administrare	85
Secvență de cod 4.86 Endpoint pentru evoluția lunară a produselor adăugate	85
Secvență de cod 4.87 Statistici utilizatori: total și înscriși în luna curentă.....	85

1. INTRODUCERE

1.1 CONTEXTUL PROBLEMEI

În ultimii ani, interesul pentru produsele realizate manual a cunoscut o creștere vizibilă. Tot mai mulți oameni caută obiecte autentice, personalizate, care să reflecte o estetică aparte și o poveste individuală. Produsele handmade nu mai sunt privite doar ca simple cadouri sau obiecte decorative, ci ca expresii ale creativității și valorilor personale ale creatorilor. Cu toate acestea, pentru mulți artizani independenți, pătrunderea într-un mediu digital competitiv rămâne o provocare majoră. Lipsa resurselor, a vizibilității și a unor instrumente de promovare adaptate specificului acestui tip de creație face dificilă transformarea pasiunii într-o activitate sustenabilă.

Deși există numeroase platforme de vânzare online, acestea sunt adesea orientate spre volum, viteză și produse de serie, ignorând nevoia de conexiune autentică între creatori și cumpărători. Într-un mediu online suprasaturat informațional, produsele unicate se pierd ușor printre oferte standardizate, iar artizanii întâmpină dificultăți în a-și face cunoscute lucrările. În același timp, utilizatorii interesați de astfel de produse întâmpină provocări în identificarea ofertelor relevante sau în stabilirea unui contact direct cu creatorii din spatele lor.

Dincolo de aspectul comercial, se conturează tot mai clar și o dimensiune socială a fenomenului. Mulți creatori resimt nevoia unui spațiu în care să poată împărtăși idei și să se conecteze cu persoane care le înțeleg munca. În prezent, însă, puține medii online facilitează cu adevărat interacțiuni autentice și constructive între pasionații de produse handmade. Această lipsă de conexiune și recunoaștere afectează atât parcursul profesional al artizanilor, cât și motivația de a continua să creeze într-un mod susținut.

1.2 SCOPUL APLICAȚIEI

Scopul aplicației „CraftCircle” va fi acela de a oferi un spațiu digital dedicat promovării și vânzării produselor handmade, adresându-se în special artizanilor independenți care își doresc o modalitate accesibilă de a-și face cunoscute creațiile. Platforma va veni în sprijinul acestora cu un sistem intuitiv de creare a unui magazin virtual, oferind vizibilitate într-un mediu concentrat pe originalitate, estetică și autenticitate.

Utilizatorii vor putea încărca imagini cu produsele puse spre vânzare, vor adăuga titluri, descrieri și prețuri, iar vizitatorii vor putea răsfoi ușor creațiile, adăugându-le în liste de favorite sau plasându-le în coș pentru a iniția o comandă.

Pentru a încuraja relațiile directe între creatori și clienți, „CraftCircle” va include un sistem de mesagerie privată, care va permite schimbul liber de informații între utilizatori. Acest tip de comunicare va umaniza procesul de cumpărare și va apropia oamenii unii de ceilalți.

Pe lângă dimensiunea comercială, aplicația își va propune să funcționeze și ca o rețea socială dedicată creativității. Utilizatorii vor putea publica postări vizuale prin care să

Își prezintă ideile, proiectele în lucru sau momente din procesul de creație. Postările vor putea fi comentate sau apreciate de alți utilizatori.

Astfel, „CraftCircle” nu va fi doar un loc de vânzare, ci și un spațiu de exprimare personală, de conectare și de construire a unei comunități vii, în care creativitatea circulă liber, iar pasiunea pentru lucrul manual devine punte între oameni.

1.3 CONTRIBUȚII PERSONALE

Întreaga aplicație „CraftCircle” va fi realizată integral de mine, începând de la definirea ideii și proiectarea arhitecturii, până la implementarea completă a interfeței, logoului, a logicii backend și a tuturor funcționalităților necesare pentru fiecare tip de utilizator. Aplicația va fi concepută într-un mod modular, astfel încât să permită o organizare clară, o dezvoltare facilă și o întreținere eficientă pe termen lung.

Voi implementa două tipuri principale de utilizatori:

- utilizatorul standard, care va putea publica postări, adăuga produse în shop, plasa comenzi și interacționa cu alți utilizatori prin comentarii, aprecieri și mesaje private;
- administratorul, care va avea acces la un panou de control dedicat, de unde va putea modera postări, produse, utilizatori și categorii, precum și vizualiza comenzile și statisticile platformei.

Structura aplicației va fi împărțită în două părți principale: folderul client/, care va conține tot ce ține de frontend, și folderul server/, care se va ocupa de backend.

În directorul client/, aplicația va fi organizată pe module, în foldere precum components/, pages/, modals/, hooks/, utils/, dar și în subfoldere speciale pentru zona de administrare, cum ar fi Admin/components/ și Admin/Pages/. În plus, vor exista fișiere-cheie precum App.js, care va gestiona toate rutele aplicației și index.js, care va fi responsabil cu încărcarea aplicației în browser.

Pe partea de interfață, aplicația va fi construită în React și va include o varietate de componente și pagini personalizate, cu navigare gestionată prin „react-router-dom”. Accesul global la date și funcționalități va fi realizat prin intermediul unor provideri specifici, care vor acoperi utilizatorii, coșul de cumpărături și lista de produse favorite.

Voi integra și un set diversificat de ferestre modale, care vor permite acțiuni precum editarea profilului, schimbarea parolei, plasarea comenzilor, resetarea contului sau raportarea conținutului.

Voi utiliza CSS Modules pentru a asigura izolarea stilurilor la nivel de componentă, evitând astfel suprascriderile accidentale și asigurând un control precis asupra fiecărui element din interfață. Fiecare div, buton, card, bară de filtrare sau fereastră modală va fi personalizat vizual prin clase unice și reguli CSS adaptate, contribuind la o estetică unitară și profesională a platformei.

Folderul server/ va conține, printre altele, folderul config/ cu fișierele de configurare (baza de date, autentificare, email), routes/ unde vor fi organizate toate rutele backend, și node_modules/ pentru pachetele folosite. Fișierul central va fi server.js, care va porni

serverul Express, va înregistra toate rutele aplicației, va aplica middleware-urile necesare (cum ar fi express.json, cors, passport și session) și va configura Socket.IO pentru ca aplicația să poată susține un sistem de mesagerie în timp real.

Pe partea de backend, aplicația va fi dezvoltată folosind Node.js și Express, iar baza de date va fi gestionată prin PostgreSQL. Rutele backend vor fi organizate în fișiere separate, grupate pe funcționalități, și vor acoperi toate acțiunile esențiale ale aplicației, precum autentificare, gestionarea utilizatorilor, conținut, comenzi, notificări și moderare. De asemenea, voi integra Socket.IO pentru a implementa un sistem de mesagerie în timp real.

Structura bazei de date va include mai multe tabele interconectate, necesare pentru funcționarea completă a aplicației. Printre acestea se vor regăsi: accounts, posts, marketplace_items, orders, comments, likes, favorites, messages, conversations, categories, post_reports, post_media, post_search_history și altele.

Toate datele din aceste tabele vor fi introduse manual de către mine, fără a utiliza fișiere externe sau baze de date predefinite. Conținutul de test va fi creat treptat, în mod controlat, pentru a permite validarea funcționalităților și simularea diverselor scenarii de utilizare. Pentru a evidenția vizual funcționalitățile de publicare, promovare și achiziționare a produselor, voi utiliza imagini reprezentative preluate de pe Pinterest, exclusiv în scop demonstrativ și fără intenții comerciale. Acestea vor avea rol ilustrativ și vor ajuta la înțelegerea comportamentului aplicației în context real.

În plus, toate datele sensibile ale aplicației, ca de exemplu: parolele, token-urile API, credențialele de conectare la baza de date sau la serverul de email vor fi gestionate în mod sigur prin fișiere .env, atât în frontend, cât și în backend. Aceste fișiere vor fi adăugate în .gitignore pentru a preveni urcarea lor pe GitHub și pentru a proteja aplicația împotriva accesului neautorizat. De asemenea, pentru a evita încărcarea inutilă a pachetelor externe, folderul node_modules din backend va fi exclus din controlul versiunilor. Aceste măsuri reflectă atenția acordată securității și buneii organizări a proiectului pe tot parcursul dezvoltării.

1.4 ANALIZA DOMENIULUI STATE-OF-THE-ART

Pentru o înțelegere mai clară a contextului actual în care se încadrează aplicația „CraftCircle”, este importantă analiza unor platforme deja cunoscute ce oferă funcționalități similare în sfera comerțului produselor handmade. Trei dintre cele mai relevante exemple în acest sens sunt: Etsy, Amazon Handmade și Breslo.

Tabelul 1.1. prezintă o comparație între funcționalitățile cheie ale aplicației „CraftCircle” și ale platformelor existente, evidențiind asemănările, diferențele și potențialele avantaje ale sistemului propus, care îmbină funcționalități comerciale cu elemente de rețea socială.

1.4.1 Etsy

Etsy este una dintre cele mai cunoscute platforme online dedicate vânzării produselor handmade, vintage și a materialelor creative. Platforma oferă funcționalități precum listarea produselor cu poze și descrieri, personalizarea magazinului virtual, gestionarea comenzilor și integrarea cu metode internaționale de plată.

Printre avantaje se numără interfața intuitivă, vizibilitatea globală și încrederea cumpărătorilor în brandul Etsy. De asemenea, creatorii pot oferi produse personalizate și pot primi recenzii direct de la clienți.

Totuși, există și dezavantaje: competiția este foarte mare, ceea ce face dificilă promovarea fără buget pentru reclame, iar taxele aplicate (listare, comision de vânzare, procesare plăți) pot fi împovărătoare pentru artizanii la început de drum. De asemenea, platforma este axată mai mult pe tranzacții decât pe relații de comunitate.

1.4.2 Amazon Handmade

Amazon Handmade este platforma dedicată creatorilor de produse artisanale din cadrul Amazon. Oferă funcționalități precum listarea produselor, procesarea comenzilor și livrarea rapidă prin „Fulfillment by Amazon”.

Printre avantaje se numără expunerea globală, logistica eficientă și protecția consumatorului. Cu toate acestea, dezavantajele includ un proces de înscriere mai strict, comisioane consistente și o interfață mai comercială, cu puține opțiuni de interacțiune directă între utilizatori. Accentul cade pe eficiență, nu pe comunitate.

1.4.3 Breslo

Breslo, este cea mai importantă platformă din România dedicată produselor handmade. Permite utilizatorilor să creeze magazine virtuale, să listeze produse personalizate și să vândă într-un mediu familiar și adaptat pieței locale.

Printre avantaje se numără simplitatea utilizării, apropierea culturală de publicul român și susținerea creatorilor locali. În schimb, dezavantajele includ o interfață depășită, lipsa unor actualizări moderne și o activitate comunitară redusă în comparație cu alte rețele internaționale.

Tabel 1.1 Compararea principalelor funcționalități între cele mai apreciate trei aplicații (Etsy, Amazon Handmade și Breslo) și „CraftCircle”. Funcționalitățile disponibile sunt notate cu X, iar cele indisponibile cu un –

Caracteristici	Etsy	Amazon Handmade	Breslo	„CraftCircle”
Crearea unui cont	X	X	X	X

Crearea unui magazin virtual	X	X	X	X
Listarea produselor (titlu, descriere, preț, imagini)	X	X	X	X
Gestionarea comenzilor	X	X	X	X
Recenzii și evaluări	X	X	X	-
Sistem de mesagerie cumpărător-vânzător	X	X	X	X
Sistem de mesagerie cu alți utilizatori	X	-	X	X
Interacțiune socială (like, follow, postări, comments)	-	-	-	X
Notificări în timp real	X	X	X	X
Costuri de listare și comision	X	X	X	-
Acces internațional	X	X	-	X
Dimensiune comunitară	Medie	Foarte scăzută	Scăzută-medie	Ridicată

2. TEHNOLOGII UTILIZATE ȘI BAZA TEORETICĂ

2.1 FRONTEND

2.1.1 React

React este o bibliotecă JavaScript open-source dezvoltată de Facebook în 2013, utilizată în principal pentru aplicații web de tip Single Page Application (SPA), unde interfața se actualizează dinamic fără reîncărcarea completă a paginii [1].

Elementul de bază în React este componenta – o unitate reutilizabilă a interfeței, care poate reprezenta un buton, un formular sau un întreg ecran. Componentele gestionează o stare locală (state) și comunică între ele prin props – valori transmise de la componenta părinte, nemodificabile de componenta copil, ceea ce asigură un flux de date predictibil [2].

Pentru scrierea interfeței se folosește JSX – o extensie a JavaScript-ului care permite scrierea markup-ului într-un mod similar cu HTML, direct în codul sursă [3]. React optimizează performanța prin Virtual DOM, o copie internă a DOM-ului real care este comparată cu o versiune anterioară la fiecare modificare. Diferențele sunt apoi aplicate eficient în DOM-ul real, proces numit reconciliere [4].

Controlul logicii în componente se face cu hook-uri, funcții speciale precum:

- `useState` – gestionează valorile ce se modifică în timp (ex: input-uri, liste) [5];
- `useEffect` – declanșează acțiuni după randare sau actualizare (ex: fetch API) [6];
- `useRef` – creează referințe persistente către elemente sau valori [7].

Navigarea în aplicație este gestionată cu `react-router-dom`, folosind:

- `useNavigate` – pentru redirectionarea programatică între pagini [8];
- `useLocation` – pentru acces la informații despre ruta curentă [9].

React permite utilizarea `localStorage` pentru salvarea locală a datelor (ID utilizator, preferințe) care persistă chiar și după închiderea aplicației [10].

Aplicația extinde funcționalitățile React prin biblioteci externe:

- `react-toastify` – notificări vizuale către utilizator [11];
- `react-icons` – integrarea pictogramelor SVG [12];
- `chart.js` și `react-chartjs-2` – generarea de grafice în panoul administrativ [13];
- `@stripe/react-stripe-js` și `@stripe/stripe-js` – integrarea plăților online [14];
- `socket.io-client` – comunicare în timp real, folosită în sistemul de mesagerie [15].

2.1.2 CSS Modules

CSS Modules este o tehnologie care permite aplicarea locală a stilurilor CSS, astfel încât clasele definite într-un fișier nu afectează alte părți ale aplicației. La momentul

compilării, numele claselor sunt transformate automat în variante unice, pentru a evita conflictele sau suprascrierile accidentale. [16]

Această abordare contribuie la organizarea mai clară a stilurilor și la o întreținere mai ușoară a interfeței. Este utilizată frecvent în aplicații React, fiind compatibilă nativ cu Create React App. [17]

2.1.3 Figma

Figma este un instrument de design UI/UX bazat pe cloud, conceput pentru a facilita colaborarea în timp real între membri ai unei echipe. Platforma permite proiectarea de interfețe vizuale direct în browser, fără instalare locală, și oferă funcționalități precum prototipare interactivă, biblioteci de componente reutilizabile și suport pentru designuri responsive [18].

În proiectul „CraftCircle”, Figma a fost folosit pentru schițarea paginilor de login și înregistrare, stabilind direcția vizuală inițială. Restul designului a fost construit iterativ direct în React, adaptându-se pe parcursul dezvoltării.

2.1.4 Servicii API externe

Pentru a face completarea adreselor cât mai simplă și precisă, aplicația „CraftCircle” integrează mai multe servicii externe care oferă informații actualizate despre țări, orașe, coduri poștale și distanțe geografice. Scopul este să oferim o experiență cât mai comodă pentru utilizatori, mai ales în procesul de checkout.

Pe partea de frontend, cererile către aceste API-uri sunt gestionate cu ajutorul bibliotecii axios, care este un client HTTP foarte popular, folosit pentru a trimite și primi date din aplicațiile React [19]. De exemplu, lista țărilor este obținută printr-un API public dedicat [20]. După selectarea unei țări, aplicația trimite o cerere POST către un serviciu extern care returnează lista orașelor corespunzătoare. Acest API necesită transmiterea numelui țării în corpul cererii, în format JSON, și răspunde doar la solicitări de tip POST [21]. Validarea codurilor poștale se face printr-o combinație între reguli proprii (de exemplu, pentru România) și un API care verifică dacă un cod poștal există în țara aleasă [22].

Pentru transport, aplicația calculează costul livrării în funcție de distanța dintre vânzător și cumpărător. Acest lucru se realizează cu ajutorul OpenRouteService, un serviciu care oferă calcule de distanță rutieră. Astfel, în funcție de locații și de valoarea comenzii, costul de livrare poate varia, iar comenzile mai mari pot beneficia chiar de livrare gratuită [23].

2.2 BACKEND

2.2.1 Node.js și Express

Node.js este un mediu de execuție JavaScript modern, creat pentru a permite rularea codului JavaScript în afara unui browser web. Lansat în 2009 și construit pe motorul

V8 al browserului Chrome, acesta a devenit rapid o alegere populară pentru dezvoltatorii care doresc să creeze aplicații web rapide și scalabile. Spre deosebire de alte tehnologii server-side, Node.js funcționează pe baza unui model asincron, bazat pe evenimente, ceea ce îl face ideal pentru gestionarea unui număr mare de cereri în paralel, fără blocaje. [24]

Unul dintre marile avantaje ale folosirii Node.js este faptul că dezvoltatorii pot lucra cu un singur limbaj – JavaScript – atât pentru frontend, cât și pentru backend. De asemenea, comunitatea din jurul său este foarte activă, iar prin intermediul managerului de pachete npm, este ușor să adaugi rapid funcționalități noi în aplicație. [25]

Pentru a simplifica munca cu Node.js, a fost creat Express – un framework minimalist care face definirea rutelor, trimiterea răspunsurilor către client și lucrul cu date mult mai ușoare. Express este construit direct peste Node.js și oferă o structură clară, suport pentru middleware-uri și integrare rapidă cu baze de date sau alte servicii externe. [26]

În cadrul proiectului „CraftCircle”, Express este utilizat pentru definirea rutelor backend-ului, pentru trimiterea răspunsurilor către frontend și pentru conectarea la baza de date. Este practic „coloana vertebrală” a logicii serverului.

Pentru a extinde funcționalitățile de bază, aplicația folosește mai multe biblioteci externe, instalate prin npm. Printre cele mai importante se numără:

- dotenv, care permite salvarea variabilelor sensibile într-un fișier .env, util
- pentru protejarea informațiilor de tip cheie API [27];
- body-parser, care ajută serverul să înțeleagă datele transmise în format JSON[28];
- cors, care permite aplicației frontend să comunice cu backend-ul chiar și de pe un domeniu diferit [29];
- nodemailer, utilizat pentru trimiterea emailurilor (de exemplu, confirmări de comandă sau resetări de parolă) [30];
- multer, folosit pentru încărcarea fișierelor (precum imagini) [31];
- passport, împreună cu passport-facebook și passport-google-oauth20, care oferă login prin conturi sociale [32];
- pg și pg-pool, biblioteci care facilitează conectarea la baza de date PostgreSQL și executarea interogărilor [33];
- stripe, folosit pentru implementarea plăților online prin card [34].

Aceste tehnologii au fost alese pentru că s-au potrivit perfect cu nevoile aplicației „CraftCircle”, oferind suport pentru funcții esențiale ca autentificarea, plățile și mesageria.

2.2.2 Cloudinary

Cloudinary este o platformă cloud pentru gestionarea fișierelor media (imagini, videoclipuri), oferind funcții de încărcare, optimizare, transformare și livrare rapidă prin CDN [35].

În „CraftCircle”, Cloudinary este folosit pentru stocarea imaginilor de profil, ale produselor și postărilor. După upload, fiecare fișier primește un link optimizat, salvat în baza de date și folosit în interfață [36].

Platforma permite și ștergerea automată a fișierelor media atunci când nu mai sunt necesare, contribuind la menținerea unui spațiu de stocare curat și eficient.

2.2.3 Passport.js

Passport.js este o bibliotecă middleware pentru Node.js, folosită în special pentru integrarea unor metode de autentificare externe, precum login-ul cu Google sau Facebook. Baza sa modulară, construită pe un sistem de strategii, permite adăugarea rapidă a diferitelor servicii de autentificare, fără a complica logica principală a aplicației. [32]

În cadrul aplicației „CraftCircle”, Passport.js a fost utilizat exclusiv pentru autentificarea socială. Integrarea cu Google și Facebook a permis utilizatorilor să se conecteze fără a crea manual un cont, deoarece datele de profil au fost preluate automat. Astfel, procesul de înregistrare este simplificat, iar accesul în platformă devine mai rapid și mai convenabil pentru utilizatori.

Pentru autentificarea clasică, prin email și parolă, a fost implementată o soluție personalizată. Parolele sunt criptate cu ajutorul bibliotecii bcrypt [37], iar verificarea autentificării se face prin interogări directe în baza de date, fără a folosi Passport.js.

2.2.4 Socket.IO

Socket.IO este o bibliotecă care permite trimiterea și primirea de date în timp real între client și server, printr-o conexiune bidirecțională. Această tehnologie este utilă mai ales în aplicații care necesită actualizări rapide, fără reîncărcarea paginii, cum ar fi chat-urile, sistemele de notificări sau aplicațiile colaborative. [38]

În aplicația „CraftCircle”, Socket.IO este folosit pentru a trimite notificări în timp real atunci când un utilizator primește un like, un follow sau un comentariu, dar și pentru funcționalitatea de mesagerie. Astfel, interacțiunile din aplicație se reflectă imediat în interfața utilizatorului, fără întârziere. Fiecare utilizator conectat este asociat cu un canal propriu, iar serverul trimite notificările doar către destinatarul corect, exact în momentul în care are loc acțiunea.

Această integrare aduce un plus de interactivitate și face ca aplicația să se simtă mai rapidă și mai dinamică. Comparativ cu metodele clasice de verificare a noutăților (cum ar fi reîncărcarea sau solicitările repetate), Socket.IO oferă o experiență mai fluidă și mai apropiată de ceea ce utilizatorii așteaptă de la o platformă de actualitate.

2.2.5 Stripe

Pentru gestionarea plăților online în aplicația „CraftCircle”, a fost integrată soluția Stripe, o platformă cunoscută pentru securitatea ridicată, fiabilitate și ușurință de integrare în aplicații web moderne. [39] Utilizatorii pot achita comenzile direct cu cardul, fără a fi redirecționați către alte site-uri externe. Colectarea datelor de plată se face în siguranță, prin componentele furnizate de Stripe pentru frontend. [14]

Procesul de plată este împărțit în două etape: în primul rând, backend-ul creează un payment intent care definește suma de plată și moneda; în pasul următor, utilizatorul introduce datele cardului, iar Stripe validează și confirmă tranzacția. Datele sensibile nu sunt accesate sau stocate de aplicație, ci sunt transmise direct către infrastructura Stripe prin conexiuni criptate. [40]

Această abordare contribuie la o experiență de cumpărare eficientă și sigură, specifică platformelor comerciale moderne. În plus, aplicația oferă și opțiunea de plată ramburs, acoperind preferințele utilizatorilor care aleg metode tradiționale de plată.

2.2.6 Nodemailer

În cadrul aplicației „CraftCircle”, trimiterea emailurilor automate este realizată cu ajutorul bibliotecii Nodemailer, o soluție stabilă și ușor de configurat pentru comunicarea prin SMTP. Cu ajutorul acesteia, aplicația poate expedia mesaje personalizate direct din backend, folosind un cont Gmail setat în fișierul de configurare. [30]

Funcționalitatea este integrată în mai multe puncte ale aplicației: utilizatorii primesc emailuri de confirmare la plasarea comenzilor, notificări la schimbarea parolei sau mesaje informative atunci când o postare sau un produs este șters de către administrator. Aceste mesaje contribuie la o experiență transparentă și profesională.

Configurarea Nodemailer a fost realizată printr-un fișier dedicat în backend, care definește contul expeditorului și gestionează trimiterea emailurilor. Este o soluție simplă, dar eficientă, ce contribuie la profesionalismul și claritatea aplicației în interacțiunea cu utilizatorii.

2.3 BAZA DE DATE

2.3.1 PostgreSQL

PostgreSQL este un sistem de baze de date relaționale open-source, cunoscut pentru stabilitatea și flexibilitatea sa. Este dezvoltat activ încă din 1996 și este folosit pe scară largă în mai multe proiecte, atât persoanele cât și academice. [41]

Față de alte soluții, PostgreSQL vine cu suport pentru tipuri de date complexe, relații între tabele, tranzacții și reguli avansate de validare, ceea ce îl face potrivit în special pentru aplicații care lucrează cu date structurate și în care consistența este importantă. [42]

În cadrul aplicației „CraftCircle”, PostgreSQL gestionează toate informațiile importante: conturile utilizatorilor, postările, produsele, comenzile, comentariile, mesajele și notificările. Structura relațională a bazei de date a permis definirea unor relații clare între entități, prin utilizarea cheilor primare (pentru identificarea unică a fiecărui rând) și a cheilor externe (pentru legarea logică între tabele, cum ar fi legătura dintre un comentariu și postarea la care a fost adăugat).

Pentru păstrarea consistenței datelor în momentul ștergerii, s-au folosit constrângeri de tip ON DELETE CASCADE. Astfel, atunci când un utilizator este șters, toate datele

asociate acestuia (postări, comentarii, mesaje, produse, etc.) sunt eliminate automat, asigurându-se că nu rămân informații neactualizate sau inutile în baza de date .

Totodată, s-au definit constrângeri UNIQUE pentru câmpuri care trebuie să fie unice, cum ar fi adresele de email sau ID-urile comenzilor, și constrângeri CHECK pentru a impune reguli simple asupra valorilor (de exemplu, statusul comenzii să fie doar „pending”, „confirmed” sau „delivered”) [43].

Conectarea aplicației la baza de date se face prin pachetul pg, parte din ecosistemul Node.js. Acesta permite trimiterea de interogări SQL și prelucrarea răspunsurilor într-un mod eficient și direct. [44]

PostgreSQL a fost o alegere inspirată pentru această aplicație, datorită fiabilității și modului în care se pot organiza clar și sigur toate datele aplicației.

2.3.2 pgAdmin

Pentru gestionarea bazei de date PostgreSQL în timpul dezvoltării aplicației „CraftCircle” s-a folosit pgAdmin — una dintre cele mai populare platforme de administrare pentru PostgreSQL, compatibilă cu sistemele Linux, macOS, Windows și Unix. Interfața sa prietenoasă permite rularea de interogări SQL, monitorizarea activității bazei de date și navigarea rapidă între tabele, relații și vizualizări [45].

Acest instrument grafic a fost util în timpul testării, oferind o modalitate rapidă de a inspecta structura bazei de date, relațiile dintre tabele și răspunsurile la interogări.

2.3.3 Postman

Pe lângă utilizarea interfeței grafice pgAdmin, în procesul de testare a interacțiunilor cu baza de date s-a folosit și Postman — un instrument dedicat testării API-urilor. Acesta a permis trimiterea de cereri HTTP către backend-ul aplicației și verificarea răspunsurilor generate de server în urma interogărilor SQL. Cu ajutorul Postman au fost testate operații precum autentificarea utilizatorilor, trimiterea comentariilor, plasarea comenzilor sau actualizarea statusului acestora, fără a fi nevoie de interfața frontend. Acest flux a contribuit la o dezvoltare mai rapidă și la depistarea timpurie a erorilor în logica de comunicare cu PostgreSQL. [46]

2.4 CONTROL VERSIUNI

2.4.1 Git

Git a fost utilizat în cadrul proiectului „CraftCircle” ca instrument de control al versiunilor, având rolul de a salva evoluția aplicației pe măsură ce erau finalizate diferite funcționalități. De fiecare dată când o componentă devenea stabilă și funcțională, aceasta

era adăugată într-un commit, oferind astfel un punct de restaurare sigur în caz de erori sau regresii ulterioare.

Având în vedere că proiectul a fost realizat individual, nu s-a ales utilizarea ramurilor (branches), iar commit-urile au fost gestionate pe linia principală a dezvoltării, permițând păstrarea istoricului modificărilor și facilitând revenirea rapidă la versiuni anterioare. Toate commit-urile au fost centralizate într-un repository online, asigurând astfel și un backup extern al întregului proiect.

Această abordare, simplă dar eficientă, s-a dovedit potrivită pentru un proiect individual, oferind siguranță și transparență în gestionarea codului sursă.

3. PROIECTAREA SISTEMULUI

3.1 PROIECTAREA GENERALĂ A APLICAȚIEI

Pentru a asigura o dezvoltare clară, scalabilă și ușor de întreținut, aplicația „CraftCircle” a fost organizată în module distincte, fiecare având un rol bine definit. Structura arhitecturală ([Figura 3.1](#)) este împărțită în patru componente majore: interfața vizibilă pentru utilizator (frontend), logica de procesare de pe server (backend), baza de date relațională și un set de servicii externe integrate. Pe parcursul procesului de dezvoltare au fost utilizate și instrumente de asistență bazate pe inteligență artificială, precum ChatGPT, pentru clarificarea unor erori de implementare (buguri), optimizarea structurii relațiilor dintre tabele, obținerea de sugestii privind îmbunătățirea interfeței utilizatorului (UI) în direcția unei experiențe mai prietenoase, precum și pentru redactarea documentației într-o manieră mai clară și mai elevată.

3.1.1 Frontend-ul aplicației (React)

Partea de frontend, dezvoltată cu React, este responsabilă de interacțiunea directă cu utilizatorul. Aceasta cuprinde mai multe secțiuni funcționale:

Autentificare – permite înregistrarea și conectarea utilizatorilor. Odată autentificați, aceștia pot accesa toate funcționalitățile platformei.

Feed postări și produse – utilizatorii pot vizualiza conținutul publicat de alții.

Shop-ul – oferă funcționalități pentru listarea, adăugarea sau modificarea produselor, fiecare produs fiind ilustrat printr-un carusel de imagini și asociat cu un status de stoc.

Coșul de cumpărături și checkout-ul – permit adăugarea produselor, calcularea prețului total (inclusiv costul transportului) și lansarea comenzii, cu opțiuni de plată ramburs sau online.

Mesajele – oferă un sistem de comunicare directă între utilizatori, cu actualizare în timp real prin WebSocket.

Notificările – asigură informarea în timp real cu privire la interacțiuni precum aprecieri, urmăritori noi sau comentarii.

Profilul utilizatorului – permite modificarea informațiilor personale, actualizarea biografiei și afișarea postărilor și produselor proprii.

Interfața de administrare – este dedicată moderatorilor și oferă funcții de gestionare a postărilor, utilizatorilor, produselor și statisticilor.

Pentru stilizarea componentelor, aplicația utilizează CSS Modules, o tehnologie care izolează clasele CSS la nivel de componentă, prevenind conflictele între stiluri și facilitând organizarea codului vizual.

3.1.2 Backend-ul aplicației (Node.js + Express)

Pe partea de backend, aplicația folosește un server Node.js împreună cu Express pentru a gestiona cererile venite din frontend și pentru a interacționa cu baza de date și serviciile externe. Logica este împărțită pe module specializate:

Autentificare – înregistrează și autentifică utilizatorii, gestionând sesiuni sigure.

Postări – gestionează crearea, modificarea și ștergerea postărilor, alături de comentarii și aprecieri.

Shop – administrează produsele, imaginile aferente și detaliile asociate.

Coș de cumpărături și comenzi – reține produsele adăugate, cantitățile și comenzile efectuate, iar în cazul plăților cu cardul se conectează la serviciul Stripe.

Mesaje – gestionează conversațiile și trimiterea mesajelor în timp real prin Socket.IO.

Notificări – generează notificări relevante și le transmite către frontend.

Administrare – oferă posibilitatea de a șterge conținut raportat sau conturi și de a consulta statistici.

3.1.3 Baza de date (PostgreSQL)

Baza de date utilizată este PostgreSQL, un sistem relațional robust care stochează toate datele aplicației: conturi, postări, produse, comenzi, mesaje, comentarii, rapoarte, notificări și istoricul căutărilor etc.

3.1.4 Servicii externe

Pentru a extinde funcționalitățile aplicației și a simplifica anumite procese, sunt integrate mai multe servicii externe:

Cloudinary – pentru stocarea și livrarea rapidă a imaginilor de profil, produse sau postări.

Nodemailer – pentru trimiterea automată de emailuri către utilizatori.

Stripe – pentru procesarea plăților online, în condiții de siguranță.

OpenRouteService – pentru estimarea costului de livrare în funcție de distanță.

API-uri de autocompletare – pentru completarea țării, orașului sau codului poștal în formularul de comandă.

Socket.IO – pentru actualizarea în timp real a mesajelor și notificărilor.

Această arhitectură modulară asigură o separare clară între componente, susține scalabilitatea aplicației și facilitează atât dezvoltarea de noi funcționalități, cât și mentenanța pe termen lung.

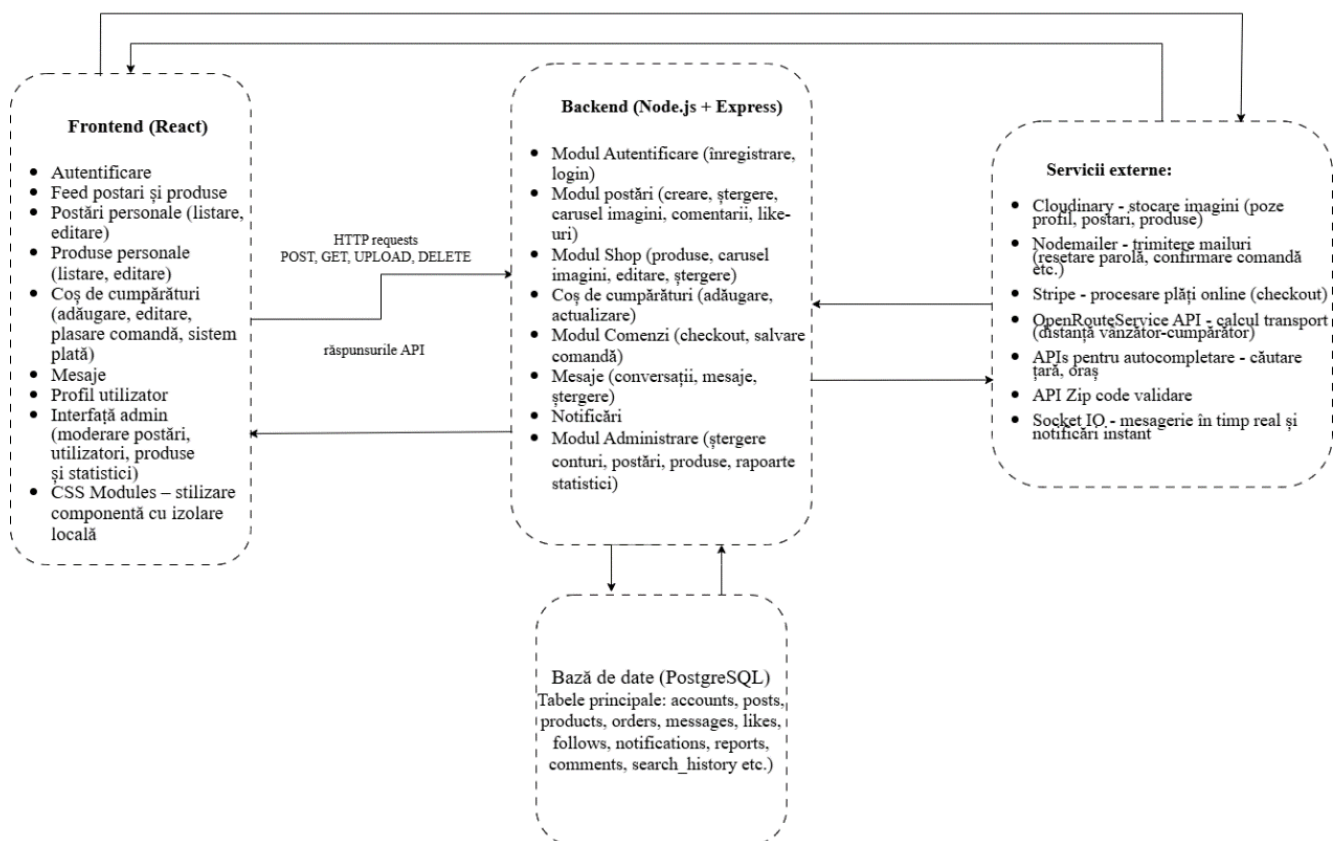


Figura 3.1 Arhitectura Aplicației

3.2 DIAGRAMA UML A CAZURILOR DE UTILIZARE

Diagrama cazurilor de utilizare (Use Case Diagram) are rolul de a evidenția modul în care diferitele tipuri de utilizatori interacționează cu aplicația, punând accent pe funcționalitățile accesibile pentru fiecare actor. Această reprezentare ajută la înțelegerea structurii aplicației dintr-o perspectivă practică – ce poate face utilizatorul și cum răspunde sistemul la acțiunile sale.

În cadrul aplicației „CraftCircle”, se disting două roluri principale: utilizatorul obișnuit și administratorul.

- Utilizatorul este cel care interacționează direct cu interfața aplicației pentru a crea conținut, a publica produse, a comunica cu alți utilizatori și a plasa comenzi. De asemenea, poate da like, comenta, urmări alți utilizatori, salva produse la favorite și primi notificări în timp real. De asemenea poate efectua activități comerciale specifice, precum adăugarea în coș, alegerea metodei de plată și vizualizarea propriilor comenzi.
- Administratorul, în schimb, are acces la o interfață separată, de tip panou de control, unde poate modera activitatea utilizatorilor: poate vizualiza și șterge postări sau produse raportate, poate gestiona conturi și poate analiza statistici despre utilizarea aplicației.

Fiecare funcționalitate principală a fost menționată în diagramă sub formă de „caz de utilizare”, iar conexiunile dintre actori și acțiuni arată pașii reali pe care aceștia îi urmează în aplicație. Pentru a evita aglomerarea elementelor, diagrama a fost împărțită în două componente: una dedicată interacțiunilor utilizatorului obișnuit ([Figura 3.2](#)) și una dedicată rolului de administrator ([Figura 3.3](#)).

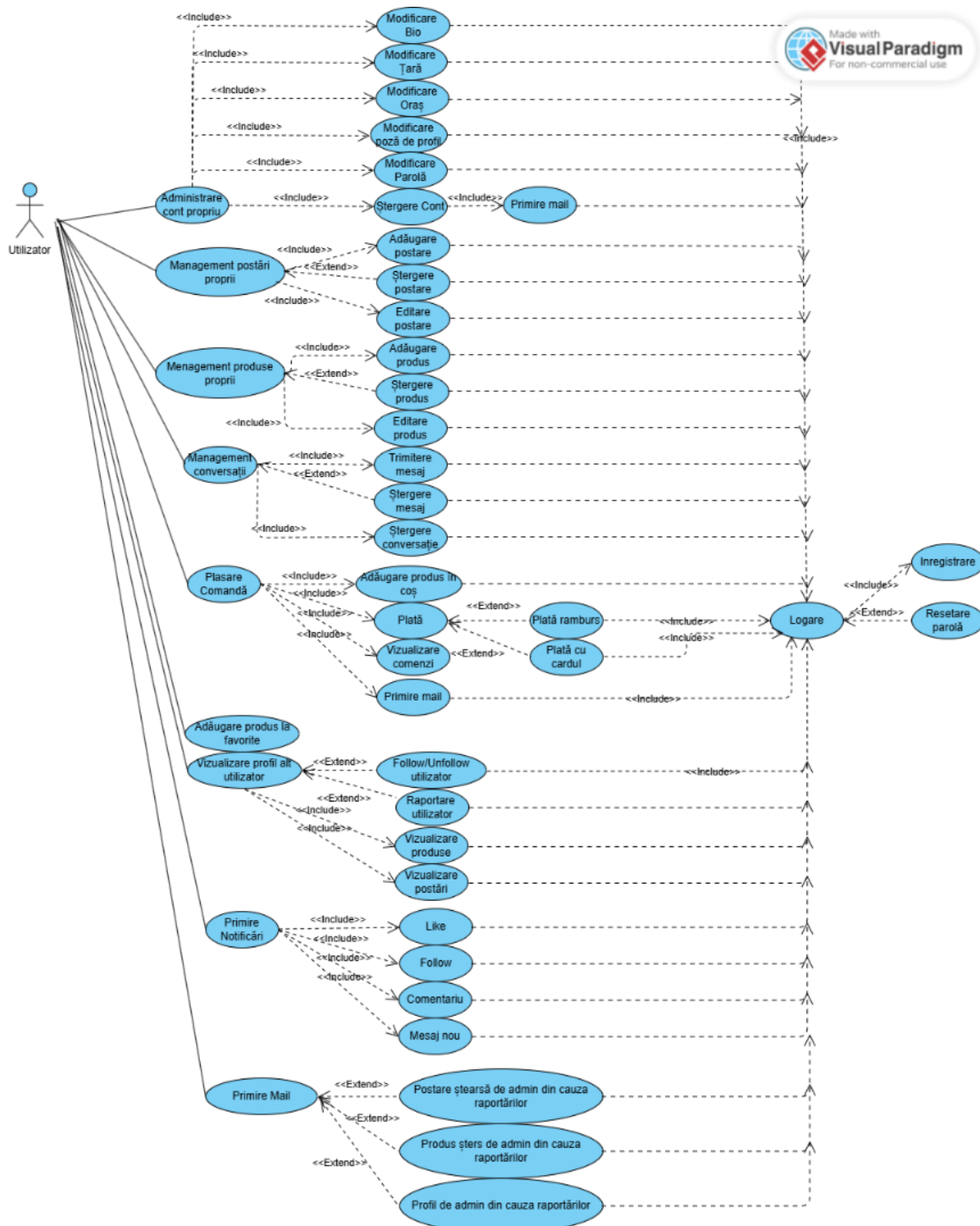


Figura 3.2 Diagrama UML a cazurilor de utilizare pentru utilizator – Aplicația „CraftCircle”

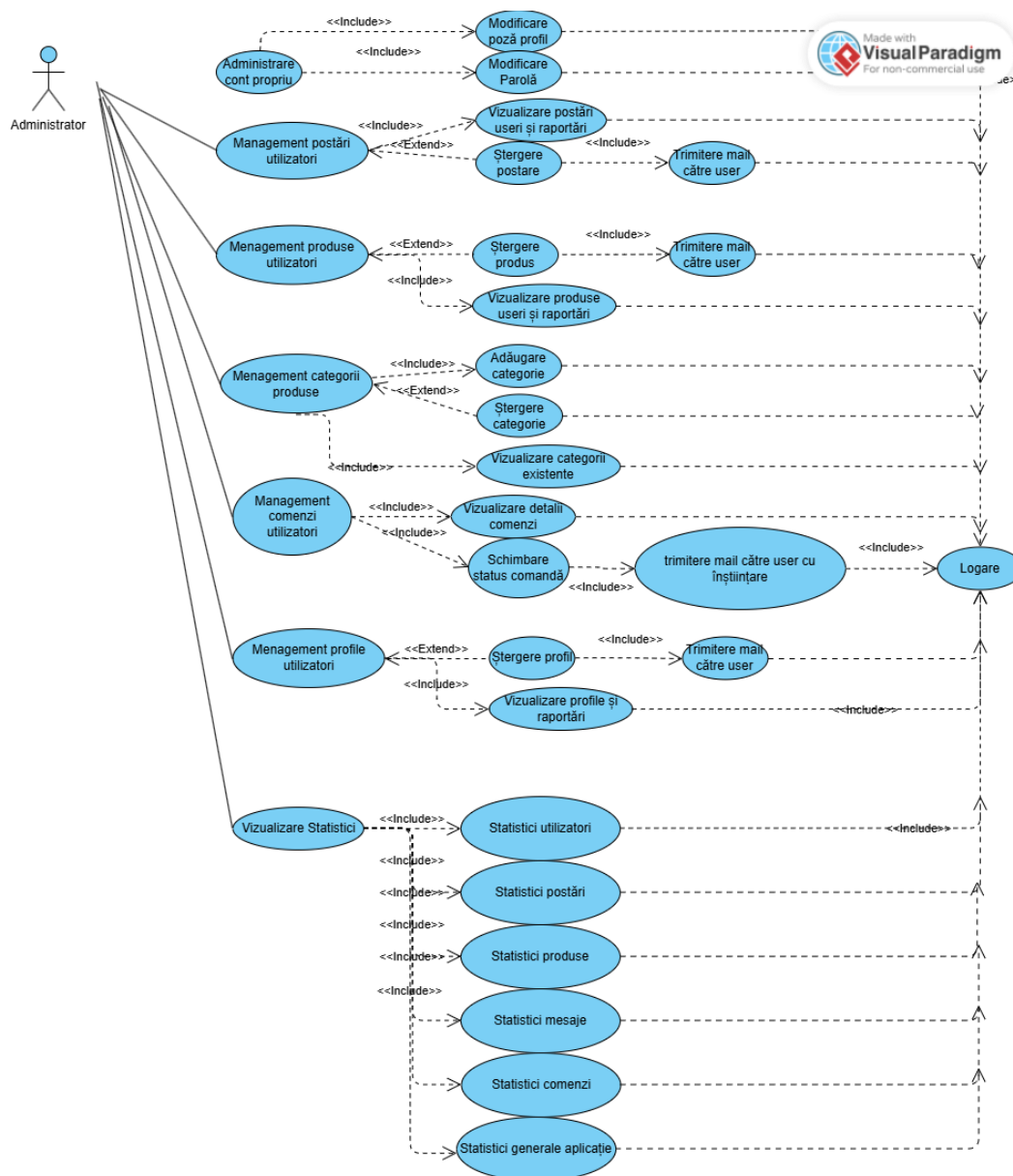


Figura 3.3 Diagrama UML a cazurilor de utilizare pentru administrator

3.3 NAVIGAREA ÎN APLICAȚIE

Accesul în aplicația „CraftCircle” începe cu procesul de înregistrare sau autentificare, în funcție de situație.

3.3.1 Înregistrare și autentificare

Pentru a utiliza aplicația, utilizatorul normal trebuie mai întâi să își creeze un cont. Acest lucru se poate face prin:

- Formular manual, unde utilizatorul introduce adresa de email, o parolă și informații suplimentare precum țara, orașul și o descriere personală (bio). După completarea acestor date, contul este creat și utilizatorul este direcționat către pagina de profil, „Home” ([Figura 3.6](#)).
- Autentificare socială, folosind conturile Google sau Facebook. În acest caz, numele și emailul, și poza sunt preluate automat, iar utilizatorul completează doar informațiile adiționale menționate anterior ([Figura 3.6](#)).

Administratorul, în schimb, are deja contul creat în prealabil și se autentifică doar folosind emailul și parola alocate ([Figura 3.7](#)).

3.3.2 Redirecționare după logare

Utilizatorul obișnuit este direcționat către propria pagină de profil, unde își poate vedea postările, produsele din shop, numărul de followers, conturile urmărite. De aici are acces și la bara principală de navigare ([Figura 3.6](#)).

Administratorul este direcționat către pagina sa de profil administrativ, care afișează informații personale și o secțiune cu statistici despre activitatea sa (ex. câți utilizatori, postări și produse a șters). De aici poate accesa toate funcțiile de moderare folosind bara de navigare ([Figura 3.7](#)).

3.3.3 Bara de navigare

Pentru utilizatorul normal, bara de navigare este vizibilă în partea superioară a ecranului și conține următoarele secțiuni ([Figura 3.4](#)):

- „Discover” – explorează postări și produse de la alți utilizatori.
- „Home” – redirecționează către pagina de profil.
- „Appreciation” – afișează notificările legate de like-uri, follow-uri și comentarii.
- „Messages” – acces către conversații în timp real.
- Pictogramă ce reprezintă o inimă – lista de produse favorite.
- Pictogramă ce reprezintă un coș – coșul de cumpărături.
- „Orders” – vizualizarea comenzilor proprii.

În partea dreaptă a barei de navigare, utilizatorul are acces la un meniu derulant, marcat printr-o pictogramă de meniu (≡). Acesta oferă acces rapid la acțiuni legate de contul personal, precum:

- „Change Password” – deschide un modal pentru schimbarea parolei;
- „Edit Profile” – permite actualizarea bio-ului, țării și orașului;
- „Contact” – oferă datele de contact pentru suport;
- „Delete Account” – permite ștergerea contului curent;
- „Logout” – deloghează utilizatorul din aplicație.

Aceste opțiuni sunt implementate sub formă de ferestre modale, astfel încât acțiunile să poată fi efectuate fără a părăsi pagina curentă.

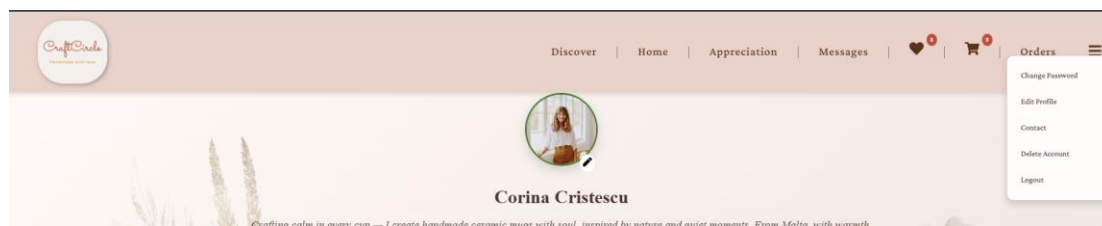


Figura 3.4 Bara de navigare pentru utilizatorul aplicației „CraftCircle”

Pentru administrator, bara de navigare este adaptată rolului său și include următoarele secțiuni (Figura 3.5):

- „Dashboard” – statistică generală și activitate personală.
- „Categories” – adăugare/ștergere/modificare categorii.
- „Moderate Users” – gestionarea conturi existente.
- „Moderate Posts” – moderarea postărilor.
- „Moderate Products” – moderarea produselor.
- „Orders” – vizualizarea și schimbarea statusului comenzilor.
- „Statistics” – acces la statistici globale ale aplicației.

În plus, în partea dreaptă sus a ecranului, administratorul are acces la un meniu drop-down discret, reprezentat printr-o pictogramă de meniu (≡). Acesta conține acțiuni legate de contul personal:

- „Change Password” – deschide un modal pentru schimbarea parolei contului.
- „Logout” – permite delogarea imediată din platformă.

Spre deosebire de utilizatorul obișnuit, administratorul are un meniu simplificat, deoarece nu i se permite editarea profilului sau ștergerea contului, aceste acțiuni fiind irelevante pentru rolul său.



Figura 3.5 Bara de navigare pentru administratorul aplicației „CraftCircle”

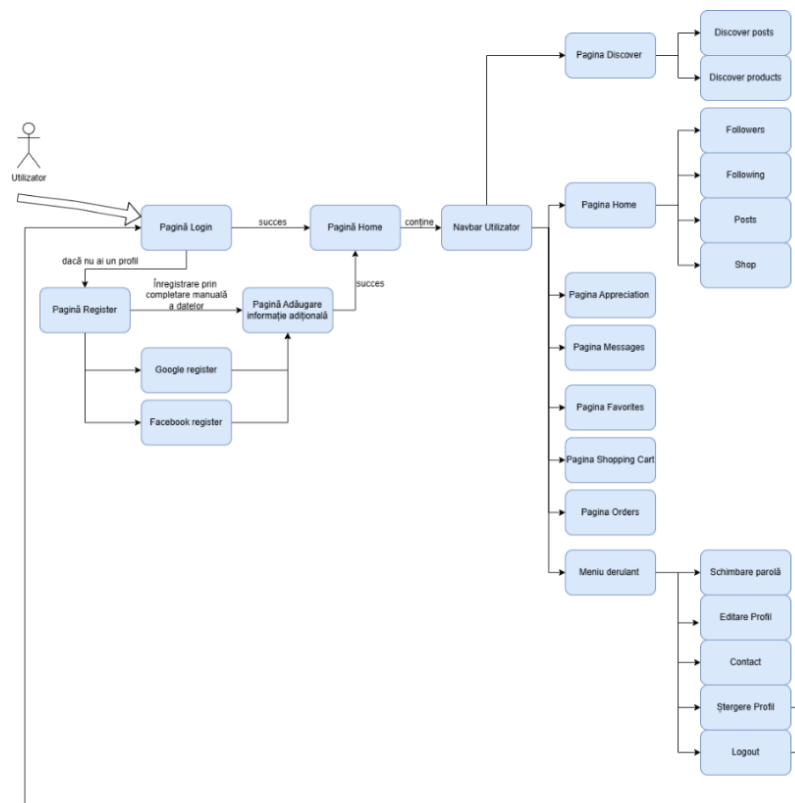


Figura 3.6 Diagrama de navigare a utilizatorului normal

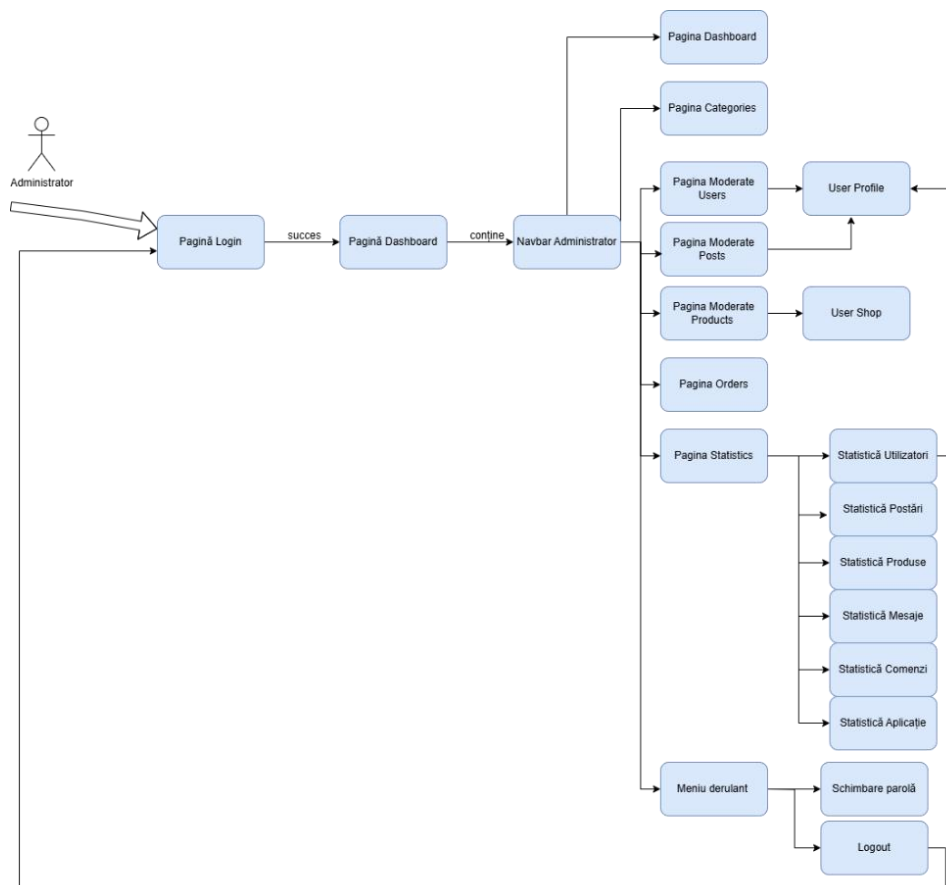


Figura 3.7 Diagrama de navigare a utilizatorului administrator

3.4 PROIECTAREA BAZEI DE DATE

Pentru gestionarea datelor aplicației „CraftCircle” s-a utilizat un sistem de baze de date relațional, implementat în PostgreSQL. Structura bazei de date reflectă arhitectura modulară a aplicației, fiind organizată în jurul principalelor funcționalități.

Tabelele sunt grupate astfel:

- Conturi și autentificare:
 - accounts: conține informațiile utilizatorilor (ID, email, parolă, nume, prenume, data nașterii, poză, locație, descrierea și data creării contului.
 - admins: datele administratorilor cu privilegii extinse.
- Conținut social și interacțiuni:
 - posts, post_media: gestionează postările și fișierele media atașate.
 - likes, comments: permit reacții și comentarii.
 - followers: rețea de tip follow/unfollow.
 - appreciation_notifications: gestionează notificările legate de aprecieri (like, follow, comentarii). Tabela conține câmpul boolean „seen”, care indică dacă notificare a fost citită de utilizator(mai exact, dacă a fost accesată pagina „Appreciation” de când a fost primită notificarea).
 - post_views, post_search_history, product_views, product_search_history : activitatea utilizatorilor, folosită pentru personalizarea conținutului.
- Funcționalități comerciale:
 - marketplace_items: produse listate de utilizatori.
 - shop_item_images: imaginile asociate produselor.
 - shopping_cart: conține articolele adăugate în coș.
 - orders: salvează comenzile, cu detalii despre livrare și plată.
 - favorites: lista produselor favorite.
- Mesagerie:
 - conversations: descrie fiecare conversație activă.
 - messages: stochează mesajele schimbate între utilizatori.
- Raportări:
 - user_reports, post_reports, product_reports: raportări ale utilizatorilor.
- Administrare:
 - câmpuri precum deleted_users, deleted_posts, deleted_products
 - categories: definește categoriile disponibile în aplicație.

Legăturile dintre entități sunt realizate prin chei externe (foreign keys), care asigură integritatea referențială. De exemplu, fiecare post este asociat cu un user_id, comenzile sunt legate de buyer_id, seller_id și item_id, iar comentariile sunt asociate cu post_id.

Diagrama completă a bazei de date este prezentă [Figura 3.8](#), fiind generată automat în pgAdmin pe baza structurii actuale a aplicației.

Oxani Corina

„CraftCircle” - Aplicație web dedicată promovării, vânzării și schimbului de idei între pasionații de produse handmade

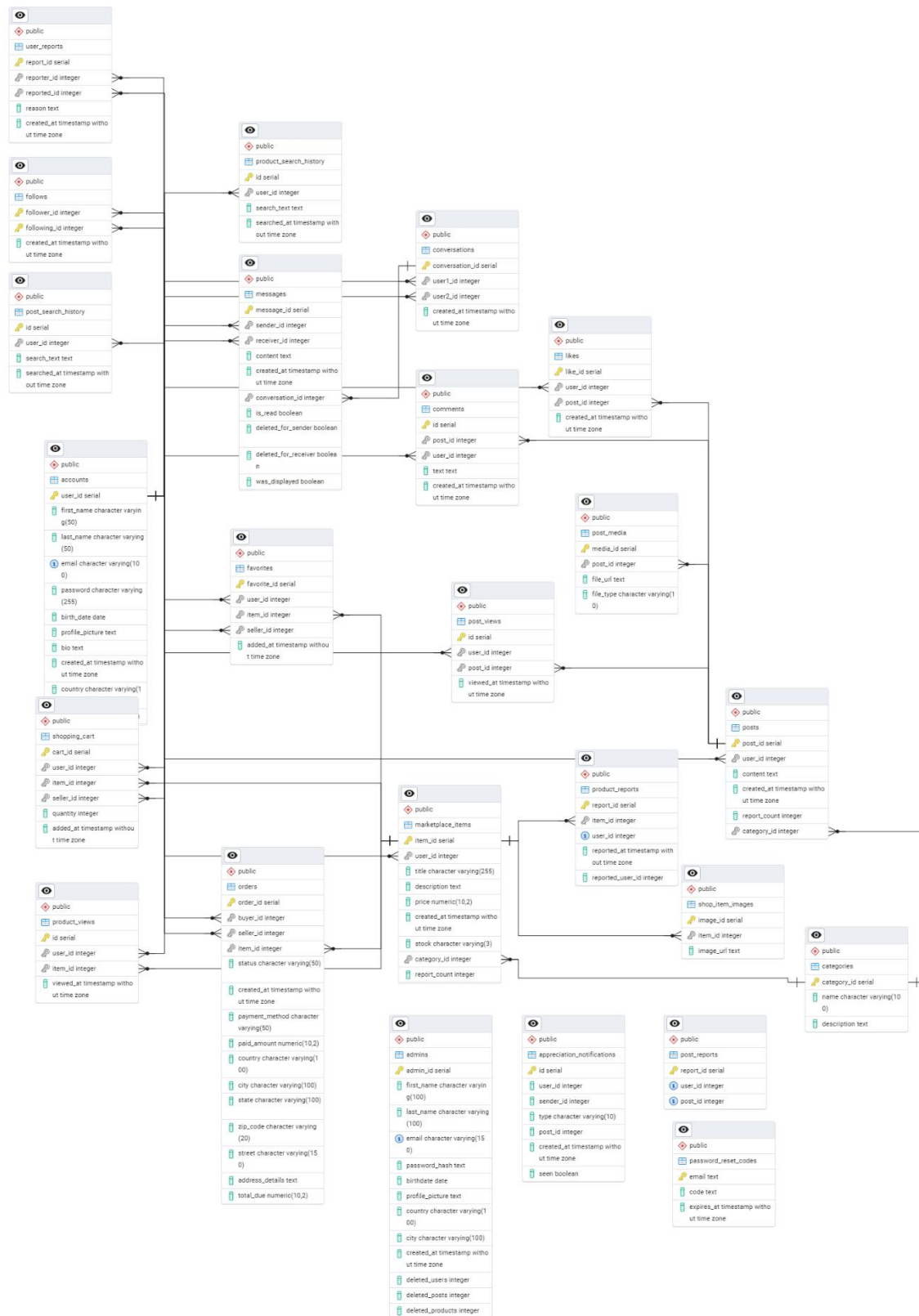


Figura 3.8 Diagrama relațională a bazei de date „CraftCircle” generată în pgAdmin (include toate entitățile, relațiile și atributele esențiale pentru funcționarea completă a aplicației)

4. IMPLEMETAREA FUNCȚIONALITĂȚILOR

Înainte de a trece la explicarea funcționalităților, este utilă o scurtă prezentare a fișierelor-cheie care definesc structura aplicației, atât pe partea de client, cât și pe cea de server.

În partea de frontend, fișierul `index.js` reprezintă punctul de intrare al aplicației React. Aici este montată în DOM componenta principală – `App.js`, folosindu-se `React.StrictMode` pentru a semnaliza potențiale probleme în timpul dezvoltării.

```
client > src > JS index.js > ...
1 import React from 'react';
2 import ReactDOM from 'react-dom/client';
3 import './index.css';
4 import App from './App';
5
6 const root = ReactDOM.createRoot(document.getElementById('root'));
7 root.render(
8   <React.StrictMode>
9     <App />
10  </React.StrictMode>
11 );
12
13
```

Secvență de cod 4.1 Punctul de intrare în aplicația React (`index.js`)

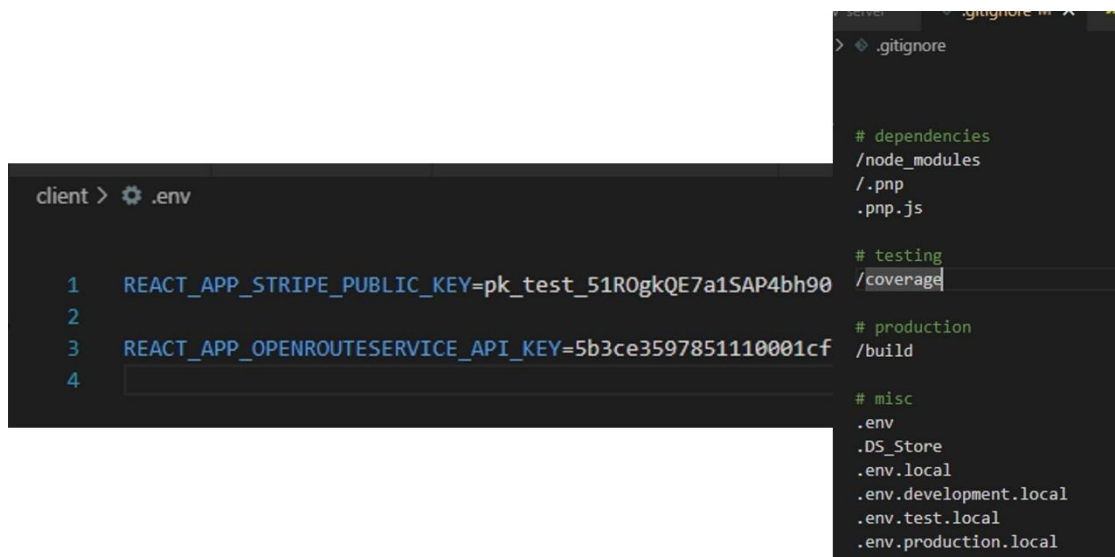
Fișierul `App.js` joacă rolul central în definirea comportamentului vizual și logic al aplicației. Aici sunt configurate toate rutele aplicației, atât pentru utilizatori obișnuiți (precum paginile de profil, magazin, coș de cumpărături, mesaje etc.), cât și pentru administratori (statistici, moderare, gestiune comenzi sau utilizatori). Tot în acest fișier sunt integrați provideri globali (`UserProvider`, `CartProvider`, `FavoritesProvider`, `ToastProvider`) care facilitează partajarea stării și a datelor între componente. În plus, funcția `ScrollToTop` asigură revenirea la începutul paginii la fiecare navigare, pentru o experiență de utilizare optimă.

```
37 function ScrollToTop() {
38   const { pathname } = useLocation();
39   useEffect(() => {
40     window.scrollTo(0, 0);
41   }, [pathname]);
42   return null;
43 }
44
45 function App() {
46   return (
47     <Router>
48       <UserProvider>
49         <CartProvider>
50           <FavoritesProvider>
51             <ToastProvider>
52               <ScrollToTop />
53               <Routes>
54                 <Route path="/" element={<Navigate to="/register" /> } />
55                 <Route path="/register" element={<Register /> } />
56                 <Route path="/login" element={<Login /> } />
57                 <Route path="/auth-success" element={<AuthRedirect /> } />
58                 <Route path="/additional-info" element={<AdditionalInfoForm /> } />
59                 <Route path="/profile" element={<UserProfile /> } />
60                 <Route path="/profile/:userId" element={<UserProfile /> } />
61                 <Route path="/shop" element={<ShopPage /> } />
62                 <Route path="/profile/:userId/shop" element={<ShopPage /> } />

```

Secvență de cod 4.2 Funcția `ScrollToTop()` și definirea rutelor (`App.js`)

În ceea ce privește gestionarea cheilor și a configurărilor externe, aplicația frontend folosește un fișier `.env` propriu, în care sunt definite variabile precum cheia publică Stripe (`REACT_APP_STRIPE_PUBLIC_KEY`) sau cheia API de la OpenRouteService. Acestea sunt accesate prin `process.env`, iar pentru ca React să le recunoască, este necesar prefixul `REACT_APP_`. Fișierul `.env` nu este urcat în GitHub deoarece este inclus în `.gitignore`, contribuind astfel la securitatea datelor sensibile.



The image shows a code editor with two files open. The left pane shows the `.env` file with the following content:

```
1 REACT_APP_STRIPE_PUBLIC_KEY=pk_test_51R0gkQE7a1SAP4bh90
2
3 REACT_APP_OPENROUTESERVICE_API_KEY=5b3ce3597851110001cf
4
```

The right pane shows the `.gitignore` file with the following content:

```
# dependencies
/node_modules
/.pnp
/.pnp.js

# testing
/coverage

# production
/build

# misc
.env
.DS_Store
.env.local
.env.development.local
.env.test.local
.env.production.local
```

Secvență de cod 4.3 Fișierul `.env` din frontend și integrarea cheilor Stripe și OpenRouteService

Pe partea de backend, fișierul `server.js` este responsabil de inițializarea aplicației Express și de integrarea tuturor modulelor și rutelor. Acesta configurează middleware-urile necesare (`express.json`, `cors`, `session` etc.) și integrează modulul `passport` pentru autentificarea utilizatorilor (inclusiv prin Google și Facebook). De asemenea, se configurează serverul HTTP împreună cu `Socket.IO`, care permite funcționalitatea de mesagerie în timp real – utilizatorii fiind înscrși în camere dedicate, în funcție de ID-ul lor.

Toate rutele aplicației sunt înregistrate aici – de la cele pentru conturi de utilizator, produse, postări, comenzi, până la cele specifice pentru administrare. Fiecare este importată și atașată aplicației Express în mod modular, pentru o mai bună organizare a codului.

În final, serverul este pornit pe portul 4000, iar aplicația devine complet funcțională atât ca REST API, cât și ca sistem de comunicație în timp real prin WebSocket-uri.

```
const app = express();

// Middleware-uri
app.use(express.json());
app.use(cors());
app.use(
  session({
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: true,
  })
);
app.use(passport.initialize());
app.use(passport.session());
app.use("/uploads", express.static("uploads"));

// Pornirea serverului
const server = http.createServer(app);

const io = new Server(server, {
  cors: {
    origin: "*",
    methods: ["GET", "POST"]
  }
});

app.use((req, res, next) => {
  req.io = io;
  next();
});

const followRoutes = require("./routes/follow")(io);
const commentsRoutes = require("./routes/comments")(io);
// Rutele aplicației
app.use("/auth", authRoutes);
app.use("/users", userRoutes);
app.use("/posts", postRoutes);
app.use("/uploads", uploadRoutes);
app.use("/shop", shopRoutes);
app.use("/cart", cartRoutes);
app.use("/favorites", favoritesRouter);
app.use("/follows", followRoutes);
app.use("/messages", messagesRoutes);
app.use("/likes", likesRouter);
app.use("/appreciation", appreciationRouter);
app.use("/admin", adminRoutes);
app.use("/admin/statistics", statisticsRoutes);
app.use("/uploads", express.static("uploads"));
app.use("/admin/categories", categoryRoutes);
app.use("/admin/moderatePosts", moderatePostsRoutes);
app.use("/admin/moderateProducts", moderateProductsRoutes);
app.use("/admin/moderateUsers", moderateUsersRoutes);
app.use("/admin/moderateOrders", moderateOrdersRoutes);
app.use("/admin/statistics", adminStatistics);
app.use("/orders", orderRoutes);
app.use("/stripe", stripeRoutes);
app.use("/discover", discoverPostsRoute);
app.use("/discover", discoverProductsRoute);
app.use("/comments", commentsRoutes);
app.use("/", userRoutes);

// Socket.IO
io.on("connection", (socket) => {
  console.log("Client connected:", socket.id);

  socket.on("join", (userId) => {
    console.log("Received join request for user: ${userId}");
    socket.join(`user-${userId}`);
    console.log("Joined room: user-${userId}");
  });

  socket.on("disconnect", () => {
    console.log("Client disconnected:", socket.id);
  });
});

// Start server
server.listen(4000, () => console.log("Server running on http://localhost:4000"));
```

Secvență de cod 4.4 Configurarea completă a serverului backend (server.js)

Un aspect important este faptul că și în backend toate datele sensibile, precum parolele de acces la baza de date, token-urile de autentificare, cheile Stripe sau parolele Nodemailer, sunt gestionate prin fișierul .env, aflat de asemenea în .gitignore. Astfel, securitatea aplicației este menținută, iar configurările pot fi adaptate ușor pentru diferite medii (development, producție etc.).

4.1 MECANISME DE AUTENTIFICARE ȘI GESTIONARE CONT

4.1.1 Înregistrare prin formular

Această metodă este destinată utilizatorilor standard și presupune completarea unui formular construit în React. Fiecare câmp este marcat ca obligatoriu, folosind atributul „required”, iar dacă formularul este trimis incomplet, utilizatorul este atenționat ([Figura 4.1](#)).

```
<div className={styles.inputGroup}>
  <input
    type="email"
    id="email"
    placeholder=" "
    required
    className={styles.input}
    data-name="email"
    value={formData.email}
    onChange={handleInputChange}
  />
  <label htmlFor="email" className={styles.label}>
    Email
  </label>
  <i className={`fas fa-envelope ${styles.icon}`}></i>
</div>
```

Secvență de cod 4.5 Exemplu de câmp obligatoriu în formularul de înregistrare

După completare, aplicația verifică local validitatea tuturor datelor. Un prim pas este asigurarea faptului că parola și confirmarea acesteia coincid. Dacă nu, apare un mesaj de eroare în interfață.

```
if (formData.password !== formData.confirmPassword) {  
  setErrorMessage("Passwords do not match.");  
  hideMessageAfterDelay(setErrorMessage);  
  return;  
}
```

Secvență de cod 4.6 Verificarea potrivirii parolelor

Pe lângă potrivirea celor două parole, aplicația impune și un standard minim de siguranță pentru parolă. Se folosește o expresie regulată în JavaScript pentru a verifica dacă parola include cel puțin o literă mare, o cifră, un caracter special și are minimum 8 caractere.

```
const isPasswordStrong = (password) => {  
  const regex = /^(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()_~?":{}|<>])(?=.*{8,})/;  
  return regex.test(password);  
};
```

Secvență de cod 4.7 Validarea complexității parolei

Dacă parola nu respectă aceste cerințe, este afișat un mesaj prietenos de tip eroare ([Figura 4.1](#)). Odată validate, datele sunt trimise la backend printr-un request POST. În cazul unui răspuns pozitiv din partea serverului, primește un obiect JSON care conține, printre altele, `user_id`. Acest ID este stocat local în `localStorage` împreună cu alte informații utile, cum ar fi un flag care indică dacă este un utilizator nou. După aceasta, utilizatorul este redirecționat către o pagină specială de completare a profilului (`/additional-info`).

```
try {  
  const response = await fetch("http://localhost:4000/adduser", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json",  
    },  
    body: JSON.stringify({  
      first_name: formData.firstName,  
      last_name: formData.lastName,  
      email: formData.email,  
      password: formData.password,  
    }),  
  });  
  
  const result = await response.json();  
  if (response.ok) {  
    setSuccessMessage("User registered successfully!");  
  
    localStorage.setItem("user_id", result.user.user_id);  
    localStorage.setItem("is_admin", "false");  
  
    localStorage.setItem("isNewUser", "true");  
  }  
}
```

Secvență de cod 4.8 Apelul către backend și salvarea datelor în `localStorage`

În backend are loc procesarea înregistrării. Se verifică mai întâi dacă există deja un cont asociat adresei de email. Dacă totul este în regulă, parola este criptată folosind algoritmul bcrypt, apoi datele sunt inserate în tabela accounts.

```
router.post("/adduser", async (req, res) => {
  try {
    const { first_name, last_name, email, password, birth_date, bio } = req.body;

    const existingUser = await pool.query(
      "SELECT * FROM accounts WHERE email = ?",
      [email]
    );

    if (existingUser.rows.length > 0) {
      return res.status(400).json({ error: "User with this email already exists." });
    }

    const hashedPassword = await bcrypt.hash(password, saltRounds);

    const result = await pool.query(
      "INSERT INTO accounts (first_name, last_name, email, password, birth_date, bio) VALUES ($1, $2, $3, $4, $5, $6) RETURNING *",
      [
        first_name,
        last_name,
        email,
        hashedPassword,
        birth_date || null,
        bio || null,
      ]
    );

    res.status(201).json({
      message: "User added successfully!",
      user: result.rows[0],
    });
  } catch (err) {
    console.error("Error adding user:", err.message);
    res.status(500).json({ error: "An error occurred while adding the user." });
  }
});
```

Secvență de cod 4.9 Înregistrarea unui nou utilizator în baza de date

Figura 4.1 Interfața de înregistrare

4.1.2 Înregistrare și autentificare socială

În paralel cu înregistrarea tradițională, „CraftCircle” oferă și o metodă de autentificare modernă, bazată pe OAuth 2.0, prin care utilizatorii se pot conecta folosind contul lor de Google sau Facebook. Această integrare este realizată folosind pachetul passport.js, împreună cu strategiile GoogleStrategy și FacebookStrategy. Din punct de vedere al utilizatorului, experiența este simplă: un singur click pe unul din butoanele cu pictograme reprezentative pentru fiecare modalitate și utilizatorul este redirecționat către providerul extern pentru confirmarea identității.

Odată înregistrat, aplicația preia automat datele relevante din contul social: nume, prenume, email și poza de profil. În backend, aplicația verifică dacă acel email există deja în baza de date, în acest caz fluxul este tratat ca un simplu login, fără a crea un cont nou.

În schimb, dacă emailul nu este regăsit, aplicația creează un cont automat, folosind datele obținute prin OAuth:

```
profilefields: ["id", "emails", "name", "picture.type(large)"],
},
async (accessToken, refreshToken, profile, done) => {
  try {
    const email = profile.emails?.[0]?.value || `facebook_${profile.id}@noemail.com`;
    const temporaryPassword = `facebook_${profile.id}`;
    let isNewUser = false;
    let user;

    const existingUser = await pool.query(
      "SELECT * FROM accounts WHERE email = ?",
      [email]
    );

    if (existingUser.rows.length > 0) {
      // Utilizatorul exista deja
      user = existingUser.rows[0];
    } else {
      // Creează un utilizator nou
      const newUser = await pool.query(
        "INSERT INTO accounts (first_name, last_name, email, profile_picture, password, created_at) VALUES (?, ?, ?, ?, ?, NOW()) RETURNING *",
        [
          profile.name?.givenName || "Facebook User",
          profile.name?.familyName || "Unknown",
          email,
          profile.photos?.[0]?.value || null,
          temporaryPassword,
        ]
      );
      user = newUser.rows[0];
      isNewUser = true;
    }
  }
}
```

Secvență de cod 4.10 Verificarea existenței utilizatorului și înregistrarea automată prin Facebook OAuth

Un aspect important este că fluxul de autentificare socială este identic, indiferent dacă este inițiat din pagina de login sau înregistrare. Dacă utilizatorul există, este autentificat; dacă nu, se creează automat un cont și se trece la completarea profilului, ca în înregistrarea clasică.

După autentificare, backend-ul redirecționează către frontend cu parametrii `user_id` și `isNewUser`, permițând aplicației să decidă dacă utilizatorul ajunge pe pagina principală sau la formularul de completare a profilului.

```
router.get(
  "/google/callback",
  passport.authenticate("google", { failureRedirect: "http://localhost:3000/login" }),
  (req, res) => {
    if (!req.user) {
      console.log("No user returned from Google.");
      return res.redirect("http://localhost:3000/login");
    }

    console.log("User:", req.user);
    console.log("isNewUser:", req.user.isNewUser);

    const redirectUrl = `http://localhost:3000/login?user_id=${req.user.user_id}&isNewUser=${req.user.isNewUser}`;
    res.redirect(redirectUrl);
  }
);
```

Secvență de cod 4.11 Redirecționarea utilizatorului după autentificare socială

Dincolo de implementare, merită subliniat faptul că protocolul OAuth 2.0 aduce beneficii importante din perspectiva securității și a experienței de utilizare. Utilizatorii nu trebuie să-și creeze sau să-și amintească încă o parolă, iar aplicația evită procesul sensibil al stocării parolelor pentru conturi externe.

4.1.3 Completarea profilului

După finalizarea procesului de înregistrare, fiecare utilizator nou este direcționat către o pagină unde trebuie să-și completeze profilul personal cu informații esențiale ([Figura 4.2](#)).

Lista țărilor este încărcată automat dintr-o sursă publică [17] ([Figura 4.2](#)), iar orașele disponibile sunt afișate în funcție de țara selectată, folosind o interfață de programare dedicată [21].

În plus, aplicația impune o validare a vârstei minime ([Figura 4.2](#)), calculul vârstei este realizat în frontend pe baza datei introduse.

```
if (!user_id || !formData.birth_date || !formData.country || !formData.city) {  
  setErrorMessage("Please fill in all fields.");  
  return;  
}  
  
const birthDate = new Date(formData.birth_date);  
const today = new Date();  
const age = today.getFullYear() - birthDate.getFullYear();  
const monthDiff = today.getMonth() - birthDate.getMonth();  
const dayDiff = today.getDate() - birthDate.getDate();  
const isAtLeast16 = age > 16 || (age === 16 && (monthDiff > 0 || (monthDiff === 0 && dayDiff >= 0)));  
  
if (!isAtLeast16) {  
  setErrorMessage("You must be at least 16 years old to use this platform.");  
  setTimeout(() => {  
    setErrorMessage("");  
  }, 5000);  
  return;  
}
```

Secvență de cod 4.12 Validarea vârstei minime de 16 ani pentru înregistrare

După ce toate condițiile sunt îndeplinite, datele sunt trimise către backend, unde ruta „/additional-info” salvează informațiile în baza de date în câmpurile corespunzătoare din tabela accounts.

Figura 4.2 Interfața de adăugare informații adiționale

4.1.4 Integrarea în contextul global

După salvarea datelor, aplicația folosește funcția login() definită în UserContext.js pentru a actualiza global starea autentificării. Această funcție setează userId și, după caz, flag-ul isAdmin, permițând accesul la funcționalitățile specifice utilizatorului autentificat în toate componentele aplicației.

Hook-ul useUser() oferă acces rapid la aceste informații, permițând altor componente să afle dacă utilizatorul este logat și să reacționeze corespunzător. De asemenea, componenta se ocupă de deschiderea conexiunii WebSocket pentru mesagerie în cazul utilizatorilor standard (non-admin), trimițând automat un eveniment join către server.

```
useEffect(() => {  
  if (userId && !isAdmin) {  
    const newSocket = io("http://localhost:4000");  
  
    newSocket.on("connect", () => {  
      /*console.log("Frontend socket connected:", newSocket.id);  
      console.log("Emitting JOIN after connect:", userId);*/  
      newSocket.emit("join", userId);  
    });  
  
    setSocket(newSocket);  
  
    return () => {  
      /*console.log("Disconnecting socket...");*/  
      newSocket.disconnect();  
    };  
  }  
}, [userId, isAdmin]);
```

Secvență de cod 4.13 Emiterea automată a evenimentului join după login

Această integrare asigură că, imediat după completarea profilului, utilizatorul este autentificat complet și conectat la toate funcționalitățile aplicației. În plus, variabila isNewUser este ștearsă din localStorage pentru a evita redirecționări ulterioare către formular.

4.1.5 Autentificare standard (administratori și utilizatori)

Autentificarea se face prin completarea unui formular cu două câmpuri principale: email și parolă (Figura 4.3). Acestea sunt reținute într-un obiect de stare, adică o structură React folosită pentru a memora și actualiza valorile introduse de utilizator în timp real, fără a reîncărca pagina.

```
const [formData, setFormData] = useState({  
  email: "",  
  password: "",  
});
```

Secvență de cod 4.14 Inițializarea stării formularului de autentificare

Pe măsură ce utilizatorul tastează, valorile se actualizează:

```
const handleInputChange = (e) => {  
  const { id, value } = e.target;  
  setFormData((prevData) => ({  
    ...prevData,  
    [id]: value,  
  }));  
};
```

Secvență de cod 4.15 Actualizarea dinamică a valorilor din formular

Odată ce ambele câmpuri sunt completate cu datele corespunzătoare și este apăsat butonul „Log In”, se verifică în frontend dacă email-ul introdus are sau nu terminația „@craft.com”, în caz afirmativ, aplicația va trimite cererea de autentificare către ruta „/admin/login”, iar în caz contrar, către „/login”, destinată utilizatorilor standard.

```
const isAdmin = formData.email.endsWith("@craft.com");
const url = isAdmin
  ? "http://localhost:4000/admin/login"
  : "http://localhost:4000/login";

try {
  const response = await fetch(url, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      email: formData.email,
      password: formData.password,
    }),
  });

  const result = await response.json();
```

Secvență de cod 4.16 Determinarea tipului de utilizator și trimiterea cererii de autentificare

În backend, atât ruta „/login”, cât și „/admin/login”, verifică dacă email-ul introdus se regăsește în tabela corespunzătoare din baza de date, în caz pozitiv, parola este verificată folosind biblioteca bcrypt, pentru a compara valoarea introdusă cu cea hash-uita salvată în baza de date.

```
// LOGIN
router.post("/login", async (req, res) => {
  const { email, password } = req.body;

  try {
    const user = await pool.query("SELECT * FROM accounts WHERE email = ?", [
      email,
    ]);

    if (user.rows.length === 0) {
      return res.status(404).json({ error: "User not found." });
    }

    const isMatch = await bcrypt.compare(password, user.rows[0].password);

    if (!isMatch) {
      return res.status(401).json({ error: "Invalid password." });
    }

    res.json({
      message: "Login successful.",
      user: {
        user_id: user.rows[0].user_id,
        email: user.rows[0].email,
        first_name: user.rows[0].first_name,
        last_name: user.rows[0].last_name,
      },
    });
  } catch (err) {
    console.error("Error during login:", err);
    res.status(500).json({ error: "An error occurred." });
  }
});

// POST /admin/login
router.post("/admin/login", async (req, res) => {
  const { email, password } = req.body;

  if (!email.endsWith("@craft.com")) {
    return res.status(403).json({ error: "Access restricted to admin accounts." });
  }

  try {
    const result = await pool.query("SELECT * FROM admins WHERE email = ?", [email]);
    if (result.rows.length === 0) {
      return res.status(401).json({ error: "Admin account not found." });
    }

    const admin = result.rows[0];

    const validPassword = await bcrypt.compare(password, admin.password_hash);
    if (!validPassword) {
      return res.status(401).json({ error: "Incorrect password." });
    }

    res.json({
      admin: {
        admin_id: admin.admin_id,
        first_name: admin.first_name,
        last_name: admin.last_name,
        email: admin.email,
        profile_picture: admin.profile_picture,
      },
    });
  } catch (err) {
    console.error("Login admin error:", err);
    res.status(500).json({ error: "Server error." });
  }
});
```

/login

admin/login

Secvență de cod 4.17 Backend autentificare admin și user normal

Dacă datele sunt valide, utilizatorul este autentificat, iar în frontend context este salvat `user_id` și un boolean care indică dacă utilizatorul logat este sau nu admin. Redirecționarea către pagina de profil se face în funcție de tipul utilizatorului.

```
if (response.ok) {  
  if (isAdmin) {  
    console.log("Admin ID saved:", result.admin.admin_id);  
    localStorage.setItem("is_admin", "true");  
    login(result.admin.admin_id, true);  
    navigate(`/admin_profile/${result.admin.admin_id}`);  
  } else {  
    console.log("User ID saved:", result.user.user_id);  
    localStorage.setItem("is_admin", "false");  
    login(result.user.user_id, false);  
    navigate(`/profile/${result.user.user_id}`);  
  }  
}
```

Secvență de cod 4.18 Redirecționarea către pagina de profil în cazul unei autentificări reușite

4.1.6 Resetarea parolei

În vederea asigurării unei experiențe complete, formularul de login conține și butonul care permite restarea parolei, în cazul în care utilizatorul și-a uitat parola. Acest formular este compus din 3 pași ([Figura 4.3](#)).

- Trimiterea codului de verificare:

Utilizatorul introduce adresa de email într-un formular special. Backend-ul generează un cod numeric din 6 cifre, salvat în baza de date și trimis utilizatorului pe email, folosind Nodemailer.

```
router.post("/send-reset-code", async (req, res) => {  
  const { email } = req.body;  
  
  try {  
    const result = await pool.query(  
      "SELECT first_name FROM accounts WHERE email = ?",  
      [email]  
    );  
  
    if (result.rows.length === 0) {  
      return res.status(404).json({ error: "Email not found" });  
    }  
  
    const firstName = result.rows[0].first_name;  
    const code = Math.floor(100000 + Math.random() * 900000); // 6 cifre  
  
    await pool.query(  
      "INSERT INTO password_reset_codes (email, code, expires_at)  
      VALUES (?, ?, NOW() + INTERVAL '15 minutes')  
      ON CONFLICT (email) DO UPDATE SET code = ?, expires_at = NOW() + INTERVAL '15 minutes'",  
      [email, code]);  
  
    // Trimite emailul  
    await transporter.sendMail({  
      from: "oxanicorina@gmail.com",  
      to: email,  
      subject: "CraftCircle - Your Password Reset Code",  
      html: `

Hi ${firstName},



Your password reset code is: ${code}.



This code is valid for 15 minutes.

`  
    });  
  }  
});
```

Secvență de cod 4.19 Generarea unui cod de securitate și trimiterea prin email

- Verificare codului:

Utilizatorul introduce codul în aplicație. Acesta este validat ulterior în backend prin comparație cu cel din baza de date.

- Schimbarea parolei:

După validarea codului, utilizatorul poate introduce o nouă parolă. Aceasta este securizată cu bcrypt și actualizată în baza de date, după care este trimis și un mail de confirmare către utilizator.

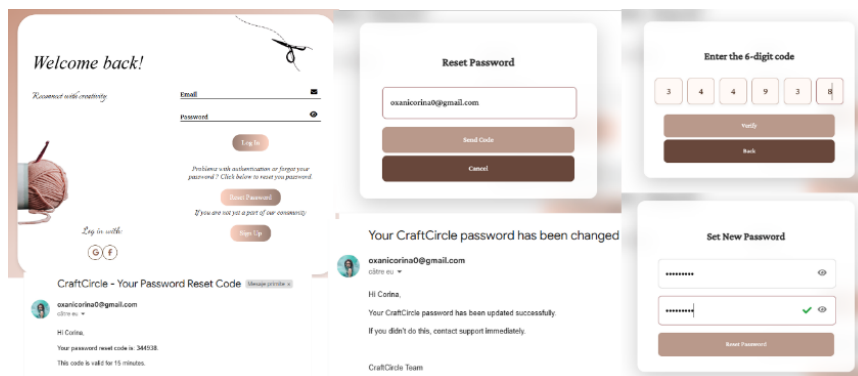


Figura 4.3 Interfața de login și modalul de resetare a parolei

4.1.7 Editarea datelor personale și schimbarea parolei

Utilizatorii standard pot edita informațiile de profil (țară, oraș și descriere) printr-un modal ce preia automat datele curente și le permite actualizarea. Țările și orașele sunt încărcate în mod similar ca în pagina de adăugare a datelor adiționale în urma înregistrării. Datele noi sunt trimise către backend, la ruta „users/update-profile”, unde sunt reactualizate campurile corespunzătoare.

```
useEffect(() => {
  const fetchCountries = async () => {
    try {
      const res = await axios.get("https://restcountries.com/v3.1/all?fields=name,cca2");
      const countryList = res.data.map(c => c.name.common);
      setCountries(countryList.sort());
    } catch (err) {
      console.error("Error loading countries", err);
    }
  };
  fetchCountries();
}, []);

useEffect(() => {
  const fetchCities = async () => {
    if (!formData.country) return;
    try {
      const res = await axios.post("https://countriesnow.space/api/v0.1/countries/cities", {
        country: formData.country,
      });
      if (!res.data.error) {
        setCities(res.data.data);
      } else {
        setCities([]);
      }
    } catch {
      setCities([]);
    }
  };
  fetchCities();
}, [formData.country]);
```

Secvență de cod 4.20 Preluarea dinamică a țărilor și orașelor

```
try {
  const res = await fetch("http://localhost:4000/users/update-profile", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ user_id, country, city, bio }),
  });

  const data = await res.json();

  if (res.ok) {
    setMessage("Profile updated successfully!");
    onSave?.();
    setTimeout(() => onClose(), 1500);
  } else {
    setMessage(data.error || "Failed to update profile.");
  }
} catch {
  setMessage("Server error.");
}
```

Secvență de cod 4.21 Trimiterea datelor modificate către backend

De asemenea, aplicația permite modificarea pozei de profil pentru ambele tipuri de conturi (utilizator normal și administrator). Butonul de editare apare doar pentru utilizatorul propriu lângă poza de profil. După selectarea unei imagini, aceasta este previzualizată și poate fi salvată sau anulată. Salvarea duce la un request POST către endpoint-ul specific (/uploads/upload-profile pentru utilizatori, /admin/upload-admin-profile pentru administratori), iar imaginea este stocată și eventual imaginea anterioară este ștearsă de pe server dacă există.

```
router.post("/upload-admin-profile", upload.single("file"), async (req, res) => {
  try {
    const { admin_id } = req.body;
    if (!req.file) return res.status(400).json({ error: "No file uploaded" });

    const fileUrl = req.file.path;

    const result = await pool.query("SELECT profile_picture FROM admins WHERE admin_id = ?", [admin_id]);
    const oldImageUrl = result.rows[0].profile_picture;

    if (oldImageUrl?.includes("res.cloudinary.com") && oldImageUrl.includes("/admin_uploads/")) {
      const filePath = oldImageUrl.split("/admin_uploads/")[1];
      const publicId = "admin_uploads/" + filePath.split(".").join("");
      await cloudinary.uploader.destroy(publicId, { resource_type: "image" });
      console.log("Deleted old admin image:", publicId);
    }

    await pool.query("UPDATE admins SET profile_picture = ? WHERE admin_id = ?", [fileUrl, admin_id]);

    res.json({ imageUrl: fileUrl });
  } catch (error) {
    console.error("Admin profile upload error:", error);
    res.status(500).json({ error: "Failed to upload profile picture" });
  }
});

router.post("/upload-profile", upload.single("file"), async (req, res) => {
  try {
    const { user_id } = req.body;
    if (!req.file) return res.status(400).json({ error: "No file uploaded" });

    const fileUrl = req.file.path;

    const userResult = await pool.query("SELECT profile_picture FROM accounts WHERE user_id = ?", [user_id]);
    const oldImageUrl = userResult.rows[0].profile_picture;

    if (oldImageUrl) {
      const filePath = oldImageUrl.split("/user_uploads/")[1];
      if (filePath) {
        const publicId = "user_uploads/" + filePath.split(".").join("");
        await cloudinary.uploader.destroy(publicId, { resource_type: "image" });
        console.log("Old profile picture deleted from Cloudinary:", publicId);
      }
    }

    await pool.query("UPDATE accounts SET profile_picture = ? WHERE user_id = ?", [fileUrl, user_id]);

    res.json({ imageUrl: fileUrl });
  } catch (error) {
    console.error("Upload error:", error);
    res.status(500).json({ error: "Error uploading profile picture" });
  }
});
```

/admin/upload-admin-profile /uploads/upload-profile

Secvență de cod 4.22 Backend - rutele pentru schimbarea pozei de profil administrator + utilizator

Aplicația oferă posibilitatea schimbării parolei pentru ambele tipuri de conturi (utilizator standard și administrator), folosind o componentă unificată PasswordModal.js. Acest modal poate fi accesat din meniul dropdown din Navbar, iar comportamentul intern este controlat printr-un parametru boolean isAdmin, importat din UserContext, care determină dacă apelul către backend trebuie să se adreseze rutei pentru utilizatori sau celei pentru administratori.

Componenta permite introducerea parolei vechi, a noii parole și a confirmării acesteia. Siguranța introducerii unei parole noi puternice sunt aceleași ca în cazul înregistrării. Dacă toate condițiile sunt îndeplinite, iar parolele noi coincid, datele sunt trimise către backend printr-un request POST, iar endpoint-ul este ales dinamic, în funcție de tipul contului.

```
1 import React, { useState } from "react";
2 import styles from "../CSSfiles/PasswordModal.module.css";
3 import { useUser } from "../UserContext";
4 import { FaCheck, FaTimes, FaEye, FaEyeSlash } from "react-icons/fa";
5
6 export default function PasswordModal({ onClose }) {
7   const { userId, isAdmin } = useUser();
```

Secvență de cod 4.23 Accesarea contextului utilizatorului

```
try {
  const endpoint = isAdmin
    ? "http://localhost:4000/admin/change-password"
    : "http://localhost:4000/users/change-password";

  const res = await fetch(endpoint, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      [isAdmin ? "admin_id" : "user_id"]: userId,
      oldPassword,
      newPassword,
    }),
  });
}
```

Secvență de cod 4.24 Trimiterea cererii de actualizare a parolei către backend

După validări și actualizarea parolei în baza de date, serverul trimite automat un email de confirmare folosind Nodemailer.

4.2 FUNCȚIONALITĂȚI SOCIALE

4.2.1 Gestionarea postărilor: adăugare, editare și ștergerea

În aplicația „CraftCircle”, utilizatorii logați își pot exprima creativitatea prin publicarea de postări multimedia, folosind formularul vizibil în pagina lor de profil. Acest formular este reprezentat de componenta PostForm.js și este vizibil doar atunci când utilizatorul își vizitează propriul profil:

```
{parseInt(userId) === parseInt(loggedInUserId) && (
  <PostForm
    postContent={postContent}
    setPostContent={setPostContent}
    handlePostFilesChange={handlePostFilesChange}
    handleSubmitPost={handleSubmitPost}
    previewFiles={previewFiles}
    handleRemovePreview={handleRemovePreview}
    isPosting={isPosting}
    selectedCategory={selectedCategory}
    setSelectedCategory={setSelectedCategory}
    categorySearch={categorySearch}
    setCategorySearch={setCategorySearch}
    fileError={fileError}
    setFileError={setFileError}
  />
```

Secvență de cod 4.25 Renderează formularul de postare doar pentru utilizatorul autentificat

Formularul permite:

- introducerea de text (postContent)
- alegerea unei categorii predefinite, cu auto-complete (categorySearch)
- adăugarea de imagini și videoclipuri (postFiles)
- afișarea de previzualizări pentru fișierele selectate (previewFiles)

- validare locală a câmpurilor obligatorii

Totul este controlat prin state hooks: `postFiles` este array-ul care conține fișierele reale ce vor fi trimise către backend, iar `previewFiles` este array-ul care stochează doar linkuri temporare, folosite pentru a permite utilizatorului să vadă un preview al imaginilor sau videoclipurilor încărcate.

```
const [postFiles, setPostFiles] = useState([]);  
const [previewFiles, setPreviewFiles] = useState([]);
```

Secvență de cod 4.26 Inițializarea stării pentru fișierele postării și previzualizarea acestora

Funcția `handlePostFilesChange` este apelată atunci când utilizatorul selectează fișiere dintr-un input de tip `<input type="file" multiple />`. Aceasta primește evenimentul generat de input, care conține obiectul `event.target.files`, un `FileList` cu fișierele selectate. Prin `Array.from()`, acest `FileList` este convertit într-un array real de obiecte `File`.

Linia `„setPostFiles(prev => [...prev, ...newFiles])”` adaugă fișierele noi la cele deja existente în `postFiles`, fără a le înlocui. `„...prev”` preia fișierele deja selectate, iar `„...newFiles”` adaugă cele noi.

În continuare, se construiește un array de obiecte `newPreviews`, fiecare având un link temporar generat cu `URL.createObjectURL(file)` și un câmp `type`, care este setat în funcție de tipul fișierului (`image` sau `video`). Aceste preview-uri sunt apoi adăugate în `previewFiles`, pentru a fi afișate în interfață.

```
// Upload de fișiere pentru postare  
const handlePostFilesChange = (event) => {  
  const newFiles = Array.from(event.target.files);  
  if (newFiles.length > 0) setFileError(false);  
  setPostFiles((prevFiles) => [...prevFiles, ...newFiles]);  
  
  const newPreviews = newFiles.map((file) => ({  
    url: URL.createObjectURL(file),  
    type: file.type.startsWith("image") ? "image" : "video",  
  }));  
  setPreviewFiles((prevPreviews) => [...prevPreviews, ...newPreviews]);  
};
```

Secvență de cod 4.27 Gestionarea fișierelor încărcate și generarea previzualizărilor

Fiecare imagine sau video afișată în interfață include un buton „x” în colțul din dreapta sus, care permite eliminarea fișierului din listă. Când acest buton este apăsat, funcția `handleRemovePreview` primește indexul fișierului care trebuie eliminat. Deoarece `postFiles` și `previewFiles` au aceeași ordine, este suficient să filtrăm după index pentru a elimina sincron ambele elemente. Se aplică metoda `filter((_, i) => i !== indexToRemove)` pe ambele array-uri, păstrând toate elementele cu excepția celui de pe poziția indicată. Astfel, fișierul este eliminat atât din interfață, cât și din lista de fișiere ce urmează să fie trimisă spre backend.

```
//stergere poza incarcata in formular  
const handleRemovePreview = (indexToRemove) => {  
  setPreviewFiles(prev => prev.filter((_, i) => i !== indexToRemove));  
  setPostFiles(prev => prev.filter((_, i) => i !== indexToRemove));  
};
```

Secvență de cod 4.28 Eliminarea fișierelor încărcate din formularul de postare

Când utilizatorul apasă butonul „Post”, datele sunt împachetate într-un FormData, iar mai apoi cererea este trimisă printr-un fetch POST către backend.

```
const formData = new FormData();
formData.append("user_id", userId);
formData.append("content", postContent);
postFiles.forEach((file) => formData.append("files", file));

if (selectedCategory) {
  formData.append("category_id", selectedCategory.category_id);
}

try {
  setIsPosting(true);
  const response = await fetch("http://localhost:4000/uploads/upload-post", {
    method: "POST",
    body: formData,
  });
}
```

Secvență de cod 4.29 Construirea și trimiterea datelor postării către backend

În timpul încărcării postării în baza de date, butonul „Post” este dezactivat și apare un mesaj intuitiv, măsură luată pentru că s-a observat un timp de încărcare direct proporțional cu mărimea fișierelor încărcate.

După ce serverul trimite un răspuns de succes, mesajul dispare, câmpurile formularului sunt resetate, iar lista postărilor (fetchUserPosts()) și datele profilului (fetchUserProfile()) sunt actualizate pentru a reflecta noua postare și numărul total.

În backend, sunt extrase user_id, content, category_id și fișierele încărcate. Dacă lipsesc sau nu există fișiere, se returnează eroare 400. Dacă totul este valid, se inserează o postare nouă în tabela posts, iar câmpul created_at este completat automat cu data curentă prin NOW(). După obținerea post_id, fiecare fișier este încărcat pe Cloudinary și inserat în post_media cu URL-ul generat și tipul (image sau video), dedus din mimetype. La final, serverul trimite un răspuns JSON cu mesajul de succes și ID-ul noii postări. În caz de eroare, procesul este oprit, iar problema este logată.

```
router.post("/upload-post", upload.array("files", 10), async (req, res) => {
  try {
    const { user_id, content, category_id } = req.body;

    if (!user_id) {
      return res.status(400).json({ error: "User ID is required." });
    }

    const files = req.files;
    if (!files || files.length === 0) {
      return res.status(400).json({ error: "At least one file is required." });
    }

    const postResult = await pool.query(
      `INSERT INTO posts (user_id, content, category_id, created_at)
      VALUES ($1, $2, $3, NOW())
      RETURNING post_id,
      [user_id, content || null, category_id || null]`
    );

    const postId = postResult.rows[0].post_id;

    for (const file of files) {
      await pool.query(
        `INSERT INTO post_media (post_id, file_url, file_type) VALUES ($1, $2, $3)`,
        [postId, file.path, file.mimetype.startsWith("image") ? "image" : "video"]
      );
    }

    res.json({ message: "Post added successfully!", postId });
  } catch (error) {
    console.error("Upload error:", error);
    res.status(500).json({ error: "Error uploading post." });
  }
});
```

Secvență de cod 4.30 Endpoint backend pentru încărcarea postării și fișierelor media în baza de date

Funcționalitatea de editare a postărilor în „CraftCircle” este rezervată exclusiv autorului postării, oferindu-i acestuia posibilitatea de a modifica textul oricând după publicare. Când utilizatorul apasă butonul „Edit Post”, aplicația salvează în state ID-ul postării și conținutul actual, pentru a le putea prelucra. Acest eveniment face ca textul static să fie înlocuit cu un câmp <textarea>, în care conținutul poate fi modificat în timp real. Tot ce scrie utilizatorul în acest câmp este salvat într-o variabilă controlată, editedContent, care reflectă modificările aduse până în momentul salvării.

```
{editingPostId === post.post_id ? (  
  <div>  
    <textarea  
      className={styles.editInput}  
      value={editedContent}  
      onChange={(e) => setEditedContent(e.target.value)}  
    />  
    <button  
      className={styles.saveButton}  
      onClick={() => onSaveEdit(post.post_id)}  
    >  
      Save  
    </button>  
  </div>  
)
```

Secvență de cod 4.31 Formular de editare a conținutului unei postări

După ce modificările au fost realizate, utilizatorul poate apăsa butonul "Save". În acel moment, aplicația trimite o cerere HTTP de tip PUT către backend, conținând ID-ul postării și noul conținut sub formă de obiect JSON. Serverul actualizează postarea în baza de date și, dacă operația reușește, frontend-ul actualizează local lista postărilor, înlocuind conținutul vechi cu cel nou pentru postarea respectivă.

Utilizatorul își poate șterge propria postare cu un simplu click, ceea ce declanșează funcția handleDeletePost.

```
// Ștergerea unei postări  
const handleDeletePost = async (postId) => {  
  try {  
    const response = await fetch(`http://localhost:4000/posts/${postId}`, {  
      method: "DELETE",  
    });  
  
    if (response.ok) {  
      setPosts((prevPosts) => prevPosts.filter((post) => post.post_id !== postId));  
      showToast("Post deleted successfully!");  
    } else {  
      console.error("Failed to delete post, status:", response.status);  
      showToast("Failed to delete post.");  
    }  
  } catch (error) {  
    console.error("Error deleting post:", error);  
    showToast("An error occurred while deleting the post.");  
  }  
};
```

Secvență de cod 4.32 Funcția de ștergere a unei postări

În backend, postId-ul este mai întâi validat pentru a verifica dacă este un număr corect. Dacă postarea există în baza de date, sunt preluate fișierele media asociate din tabela post_media, inclusiv linkurile și tipul fișierelor. Pentru fiecare fișier, se extrage publicId din URL-ul Cloudinary, se determină tipul (image sau video), apoi fișierul este șters din Cloudinary cu „cloudinary.uploader.destroy”. După eliminarea fișierelor din cloud, se șterg și din baza de date, iar apoi se elimină postarea din posts, folosind „RETURNING * ” pentru a recupera datele postării șterse ca mai apoi să le trimită către frontend. La final, serverul trimite un răspuns JSON cu confirmarea ștergerii și detaliile postării eliminate.

```
router.delete("/:postId", async (req, res) => {
  const postId = parseInt(req.params.postId, 10);

  if (isNaN(postId)) {
    return res.status(400).json({ error: "Invalid post ID" });
  }

  try {
    console.log("Checking if post exists with ID:", postId);

    const postCheck = await pool.query("SELECT * FROM posts WHERE post_id = $1", [postId]);

    if (postCheck.rows.length === 0) {
      console.error("Post not found, cannot delete:", postId);
      return res.status(404).json({ error: "Post not found" });
    }

    const mediaFiles = await pool.query("SELECT file_url, file_type FROM post_media WHERE post_id = $1", [postId]);

    for (const file of mediaFiles.rows) {
      const fileUrl = file.file_url;

      const filePath = fileUrl.split('/user_uploads/')[1];
      if (!filePath) {
        console.error("Invalid Cloudinary URL: ${fileUrl}");
        continue;
      }

      const publicId = `user_uploads/${filePath.split('.')[0]}`;
      const resourceType = file.file_type === "video" ? "video" : "image";

      try {
        const result = await cloudinary.uploader.destroy(publicId, { resource_type: resourceType });
        console.log("Deleted from Cloudinary: ${publicId}", result);
      } catch (cloudinaryError) {
        console.error("Cloudinary delete error:", cloudinaryError);
      }

      await pool.query("DELETE FROM post_media WHERE post_id = $1", [postId]);
    }

    const deletePost = await pool.query("DELETE FROM posts WHERE post_id = $1 RETURNING *", [postId]);

    console.log("Post deleted successfully:", deletePost.rows[0]);

    res.json({ message: "Post deleted successfully", deletedPost: deletePost.rows[0] });
  } catch (error) {
    console.error("Error deleting post:", error);
    res.status(500).json({ error: "Internal server error" });
  }
});
```

Secvență de cod 4.33 Backend, ruta de ștergere a unei postări

4.2.2 Aprecierea, comentarea și raportarea postărilor

Funcționalitatea este disponibilă doar pentru postări străine, la postările personale utilizatorii pot doar să vadă numărul total de aprecieri.

Pentru fiecare postare, componenta LikeButton este responsabilă de afișarea inimii de tip FaHeart, alături de numărul total de aprecieri. La accesarea paginii „userProfile”, este efectuat un fetch către backend (GET /likes/:postId/:userId) pentru a determina dacă utilizatorul curent a dat deja like și câte aprecieri are postarea. Valorile returnate sunt stocate în state-urile „liked” și „likeCount”.

```
useEffect(() => {
  fetch(`http://localhost:4000/likes/${postId}/${userId}`)
    .then(res => res.json())
    .then(data => {
      setLiked(data.userLiked);
      setLikeCount(data.totalLikes);
    })
    .catch(err => console.error("Error loading like status:", err));
}, [postId, userId]);
```

Secvență de cod 4.34 Obținerea stării de like

Apăsarea pe iconița inimii declanșează funcția `toggleLike`, care trimite o cerere POST fie către „/likes/add”, fie către „/likes/remove”, în funcție de starea actuală a aprecierii.

```
const url = liked ? "remove" : "add";
const res = await fetch(`http://localhost:4000/likes/${url}`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ post_id: postId, user_id: userId }),
});

if (res.ok) {
  setLiked(!liked);
  setLikeCount(prev => liked ? prev - 1 : prev + 1);
}
```

Secvență de cod 4.35 Comutarea stării de like la click

Butonul de comentarii este vizibil sub fiecare postare. Atunci când un utilizator face click pe acest buton, se deschide un modal, „CommentsModal”, care afișează toate comentariile asociate acelei postări.

La deschiderea modalului, aplicația trimite o cerere către backend (GET /comments/:postId) pentru a obține lista completă de comentarii. Acestea sunt apoi afișate într-un container scrollabil, fiecare comentariu fiind stilizat diferit în funcție de autor, cele scrise de utilizatorul logat sunt aliniate la dreapta, iar celelalte, la stânga. Fiecare comentariu include poza de profil, numele de utilizator, textul comentariului și data exactă a postării.

```
{comments.length === 0 ? (
  <p className={styles.noComments}>This post doesn't have any comments yet.</p>
) : (
  comments.map((c) => (
    <div key={c.id} className={` ${styles.commentBubble} ${c.user_id === userId ? styles.own : styles.other}`}>
      {c.user_id !== userId && (
        <img
          src={c.profile_picture || defaultProfile}
          className={styles.avatar}
          alt="avatar"
          onClick={() => {
            onClose();
            navigate(`/profile/${c.user_id}`);
          }}
          style={{ cursor: "pointer" }}
        />
      )}
      <div className={styles.commentText}>
        <div
          className={styles.username}
          onClick={() => {
            onClose();
            navigate(`/profile/${c.user_id}`);
          }}
          style={{ cursor: "pointer" }}
        >
          {c.username}
        </div>
      </div>
    </div>
  )}
)
```

Secvență de cod 4.36 Afișarea comentariilor și redirecționarea către profilul autorului

Sub lista de comentarii se află un câmp `<textarea>` unde utilizatorii pot scrie și trimite un comentariu nou. Dacă utilizatorul apasă Enter (fără Shift), comentariul este trimis instant printr-un POST către backend (/comments/:postId) și adăugat imediat în lista existentă de comentarii.

```
const res = await fetch(`http://localhost:4000/comments/${postId}`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    text,
    user_id: userId,
  }),
});

if (res.ok) {
  const newComment = await res.json();
  setComments((prev) => [...prev, newComment]);
  setText("");
}
```

Secvență de cod 4.37 Trimiterea unui comentariu nou și actualizarea interfeței

În cazul în care un utilizator vizualizează o postare proprie, în colțul din dreapta-sus al acesteia apare un meniu cu opțiunile „Edit” și „Delete”. Dacă însă postarea aparține altui utilizator, meniul conține doar opțiunea „Report”. Acest comportament este controlat la nivel de frontend, pe baza identificatorului utilizatorului conectat, obținut din contextul global al aplicației (UserContext).

```
{isAdmin && (
  <PostMenuButton
    postId={post.post_id}
    isOwner={isOwner}
    onDelete={onDeletePost}
    onEdit={() => onEditPost(post.post_id, post.content)}
    onReport={handleReportPost}
  />
)}

<ul>
  {isOwner ? (
    <>
      <li onClick={handleDelete}>Delete Post</li>
      <li onClick={handleEdit}>Edit Post</li>
    </>
  ) : (
    <li onClick={handleReport}>Report Post</li>
  )
)}
```

Secvență de cod 4.38 Afișarea condiționată a meniului postării în funcție de autorul postării

Raportarea se face printr-o funcție „handleReportPost”, apelată la click pe butonul „Report Post”. Nu se solicită un motiv de raportare, se trimite doar ID-ul postării și ID-ul utilizatorului

```
const handleReportPost = async (postId) => {
  try {
    const res = await fetch(`http://localhost:4000/posts/${postId}/report`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ user_id: loggedInUserId })
    });

    const data = await res.json();
    showToast(data.message === "Already reported"
      ? "You've already reported this post."
      : "Thanks! We'll review the post.");
  } catch (err) {
    console.error("Error reporting post:", err);
    showToast("Error reporting post.");
  }
};
```

Secvență de cod 4.39 Raportarea unei postări de către utilizator

Pe partea de server, endpoint-ul POST „/posts/:postId/report” primește user_id și verifică dacă utilizatorul a mai raportat deja postarea, caz în care nu se face dublă înregistrare. De asemenea, este actualizat și câmpul „report_count” din tabela posts, care reflectă câte raportări are fiecare postare.

```
router.post("/:postId/report", async (req, res) => {
  const { postId } = req.params;
  const { user_id } = req.body;

  if (!user_id || !postId) {
    return res.status(400).json({ error: "Missing postId or user_id" });
  }

  try {
    const existing = await pool.query(
      "SELECT * FROM post_reports WHERE user_id = $1 AND post_id = $2",
      [user_id, postId]
    );

    if (existing.rows.length > 0) {
      return res.status(200).json({ message: "Already reported" });
    }

    await pool.query("INSERT INTO post_reports (user_id, post_id) VALUES ($1, $2)", [user_id, postId]);
    await pool.query("UPDATE posts SET report_count = report_count + 1 WHERE post_id = $1", [postId]);

    res.status(200).json({ message: "Report submitted" });
  } catch (error) {
    console.error("Error reporting post:", error);
    res.status(500).json({ error: "Failed to report post" });
  }
});
```

Secvență de cod 4.40 Endpoint backend pentru raportarea unei postări

4.2.3 Interacțiuni între utilizatori: urmărire și raportare

În interfață, în componenta ProfileStats, sunt afișate butoanele de tip „Followers”, „Following”, „Posts” și „Shop”. Dacă utilizatorul logat vizitează profilul altcuiva, apare și un buton suplimentar care permite comutarea între stările „Start Follow” și „Stop Follow”, în funcție de relația actuală dintre cei doi.

Imediat după afișarea profilului, în useEffect, este apelată o funcție care verifică dacă utilizatorul curent (loggedInUserId) îl urmărește deja pe cel vizualizat (user.user_id). Acest lucru se face printr-un request GET către endpoint-ul:

```
try {
  const res = await fetch(
    `http://localhost:4000/follows/check?follower_id=${loggedInUserId}&following_id=${user.user_id}`
  );
  const data = await res.json();
  setIsFollowing(!!data.isFollowing);
} catch (err) {
  console.error("Error checking follow status", err);
}
```

Secvență de cod 4.41 Verificarea relației de urmărire dintre utilizatori

Dacă serverul răspunde cu isFollowing: true, butonul va afișa „Stop Follow”. Atunci când utilizatorul apasă acest buton, se declanșează „handleFollowToggle”, care decide dacă trebuie adăugată sau eliminată urmărirea. În funcție de starea actuală (isFollowing), se trimite fie o cerere POST (pentru a începe urmărirea), fie DELETE (pentru a opri urmărirea). Cererea conține un body cu follower_id și following_id, care sunt transmise ca date JSON. În cazul POST se adaugă o nouă înregistrare în tabelul follows, iar data este setată automat cu NOW(), iar în cazul unui DELETE se elimină perechea follower_id –

following_id din baza de date. Se evită duplicatele cu ON CONFLICT DO NOTHING. Dacă operația reușește, aplicația actualizează starea locală isFollowing și apoi face un nou request către GET „/users/:user_id”, pentru a obține un profil actualizat cu noul număr de urmăritori (followers_count).

```
useEffect(() => {  
  fetchFollowStatus();  
}, [loggedInUserId, user?.user_id, isOwner, isAdmin]);  
  
const handleFollowToggle = async () => {  
  if (!loggedInUserId || !user?.user_id || isAdmin) return;  
  
  const method = isFollowing ? "DELETE" : "POST";  
  
  try {  
    const res = await fetch("http://localhost:4000/follows", {  
      method,  
      headers: { "Content-Type": "application/json" },  
      body: JSON.stringify({  
        follower_id: loggedInUserId,  
        following_id: user.user_id,  
      }),  
    });  
  
    if (!res.ok) return;  
  
    const newFollowState = !isFollowing;  
    setIsFollowing(newFollowState);  
  
    const updatedUserRes = await fetch(  
      `http://localhost:4000/users/${user.user_id}`  
    );  
    const updatedUser = await updatedUserRes.json();  
    setUser(updatedUser);  
  } catch (err) {  
    console.error("Error toggling follow:", err);  
  }  
};
```

Secvență de cod 4.42 Adăugarea și eliminarea relațiilor de urmărire (follow/unfollow)

Dacă utilizatorul vizualizat nu este nici logat, nici administrator, în colțul pozei de profil este afișată o pictogramă cu un steag, care deschide un modal de tip „ReportUserModal”.

Acest modal conține un câmp textarea pentru descrierea motivului raportării și două butoane: „Send” și „Cancel”. La apăsarea „Send”, se verifică dacă motivul este completat și se trimite un POST către endpoint-ul „users/report”. Dacă utilizatorul a fost deja raportat în aceeași zi, serverul răspunde cu 409 Conflict, iar în frontend este afișat un mesaj de tip toast: "You've already reported this user today." În caz de succes, serverul inserează o nouă înregistrare în tabela user_reports și returnează status 200 OK.

```
router.post("/report", async (req, res) => {
  const { reporter_id, reported_id, reason } = req.body;

  if (!reporter_id || !reported_id || !reason) {
    return res.status(400).json({ message: "Incomplete report data." });
  }

  try {
    const existing = await pool.query(
      `SELECT 1 FROM user_reports
       WHERE reporter_id = $1
         AND reported_id = $2
         AND created_at::date = CURRENT_DATE`,
      [reporter_id, reported_id]
    );

    if (existing.rows.length > 0) {
      return res.status(409).json({ message: "You have already reported this user today." });
    }

    await pool.query(
      `INSERT INTO user_reports (reporter_id, reported_id, reason)
       VALUES ($1, $2, $3)`,
      [reporter_id, reported_id, reason]
    );

    res.status(200).json({ message: "Report submitted successfully." });
  } catch (err) {
    console.error("Error submitting report:", err);
    res.status(500).json({ message: "Internal server error." });
  }
});
```

Secvență de cod 4.43 Endpoint backend pentru raportarea unui utilizator

4.2.4 Pagina Discover – explorare de postări și produse cu filtre dinamice

Componenta React DiscoverPosts.js conține logica de afișare a postărilor și de filtrare. Aceasta include:

- Bara de filtre (FiltersBar.js) cu opțiuni: „Most Recent”, „Most Liked” și „Followed Users Only”.
- Inputuri de căutare: unul pentru nume/descriere și unul pentru categoria postării (cu sugestii dinamice).
- Grid-ul cu postările filtrate (PostCard).

Atunci când utilizatorul modifică filtrul activ, introduce un termen de căutare sau selectează o categorie, componenta DiscoverPosts generează automat un nou request către backend. Acest request include toți parametrii actuali: identificatorul utilizatorului, textul de căutare, categoria selectată și tipul de filtrare (ex. cele mai recente, cele mai apreciate sau doar de la utilizatori urmăriți). Ca răspuns, serverul returnează o listă actualizată de postări, care sunt apoi afișate în interfață printr-un grid de tip „PostCard”.

Pe partea de backend, funcționalitatea este gestionată de endpoint-ul GET „/discover/posts”. Acesta procesează toate informațiile primite și determină ce postări sunt relevante pentru utilizatorul curent. Dacă utilizatorul a introdus un termen de căutare de cel puțin 3 caractere, acest termen este salvat în tabela post_search_history, cu condiția să nu fie identic cu ultima căutare recentă.

```
try {
  // Salvăm în istoricul de căutare
  if (search.length >= 3) {
    const lastSearch = await pool.query(
      `SELECT search_text FROM post_search_history
      WHERE user_id = $1 ORDER BY searched_at DESC LIMIT 1`,
      [userId]
    );
    const lastQuery = lastSearch.rows[0]?.search_text?.toLowerCase();
    if (lastQuery !== search) {
      await pool.query(
        `INSERT INTO post_search_history (user_id, search_text, searched_at)
        VALUES ($1, $2, NOW())`,
        [userId, search]
      );
    }
  }
}
```

Secvență de cod 4.44 Salvarea căutărilor utilizatorului în istoricul postărilor

Serverul pregătește vectorii values și whereParts, generând condițiile de filtrare în funcție de parametri de căutare, categorie, follows sau sugestii personalizate.

```
let values = [];
let whereParts = [];

// Followed + categorii proprii + istoric (dacă nu e search/categorie)
if (!search && !categoryName) {
  if (selectedFilter === "followed") {
    values.push(userId);
    whereParts.push(`p.user_id IN (
      SELECT following_id FROM follows WHERE follower_id = ${values.length}
    )`);
  } else {
    const suggestionParts = [];

    // FOLLOW
    const followOffset = values.length + 1;
    let logic = `p.user_id IN (
      SELECT following_id FROM follows WHERE follower_id = ${followOffset}
    )`;
    values.push(userId);

    // Categorii proprii
    const catRes = await pool.query(
      `SELECT DISTINCT category_id FROM posts
      WHERE user_id = $1 AND category_id IS NOT NULL`,
      [userId]
    );

    const categoryIds = catRes.rows.map(r => r.category_id);
    if (categoryIds.length > 0) {
      const catPlaceholders = categoryIds.map((_, i) => `${values.length + i + 1}`).join(", ");
      logic += ` OR p.category_id IN (${catPlaceholders})`;
      values.push(...categoryIds);
    }

    suggestionParts.push(`${logic}`);
  }
}

// Istoric de căutare
const history = await pool.query(
  `SELECT search_text FROM post_search_history
  WHERE user_id = $1
  GROUP BY search_text
  ORDER BY MAX(searched_at) DESC
  LIMIT 3`,
  [userId]
);

const keywords = history.rows.map(r => `${r.search_text.toLowerCase()}%`);
if (keywords.length > 0) {
  const keywordOffset = values.length;
  const keywordConditions = keywords.map((_, i) => `
    LOWER(p.content) LIKE ${keywordOffset + i + 1}
    OR LOWER(u.first_name || ' ' || u.last_name) LIKE ${keywordOffset + i + 1}
  `);
  values.push(...keywords);
  suggestionParts.push(`${keywordConditions.join(" OR ")}`);
  whereParts.push(`${suggestionParts.join(" OR ")}`);
}

// Căutare globală
if (search) {
  values.push(`${search}%`);
  whereParts.push(
    `
    LOWER(p.content) LIKE ${values.length}
    OR LOWER(u.first_name || ' ' || u.last_name) LIKE ${values.length}
  `);
}

// Filtru după categorie
if (categoryName) {
  values.push(`${categoryName}%`);
  whereParts.push(
    `
    SELECT 1 FROM categories c
    WHERE c.category_id = p.category_id
    AND LOWER(c.name) LIKE ${values.length}
  `);
}

// Doar follow (explicit)
if (selectedFilter === "followed") {
  values.push(userId);
  whereParts.push(`p.user_id IN (
    SELECT following_id FROM follows WHERE follower_id = ${values.length}
  )`);
}

// excludem postările proprii
values.push(userId);
whereParts.push(`p.user_id != ${values.length}`);
```

Secvență de cod 4.45 Construirea condițiilor de filtrare SQL în funcție de căutare, categorie, follow și sugestii personalizate

După ce toate filtrele sunt pregătite, serverul construiește interogarea completă SQL, execută selecția postărilor și adaugă media atașată fiecărei postări.

```
let query = `
  SELECT
    p.post_id,
    p.user_id,
    p.content,
    p.created_at,
    p.category_id,
    u.first_name,
    u.last_name,
    u.profile_picture,
    COALESCE(l.like_count, 0) AS like_count
  FROM posts p
  JOIN accounts u ON u.user_id = p.user_id
  LEFT JOIN (
    SELECT post_id, COUNT(*) AS like_count
    FROM likes
    GROUP BY post_id
  ) l ON l.post_id = p.post_id
`;

if (whereParts.length > 0) {
  query += ` WHERE ${whereParts.join(" AND ")} `;
}

if (selectedFilter === "liked") {
  query += ` ORDER BY like_count DESC, p.created_at DESC `;
} else {
  query += ` ORDER BY p.created_at DESC `;
}

query += ` LIMIT 50 `;

const postsResult = await pool.query(query, values);
const posts = postsResult.rows;

// Media
const enrichedPosts = await Promise.all(
  posts.map(async (post) => {
    const media = await pool.query(
      `SELECT file_url, file_type FROM post_media
      WHERE post_id = $1 ORDER BY media_id ASC`,
      [post.post_id]
    );
    return {
      ...post,
      media_urls: media.rows.map(m => ({
        url: m.file_url,
        type: m.file_type,
      })),
    };
  })
);

res.json(enrichedPosts);
} catch (err) {
  console.error("Error fetching discover posts:", err);
  res.status(500).json({ error: "Internal server error" });
}
```

Secvență de cod 4.46 Construirea interogării SQL finale, executarea ei și atașarea media la postări

Pentru a oferi o experiență personalizată și coerentă, postările create de utilizatorul curent sunt automat excluse din rezultate.

```
values.push(userId);
whereParts.push(`p.user_id != ${values.length}`);
```

Secvență de cod 4.47 Excluderea postărilor proprii din feedul utilizatorului

După aplicarea filtrelor, interogarea returnează conținutul fiecărei postări (textul), data creării, informații despre autor (nume, poză de profil), precum și numărul de aprecieri, obținut printr-o agregare a datelor din tabela likes.

Pentru fiecare postare în parte, sistemul extrage separat fișierele media (imagini sau videoclipuri) asociate din tabela post_media, rezultând un obiect complet, care include toate datele necesare pentru afișarea postării în interfață.

Funcționalitatea pentru descoperirea produselor reutilizează aceeași structură și mecanism logic ca în cazul postărilor, adaptată însă pentru secțiunea de marketplace din aplicație.

Filtrele disponibile sunt specifice produselor și includ: cele mai recente adăugate, doar de la vânzători urmăriți, produse cu preț crescător sau descrescător.

```
2 export const postFilters = [
3   { id: "recent", label: "Most Recent" },
4   { id: "liked", label: "Most Liked" },
5   { id: "followed", label: "Followed Users Only" },
6 ];
7
8 export const productFilters = [
9   { id: "recent", label: "Most Recent" },
10  { id: "followed", label: "Followed Sellers Only" },
11  { id: "price_asc", label: "Price: Low to High" },
12  { id: "price_desc", label: "Price: High to Low" }
13 ];
14
```

Secvență de cod 4.48 Definirea filtrelor disponibile pentru postări și produse

Tabelele folosite în backend sunt diferite: în loc de posts și post_media, sunt utilizate marketplace_items și shop_item_images.

Istoricul de căutare și sugestiile inteligente sunt salvate în tabela product_search_history, iar sugestiile implicite sunt bazate pe: produsele vânzătorilor urmăriți, categoriile în care utilizatorul a publicat produse anterior, cuvinte-cheie regăsite în căutările sale recente (doar dacă nu este introdus un search activ).

Produsele proprii ale utilizatorului sunt excluse automat din rezultate, pentru a favoriza descoperirea de conținut nou.

La fel ca în cazul postărilor, backend-ul adaugă separat imaginile asociate fiecărui produs, interogând tabela shop_item_images pentru fiecare item_id. Rezultatul final este o listă de produse completate cu titlu, descriere, preț, disponibilitate, informații despre vânzător și imaginile asociate, toate transmise frontend-ului într-un singur răspuns.

4.2.5 Sistemul de notificări (like, follow, comentarii)

Pentru aprecieri, urmăriri și comentarii, aplicația folosește o tabelă dedicată în baza de date: appreciation_notifications, care stochează evenimentele și le marchează drept văzute sau nevăzute. Backendul include o funcție specială „addAppreciationNotification” care gestionează atât salvarea cât și transmiterea în timp real a notificărilor către frontend.

```
// Trimitere notificare de tip "follow"
await addAppreciationNotification({
  user_id: following_id,
  sender_id: follower_id,
  type: "follow",
  io,
});

if (postOwnerId !== user_id) {
  await addAppreciationNotification({
    user_id: postOwnerId,
    sender_id: user_id,
    type: "like",
    post_id: post_id,
    io: req.io
  });
}

//Trmite notificare doar dacă nu comentează pe propria postare
if (postOwnerId && postOwnerId !== user_id) {
  await addAppreciationNotification({
    user_id: postOwnerId,
    sender_id: user_id,
    type: "comment",
    post_id: post_id,
    io,
  });
}
```

Secvență de cod 4.49 Generarea notificărilor de apreciere (follow, like, comment)

Dacă acțiunea este de tip like sau follow, se verifică dacă există deja o notificare recentă de același tip și se înlocuiește dacă e cazul. Dacă notificarea este validă, este transmisă către frontend printr-un canal Socket.IO specific utilizatorului.

```

async function addAppreciationNotification({ user_id, sender_id, type, post_id = null, io = null }) {
  try {
    const now = new Date();

    let THRESHOLD_MINUTES = 0;
    if (type === "follow" || type === "like") {
      THRESHOLD_MINUTES = 60;
    } else if (type === "comment") {
      THRESHOLD_MINUTES = 0;
    }

    const existingNotif = await pool.query(
      `SELECT id, created_at FROM appreciation_notifications
      WHERE user_id = $1 AND sender_id = $2 AND type = $3 AND post_id IS NOT DISTINCT FROM $4
      ORDER BY created_at DESC
      LIMIT 1`,
      [user_id, sender_id, type, post_id]
    );

    let skipEmit = false;
    let notificationId = null;

    if (existingNotif.rows.length > 0) {
      const { id, created_at } = existingNotif.rows[0];
      const existingDate = new Date(created_at);
      const diffMinutes = (now - existingDate) / (1000 * 60);

      if (diffMinutes < THRESHOLD_MINUTES) {
        console.log(`Skipped ${type} notification: already exists recently`);
        return;
      }
    }

    await pool.query(`DELETE FROM appreciation_notifications WHERE id = $1`, [id]);

    const insertResult = await pool.query(
      `INSERT INTO appreciation_notifications (user_id, sender_id, type, post_id)
      VALUES ($1, $2, $3, $4)
      RETURNING id`,
      [user_id, sender_id, type, post_id]
    );
    notificationId = insertResult.rows[0].id;

    console.log(`Inserted ${type} notification:`, notificationId);

    if (io) {
      const userResult = await pool.query(
        `SELECT first_name, last_name, profile_picture FROM accounts WHERE user_id = $1`,
        [sender_id]
      );

      io.to(`user-${user_id}`).emit("new_appreciation", {
        type,
        sender_id,
        post_id,
        created_at: now,
        sender_first_name: userResult.rows[0].first_name,
        sender_last_name: userResult.rows[0].last_name,
        sender_avatar: userResult.rows[0].profile_picture,
      });
    }
  } catch (err) {
    console.error("Error handling appreciation notification:", err);
    throw err;
  }
}

```

Secvență de cod 4.50 Gestionarea notificărilor de apreciere și emiterea în timp real (backend)

În frontend, componenta useAppreciationNotifications.js ascultă evenimentele WebSocket și actualizează atât badge-ul cu numărul de notificări nevăzute, cât și afișarea unei casete temporare în colțul ecranului.

```

useEffect(() => {
  if (!socket) return;

  const handleNewAppreciation = (data) => {
    console.log("New appreciation received via socket:", data);
    setLastNotif(data);

    fetchUnseenAppreciations();
  };

  socket.on("new_appreciation", handleNewAppreciation);

  return () => {
    socket.off("new_appreciation", handleNewAppreciation);
  };
}, [socket, fetchUnseenAppreciations]);

return { count, lastNotif, refreshAppreciations: fetchUnseenAppreciations };

```

Secvență de cod 4.51 Recepționarea notificărilor de apreciere în timp real (frontend)

În NavBar se gestionează afișarea notificării cu mesajul personalizat în funcție de tipul acesteia.

```

useEffect(() => {
  console.log("Notificare apreciere primită:", lastNotif);
  if (!lastNotif || lastNotif.sender_id !== Number(userIdRef.current)) return;

  notificationSoundRef.current.play().catch((e) => {
    console.warn("Sunet blocat de browser până la interacțiune.", e);
  });

  setShowAppreciationNotif(true);
  setLiveNotifData({
    senderName: `${lastNotif.sender_first_name} || ""` + (lastNotif.sender_last_name || "").trim() || "Someone",
    message:
      lastNotif.type === "like"
        ? "liked your post"
        : lastNotif.type === "follow"
        ? "started following you"
        : lastNotif.type === "comment"
        ? "commented on your post"
        : "interacted with you",
    userId: lastNotif.sender_id,
    senderProfilePic: lastNotif.sender_avatar || null,
  });

  const timeout = setTimeout(() => {
    setShowAppreciationNotif(false);
    setLiveNotifData(null);
  }, 5000);

  return () => clearTimeout(timeout);
}, [lastNotif]);

```

Secvență de cod 4.52 Afișarea notificării vizuale pentru aprecieri primite

Toate notificările de tip follow/like/comment sunt salvate în baza de date și pot fi revizualizate în pagina „Appreciation”. Aceasta încarcă datele din endpointul „/appreciation/list”, iar după ce utilizatorul accesează pagina, notificările sunt marcate ca văzute.

```
useEffect(() => {  
  if (!userId) return;  
  //console.log("userId in AppreciationPage:", userId);  
  
  async function fetchAppreciations() {  
    try {  
      const res = await fetch(`http://localhost:4000/appreciation/list?user_id=${userId}`);  
      const data = await res.json();  
  
      if (Array.isArray(data)) {  
        setAppreciations(data);  
      } else {  
        console.error("Unexpected data format:", data);  
        setAppreciations([]);  
      }  
    } catch (err) {  
      console.error("Error fetching appreciations:", err);  
      setAppreciations([]);  
    } finally {  
      setLoading(false);  
    }  
  }  
  
  fetchAppreciations();  
  
  fetch(`http://localhost:4000/appreciation/mark-as-seen?user_id=${userId}`, {  
    method: "POST",  
  }).catch((err) => console.error("Error marking as seen:", err));  
}, [userId]);
```

Secvență de cod 4.53 Afișarea și marcarea notificărilor de apreciere ca văzute

4.2.6 Funcționalități comerciale

Componenta principală care gestionează adăugarea produselor este ProductForm. Aceasta afișează inputuri pentru titlu, descriere, preț, imagini și categorie. La completare, datele sunt reținute în variabile de stare controlate, iar imaginile ce sunt încărcate pot fi previzualizate înainte de trimitere, logica fiind similară cu cea a adăugării unei postări.

```
<div className={productStyles.priceWrapper}>  
  <span className={productStyles.euroSymbol}>€</span>  
  <input  
    type="number"  
    min="0"  
    placeholder="Price"  
    value={price}  
    onChange={(e) => setPrice(e.target.value)}  
    className={productStyles.formInput}/>  
  </div>  
</div>  
  
<textarea  
  placeholder="Description"  
  value={description}  
  onChange={(e) => setDescription(e.target.value)}  
  className={productStyles.formTextarea}/>  
</div>  
  
<div className={styles.postOptions}>  
  <label className={styles.postOption}>  
    Add Image  
    <input type="file" accept="image/*" multiple hidden onChange={handleFileChange} />  
  </label>  
</div>  
  
{fileError && (  
  <div className={styles.errorBorder}>  
    <span>Please upload at least one image.</span>  
  </div>  
)}
```

Secvență de cod 4.54 Formular pentru adăugarea unui produs – preț, descriere și imagini

La apăsarea butonului „Add Product”, se efectuează validări: titlul nu trebuie să fie gol, prețul sa nu fie zero, cel puțin o imagine trebuie selectată, iar categoria trebuie să fie validată față de lista existentă. Dacă toate sunt în regulă, datele sunt transmise sub forma unui FormData către backend. În backend, endpoint-ul POST „/shop/add-product” primește datele și le inserează în baza de date în două etape: salvarea metadatelor produsului în tabela marketplace_items și încărcarea imaginilor în Cloudinary și salvarea linkurilor, comportament similar este implementat și în cazul postărilor.

După ce produsul este adăugat cu succes, aplicația frontend reface lista de produse ale utilizatorului, apelând endpoint-ul GET „/shop/user-products/:userId”. Acesta returnează produsele împreună cu imaginile lor, preluate separat din tabela shop_item_images și atașate la fiecare produs.

```
const handleAddProduct = async ({ title, description, price, files, category_id }) => {
  const formData = new FormData();
  formData.append("user_id", userId);
  formData.append("title", title);
  formData.append("description", description);
  formData.append("price", price);
  if (category_id) {
    formData.append("category_id", category_id);
  }
  files.forEach((file) => formData.append("images", file));

  try {
    setIsPostingProduct(true);

    const response = await fetch("http://localhost:4000/shop/add-product", {
      method: "POST",
      body: formData,
    });

    const result = await response.json();

    if (response.ok) {
      const updated = await fetch(`http://localhost:4000/shop/user-products/${userId}`);
      const updatedData = await updated.json();
      setProducts(updatedData.map((p) => ({ ...p, currentIndex: 0 })));
    } else {
      console.error("Error:", result.error);
    }
  } catch (err) {
    console.error("Upload error:", err);
  } finally {
    setIsPostingProduct(false);
  }
};
```

Secvență de cod 4.55 Trimiterea datelor unui produs nou către backend și actualizarea produselor afișate

Procesul de editare începe din componenta ProductCard.js, care verifică dacă utilizatorul curent este proprietarul produsului. În cazul în care este, în colțul din dreapta sus al fiecărui card apare un buton de meniu cu opțiunea „Edit”. La apăsarea acestui buton, produsul respectiv intră în mod de editare (isEditing=true), iar conținutul său devine interactiv.

În mod de editare, componentele div obișnuite sunt înlocuite cu elemente de tip input, textarea, select, iar imaginile existente sunt afișate cu un buton „x” care permite marcarea lor pentru ștergere. De asemenea, în caruselul de imagini, dacă produsul este în editare, apare și un buton „+” la final, care deschide un input de fișiere pentru adăugarea unor imagini noi. Toate aceste modificări sunt reținute în state-uri locale precum title, description, price, stock, newFiles, imagesToDelete.

```
router.put("/update-product", upload.array("newImages"), async (req, res) => {
  const { item_id, title, description, price, stock, imagesToDelete } = req.body;

  if (!item_id || !title || !price) {
    return res.status(400).json({ error: "Missing required fields" });
  }

  try {
    // Actualizare date
    await pool.query(
      `UPDATE marketplace_items
      SET title = $1, description = $2, price = $3, stock = $4
      WHERE item_id = $5`,
      [title, description, price, stock || "yes", item_id]
    );

    const toDelete = JSON.parse(imagesToDelete || "[]");
    for (const url of toDelete) {
      const publicId = extractPublicIdFromUrl(url);
      if (publicId) {
        try {
          await cloudinary.uploader.destroy(publicId);
        } catch (err) {
          console.warn("Cloudinary deletion failed:", publicId, err.message);
        }

        await pool.query(
          `DELETE FROM shop_item_images WHERE item_id = $1 AND image_url = $2`,
          [item_id, url]
        );
      }
    }

    const files = req.files || [];
    for (const file of files) {
      const base64 = `data:${file.mimetype};base64,${file.buffer.toString("base64")}`;
      const uploadResult = await cloudinary.uploader.upload(base64, {
        folder: "shop_items",
        public_id: uuidv4(),
      });

      await pool.query(
        `INSERT INTO shop_item_images (item_id, image_url)
        VALUES ($1, $2)`,
        [item_id, uploadResult.secure_url]
      );
    }

    res.json({ message: "Product updated" });
  } catch (err) {
    console.error("Update error:", err);
    res.status(500).json({ error: "Failed to update product" });
  }
});
```

```
try {
  const res = await fetch("http://localhost:4000/shop/update-product", {
    method: "PUT",
    body: formData,
  });

  if (res.ok) {
    window.location.reload();
  } else {
    toast.error("Failed to save changes", {
      className: styles.customToast,
      bodyClassName: styles.customToastBody,
      position: "bottom-right",
      autoClose: 2500
    });
  }
}
```

Secvență de cod 4.58 Trimiterea modificărilor produsului și afișarea feedback-ului vizual

În componenta ProductCard.js, dacă utilizatorul este proprietarul produsului, apare un meniu contextual cu opțiunea „Delete”. La selectarea acestei opțiuni, se apelează funcția „onDeleteProduct”, transmisă ca prop.

```
import React, { useState, useRef, useEffect } from 'react';
import { FaEllipsisV } from 'react-icons/fa';
import styles from '../CSSfiles/ShopPage.module.css';

export default function ProductMenuButton({ productId, isOwner, onDelete, onEdit, onReport, reportedUserId }) {
  const [menuOpen, setMenuOpen] = useState(false);
  const menuRef = useRef(null); // pt a detecta clickurile inafara meniului

  useEffect(() => {
    const handleClickOutside = (event) => {
      if (menuRef.current && !menuRef.current.contains(event.target)) { //daca meniul este deschis si click-ul
        setMenuOpen(false);
      }
    };
    document.addEventListener('click', handleClickOutside);
    return () => document.removeEventListener('click', handleClickOutside);
  }, []);

  return (
    <div className={styles.productMenuButton} >
      <ProductMenuButton
        productId={product.item_id}
        isOwner={isOwner}
        onDelete={onDeleteProduct}
        onEdit={() => setEditingProduct(product.item_id)}
        onReport={handleReportProduct}
        reportedUserId={product.user_id}
      />
    </div>
  );
}
```

```
<div className={styles.productsGrid}>
  {products.map((product) => ( //pentru fiecare produs din array
    <ProductCard
      key={product.item_id}
      product={product}
      isOwner={isOwner}
      isEditing={editingProductId === product.item_id}
      setEditing={setEditingProductId}
      onNext={handleNext}
      onPrev={handlePrev}
      onDeleteProduct={handleDeleteProduct}
      onEditProduct={handleEditProduct}
      id={product.item_id}
    />
  ))}
</div>
```

ProductMenuButton.js ProductCard.js ShopPage.js

Secvență de cod 4.59 Meniu contextual pentru produse: editare, ștergere și raportare

În ShopPage.js, funcția handleDeleteProduct gestionează acțiunea de ștergere. Această trimite un request DELETE către backend și, în caz de succes, actualizează lista de produse afișate în interfață.

```
const handleDeleteProduct = async (productId) => {
  try {
    const res = await fetch(`http://localhost:4000/shop/delete-product/${productId}`, {
      method: "DELETE"
    });
    if (res.ok) {
      setProducts((prev) => prev.filter((p) => p.item_id !== productId));
      showToast("Post deleted successfully!");
    } else {
      console.error("Delete failed");
      showToast("Failed to delete post.");
    }
  }
}
```

Secvență de cod 4.60 Ștergerea unui produs din shop și actualizarea interfeței

În backend ștergerea este efectuată prin rută „/shop/delete-product/\${productId}” din shop.js, care parcurge mai mulți pași pentru a elimina complet toate datele asociate. Mai întâi, sunt șterse referințele către produs din tabele precum shopping_cart, favorites și product_reports. Apoi, imaginile produsului sunt extrase din baza de date, iar public_id-urile sunt obținute din URL folosind „extractPublicIdFromUrl” pentru a fi șterse de pe Cloudinary.

După eliminarea fișierelor media, înregistrările din `shop_item_images` sunt șterse, iar în final, produsul este eliminat din tabela `marketplace_items`. Dacă nu este găsit, se returnează un răspuns 404, altfel un mesaj de confirmare.

Funcționalitatea de adăugare și eliminare a produselor în/din coșul de cumpărături și lista de favorite este implementată într-un mod unitar, cu apeluri către backend și actualizare automată a interfeței prin context global (`CartContext`, `FavoritesContext`).

Funcțiile `fetchCartItems()` și `fetchCartCount()` sunt responsabile pentru sincronizarea coșului de cumpărături cu baza de date și actualizarea interfeței. Ambele trimit un request GET către `/cart/user-cart/:userId`: prima preia produsele și le grupează după vânzător, salvându-le în `cartItems`, iar a doua calculează totalul cantităților și actualizează `cartCount`, afișat în `Navbar`. Ele sunt apelate automat după adăugare (`/shop/add-to-cart`), ștergere (`/cart/delete/:cartId`) sau modificare cantitate (`/cart/update-quantity`), astfel încât interfața reflectă în timp real starea coșului.

Funcționalitatea pentru favorite urmează aceeași logică, cu `fetchFavoritesItems()` și `fetchFavoritesCount()` conectate la endpointul `/favorites/user-favorites/:userId`. Acestea aduc și grupează produsele favorite, respectiv actualizează numărul total (`favoritesCount`). La adăugare sau ștergere (`/favorites/add`, `/favorites/delete/:favoriteId`), informațiile se reîmprospătează automat în context, iar `Navbar` afișează întotdeauna datele corecte, fără a fi nevoie de refresh manual.

```
const fetchCartItems = async () => {
  if (!userId) return;
  try {
    const res = await fetch(`http://localhost:4000/cart/user-cart/${userId}`);
    const data = await res.json();
    const grouped = {};
    data.forEach(item => {
      if (!grouped[item.seller_id]) {
        grouped[item.seller_id] = {
          sellerName: item.seller_name,
          items: [],
        };
      }
      grouped[item.seller_id].items.push(item);
    });
    setCartItems(grouped);
  } catch (err) {
    console.error("Error fetching cart items:", err);
  }
};

const fetchCartCount = async () => {
  if (!userId) return;
  try {
    const res = await fetch(`http://localhost:4000/cart/user-cart/${userId}`);
    const data = await res.json();
    const totalItems = data.reduce((sum, item) => sum + item.quantity, 0);
    setCartCount(totalItems);
  } catch (err) {
    console.error("Error fetching cart count:", err);
  }
};

const fetchFavoritesCount = async () => {
  if (!userId) return;
  try {
    const res = await fetch(`http://localhost:4000/favorites/user-favorites/${userId}`);
    const data = await res.json();
    setFavoritesCount(data.length);
  } catch (err) {
    console.error("Error fetching favorites count:", err);
  }
};

const fetchFavoritesItems = async () => {
  if (!userId) return;
  try {
    const res = await fetch(`http://localhost:4000/favorites/user-favorites/${userId}`);
    const data = await res.json();
    const grouped = {};
    data.forEach(item => {
      if (!grouped[item.seller_id]) {
        grouped[item.seller_id] = {
          sellerName: item.seller_name,
          items: [],
        };
      }
      grouped[item.seller_id].items.push(item);
    });
    setFavoritesItems(grouped);
    setFavoritesCount(data.length); // actualizezi și countul
  } catch (err) {
    console.error("Error fetching favorites:", err);
  }
};
```

CartContext.js

FavoritesContext.js

Secvență de cod 4.61 Obținerea produselor și a numărului de articole din coș și favorite

În componenta `ProductCard`, butoanele de „Add to Cart” și „Add to Favorites” sunt disponibile doar pentru utilizatorii autentificați și doar pentru produsele altor utilizatori.

```
const handleAddToCart = async (productId) => {
  const user_id = localStorage.getItem("user_id");
  if (!user_id) {
    toast.error("You need to log in to add to cart.", {
      className: styles.customToast,
      bodyClassName: styles.customToastBody,
      position: "bottom-right",
      autoClose: 2500
    });
    return;
  }

  try {
    const res = await fetch("http://localhost:4000/shop/add-to-cart", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ user_id, product_id: productId, quantity: 1 })
    });

    const data = await res.json();
    if (res.ok) {
      toast.success("🛒 Product added to cart!", {
        className: styles.customToast,
        bodyClassName: styles.customToastBody,
        position: "bottom-right",
        autoClose: 2500
      });
      fetchCartCount();
    } else {
      toast.error(data.error || "Error adding to cart.", {
        className: styles.customToast,
        bodyClassName: styles.customToastBody,
        position: "bottom-right",
        autoClose: 2500
      });
    }
  } catch (err) {
    console.error("Error adding to cart:", err);
  }
};

const handleAddToFavorites = async () => {
  const user_id = localStorage.getItem("user_id");
  if (!user_id) {
    toast.error("You need to log in to add to favorites.", {
      className: styles.customToast,
      bodyClassName: styles.customToastBody,
      position: "bottom-right",
      autoClose: 2500
    });
    return;
  }

  try {
    const res = await fetch("http://localhost:4000/favorites/add", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        user_id,
        item_id: product.item_id,
        seller_id: product.user_id
      })
    });

    const data = await res.json();
    if (res.ok) {
      toast.success("❤️ Added to favorites!", {
        className: styles.customToast,
        bodyClassName: styles.customToastBody,
        position: "bottom-right",
        autoClose: 2500
      });
      fetchFavoritesCount();
    } else {
      toast.error(data.error || "Error adding to favorites.", {
        className: styles.customToast,
        bodyClassName: styles.customToastBody,
        position: "bottom-right",
        autoClose: 2500
      });
    }
  } catch (err) {
    console.error("Error adding to favorites:", err);
  }
};
```

Secvență de cod 4.62 Adăugarea unui produs în coș sau la favorite și afișarea notificărilor

În ambele cazuri, după adăugare, se apelează funcțiile `fetchCartCount()` și `fetchFavoritesCount()` din context.

Operațiile de ștergere sunt gestionate simetric, printr-un buton de tip coș implementat în fișierele `CartProductList.js`, `FavoriteProductList.js` care trimit un DELETE către backend:

- `/cart/delete/:cartId` pentru eliminarea unui produs din coș
- `/favorites/delete/:favoriteId` pentru eliminarea din favorite

După ștergere se apelează metodele `fetchCartCount()`, `fetchCartItems()`, `fetchFavoritesCount()` și `fetchFavoritesItems()` pentru a menține interfața sincronizată.

```
const handleDelete = async (cartId) => {
  try {
    const res = await fetch(`http://localhost:4000/cart/delete/${cartId}`, {
      method: "DELETE",
    });

    if (res.ok) {
      fetchCartItems();
      fetchCartCount();
    }
  } catch (err) {
    console.error("Error deleting cart item:", err);
  }
};

const handleDeleteFavorite = async (favId) => {
  try {
    const res = await fetch(`http://localhost:4000/favorites/delete/${favId}`, {
      method: "DELETE",
    });

    if (res.ok) {
      fetchFavoritesItems();
      fetchFavoritesCount();
    }
  } catch (err) {
    console.error("Error deleting favorite:", err);
  }
};
```

CartProductList.js

FavoriteProductList.js

Secvență de cod 4.63 Ștergerea produselor din coș și din favorite

Funcționalitatea de modificare a cantității este disponibilă în `CartProductList.js`, prin două butoane „+” și „-”, care apelează funcția „`handleQuantityChange`” menită să declanșeze actualizarea cantității în baza de date.

```
const handleQuantityChange = async (cartId, newQuantity) => {  
  if (newQuantity < 1) return;  
  
  try {  
    const res = await fetch("http://localhost:4000/cart/update-quantity", {  
      method: "PUT",  
      headers: { "Content-Type": "application/json" },  
      body: JSON.stringify({ cart_id: cartId, quantity: newQuantity }),  
    });  
  
    if (res.ok) {  
      fetchCartItems();  
      fetchCartCount();  
    }  
  } catch (err) {  
    console.error("Error updating quantity:", err);  
  }  
};
```

Secvență de cod 4.64 Actualizarea cantității unui produs din coș

Plasarea unei comenzi pornește din componenta CartSummary.js, unde utilizatorul inițiază procesul apăsând butonul „Shipping Details”, buton care deschide ShippingModal.js, unde este calculat costul de livrare pentru fiecare vânzător în parte. Pe baza orașului și țării setate în profilul utilizatorului, aplicația determină coordonatele geografice ale destinației, folosind serviciul OpenRouteService.

Pentru fiecare grup de produse din coș, aplicația extrage locația vânzătorului și determină costul de livrare în funcție de câteva reguli. Dacă valoarea totală a produselor comandate de la un vânzător depășește pragul de 100€, livrarea este gratuită. În cazul în care vânzătorul și cumpărătorul se află în aceeași țară, se aplică un tarif fix de 5€.

Pentru comenzile internaționale sub pragul gratuit, se determină distanța exactă dintre vânzător și cumpărător cu ajutorul endpoint-ului „v2/matrix/driving-car” al OpenRouteService. Pe baza acestei distanțe, aplicația calculează prețul livrării pornind de la o valoare de bază (15€), la care se adaugă un factor proporțional cu distanța (+ 5€/500km). Costul este plafonat la 50€, pentru a evita valori exagerate. Dacă serviciul OpenRouteService nu returnează un rezultat valid, aplicația recurge la o formulă Haversine pentru estimarea distanței geografice.

```
const calculateCosts = async () => {  
  setLoading(true);  
  try {  
    const userCoords = await fetchCoords(`${userLocation.city}, ${userLocation.country}`);  
    const results = [];  
  
    for (const sellerId in cartItems) {  
      const seller = cartItems[sellerId].items[0];  
      const sellerCity = seller.seller.city;  
      const sellerCountry = seller.seller.country;  
  
      if (!sellerCity || !sellerCountry) {  
        console.warn("Missing seller location for", seller.seller_name);  
        continue;  
      }  
  
      const sellerPlace = `${sellerCity}, ${sellerCountry}`;  
      const sellerCoords = await fetchCoords(sellerPlace);  
  
      const totalOrderValue = cartItems[sellerId].items.reduce(  
        (sum, item) => sum + item.price * item.quantity,  
        0  
      );  
  
      let distance = null;  
      let price = 0;  
      let note = "";  
  
      const sameCountry = sellerCountry.toLowerCase() === userLocation.country.toLowerCase();  
  
      if (totalOrderValue >= FREE_SHIPPING_THRESHOLD) {  
        price = 0;  
        note = "Free shipping (order over €100)";  
      } else if (sameCountry) {  
        price = 5.00;  
      } else {  
        const res = await fetch("https://api.openrouteservice.org/v2/matrix/driving-car", {  
          method: "POST",  
          headers: {  
            Authorization: API_KEY,  
            "Content-Type": "application/json",  
          },  
          body: JSON.stringify({  
            locations: [[sellerCoords.lng, sellerCoords.lat], [userCoords.lng, userCoords.lat]],  
            metrics: ["distance"],  
            units: "km",  
          })),  
        );  
  
        const data = await res.json();  
        console.log("Matrix API response:", data);  
        distance = data.distances[0][0];  
        if (distance || distance === 0) {  
          console.warn("No valid distance from OMS, using Haversine.");  
          const toRadians = (deg) => (deg * Math.PI) / 180;  
          const R = 6371;  
          const dLat = toRadians(userCoords.lat - sellerCoords.lat);  
          const dLng = toRadians(userCoords.lng - sellerCoords.lng);  
          const a =  
            Math.sin(dLat / 2) ** 2 +  
            Math.cos(toRadians(sellerCoords.lat)) *  
            Math.cos(toRadians(userCoords.lat)) *  
            Math.sin(dLng / 2) ** 2;  
          const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));  
          distance = R * c;  
        }  
        console.log("distance", distance);  
        const basePrice = 15;  
        const factor = 5;  
        price = basePrice + (distance / 500) * factor;  
        if (price > 50) price = 50;  
  
        const remaining = (FREE_SHIPPING_THRESHOLD - totalOrderValue).toFixed(2);  
        note = `Add €${remaining} more for free shipping`;  
      }  
    }  
  } catch (err) {  
    console.error("Error calculating costs:", err);  
  }  
  setLoading(false);  
}
```

Secvență de cod 4.65 Calculul costului de transport pe baza distanței dintre cumpărător și vânzător

După afișarea detaliilor de transport, utilizatorul trece la pasul de plată și livrare, în `PaymentModal.js`. Aici selectează metoda de plată (card sau ramburs) și completează adresa. Pentru metoda cash, funcția `handleSubmit()` salvează comanda direct în baza de date, folosind `createOrdersInDatabase()`. Pentru metoda card, se inițiază o plată Stripe prin „`StripePaymentModal`” și „`CardPaymentForm`”, unde este confirmat un payment intent.

```
const result = await stripe.confirmCardPayment(clientSecret, {
  payment_method: {
    card: elements.getElement(CardElement),
    billing_details: {
      name: fullName,
      address: {
        line1: address.street,
        postal_code: address.zip_code,
        country: countryISO,
      },
    },
  },
});
```

Secvență de cod 4.66 Confirmarea plății cu cardul prin Stripe

Dacă plata reușește, se apelează aceeași funcție „`createOrdersInDatabase`” pentru a salva comenzile.

În backend, ruta POST „`/orders`” primește datele transmise din frontend și salvează câte o comandă distinctă pentru fiecare vânzător implicat în coș. Pentru fiecare vânzător, frontend-ul calculează costul de transport și suma totală a produselor, iar dacă metoda de plată este „card”, plasează această sumă ca valoare plătită (`paid_amount`). Structura comenzii include parametrii: `buyer_id`, `seller_id`, `item_id`, `status`, `payment_method`, `paid_amount`, `total_due`, și detaliile complete ale adresei de livrare.

```
const result = await pool.query(
  `INSERT INTO orders (
    buyer_id, seller_id, item_id, status,
    created_at, payment_method, paid_amount, total_due,
    country, city, state, zip_code, street, address_details
  ) VALUES (
    $1, $2, $3, $4,
    NOW(), $5, $6, $7,
    $8, $9, $10, $11, $12, $13
  ) RETURNING *`,
  [
    buyerId, sellerId, itemId, status,
    payment_method, paidAmount, totalDue,
    country, city, state, zip_code, street, address_details
  ]
);
```

Secvență de cod 4.67 Inserarea unei comenzi în baza de date cu valorile complete ale adresei și RETURNING*

După salvarea cu succes a fiecărei comenzi în tabelul `orders`, backend-ul extrage adresa de email a cumpărătorului din baza de date și trimite automat un email de confirmare personalizat folosind Nodemailer. Acest email confirmă înregistrarea comenzii și informează cumpărătorul că va primi notificări ulterioare la expediere.

La finalizarea tuturor comenzilor, aplicația frontend golește coșul de cumpărături atât local, cât și în baza de date, trimite un eveniment custom pentru actualizarea altor componente (cartUpdated) și redirecționează utilizatorul către pagina de comenzi.

```
await fetch("http://localhost:4000/cart/clear", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ user_id: userId }),
});

fetchCartCount?();
onOrderPlaced?();
window.dispatchEvent(new CustomEvent("cartUpdated"));
if (navigate) navigate("/orders");
}
```

Secvență de cod 4.68 Golirea coșului după plasarea comenzii și notificarea componentelor frontend

Funcționalitatea de vizualizare a comenzilor este implementată prin două componente distincte: OrdersPage.js pentru utilizatorii obișnuiți și AdminOrdersPage.js pentru administratori. În ambele cazuri, comenzile sunt obținute prin requesturi către backend și afișate într-un format structurat.

În OrdersPage.js, comenzile plasate de utilizatorul curent sunt obținute folosind userId din context și endpointul: GET „/orders/user/:user_id”.

Solicitarea este făcută în useEffect, iar comenzile sunt afișate sub formă de carduri (<OrderCard />) cu informații precum titlul produsului, cantitate, preț, status, metodă de plată și adresa de livrare.

În AdminOrdersPage.js, administratorul are acces la toate comenzile din sistem, obținute prin endpointul: GET „/moderateOrders”. Acestea sunt salvate în orders și filteredOrders, permițând aplicarea de filtre locale (după status, user ID, țară, valoare, dată).

```
useEffect(() => {
  fetch("http://localhost:4000/moderateOrders")
    .then((res) => res.json())
    .then((data) => {
      if (Array.isArray(data)) {
        setOrders(data);
        setFilteredOrders(data);
      }
    });
}, []);
```

Secvență de cod 4.69 Încărcarea comenzilor în pagina de moderare pentru administratori

Administratorul poate modifica statusul unei comenzi folosind un dropdown, iar modificarea este transmisă backend-ului.

```
// PUT update status
router.put("/:orderId/status", async (req, res) => {
  const { orderId } = req.params;
  const { newStatus } = req.body;

  const allowedStatuses = ['pending', 'shipped', 'completed', 'cancelled'];
  if (!allowedStatuses.includes(newStatus)) {
    return res.status(400).json({ error: "Invalid status value" });
  }

  const client = await pool.connect();
  try {
    await client.query("BEGIN");

    // Update order
    await client.query(
      "UPDATE orders SET status = $1 WHERE order_id = $2",
      [newStatus, orderId]
    );

  } catch (err) {
    await client.query("ROLLBACK");
    return res.status(500).json({ error: "Database error" });
  }
});
```

Secvență de cod 4.70 Actualizarea statusului unei comenzi din interfața de administrare

După actualizare, statusul este salvat în baza de date și este trimis automat un email de notificare către client.

4.2.7 Funcționalități de comunicare

Funcționalitatea de trimitere și primire a mesajelor în aplicația „CraftCircle” este implementată în mod asincron, combinând apeluri HTTP către backend și actualizări în timp real prin WebSocket (Socket.IO). La trimiterea unui mesaj, componenta MessageInput.js apelează funcția „handleSendMessage”, care trimite un POST către endpointul „/messages/send”, cu sender_id, receiver_id, conversation_id și conținutul mesajului. Dacă nu există încă o conversație, se creează una automat prin „/messages/conversations”.

```
// Trimite mesajul
const res = await fetch("http://localhost:4000/messages/send", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    sender_id: userId,
    receiver_id: activeConversation.user_id,
    content,
    conversation_id: conversationId,
  }),
});

const newMessage = await res.json();
if (!res.ok || !newMessage.message_id) {
  console.warn("Message send failed:", newMessage);
  return;
}

sendMessageIds.current.add(newMessage.message_id);

// Dacă e conversație nouă, adaugă mesajul local (socket-ul vine prea târziu)
if (isNewConversation) {
  setMessages([newMessage]);
}
```

Secvență de cod 4.71 Trimiterea unui mesaj nou în conversațiile utilizatorilor

După trimitere, mesajul este adăugat local în MessageThread, iar ID-ul său este salvat într-un dicționar pentru a evita duplicarea la recepția sa ulterioară prin socket. Pe backend, mesajul este salvat în baza de date, iar apoi se emite un eveniment new_message către destinatar, folosind canalul user-<receiver_id>, configurat în server.js.

```
// Socket.IO
io.on("connection", (socket) => {
  console.log("Client connected:", socket.id);

  socket.on("join", (userId) => {
    console.log(`Received join request for user: ${userId}`);
    socket.join(`user-${userId}`);
    console.log(`Joined room: user-${userId}`);
  });

  socket.on("disconnect", () => {
    console.log("Client disconnected:", socket.id);
  });
});
```

Secvență de cod 4.72 Trimiterea unui mesaj nou în conversațiile utilizatorilor

La primirea unui mesaj prin WebSocket în MessagesPage.js, aplicația verifică dacă conversația activă corespunde cu cea din mesajul nou. Dacă da, mesajul este adăugat în setMessages(), iar dacă utilizatorul curent este destinatarul, se trimite un POST către „/messages/mark-read” pentru a marca mesajele ca citite în baza de date.

```
socket.current.on("new_message", async ({ conversation_id, message }) => {
  if (sentMessageIds.current.has(message.message_id)) {
    sentMessageIds.current.delete(message.message_id);
    return;
  }

  const isCurrent = activeConversation?.conversation_id === conversation_id;

  if (isCurrent) {
    setMessages((prev) => {
      const exists = prev.some((m) => m.message_id === message.message_id);
      return exists ? prev : [...prev, message];
    });

    if (message.receiver_id === userId) {
      try {
        await fetch("http://localhost:4000/messages/mark-read", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ user_id: userId, conversation_id }),
        });
        refreshUnread();
      } catch (err) {
        console.error("Failed to mark as read:", err);
      }
    }
  }
});
```

Secvență de cod 4.73 Recepția mesajelor noi în timp real și marcarea ca citite

Pentru notificări, aplicația utilizează două hook-uri: useUnreadMessages() și useUnreadPreview(). Primul interoghează endpoint-ul „/messages/unread/:userId” pentru a obține numărul total de mesaje necitite și distribuția acestora pe conversații, fiind folosit pentru afișarea badge-ului roșu cu număr în tab-ul „Messages”.

```

1 import { useEffect, useState } from "react";
2
3 export default function useUnreadMessages(userId) {
4   const [unreadCount, setUnreadCount] = useState(0);
5   const [perConversation, setPerConversation] = useState({});
6
7   const fetchUnread = async () => {
8     if (!userId) return;
9
10    try {
11      const res = await fetch("http://localhost:4000/messages/unread/${userId}");
12      const data = await res.json();
13      //console.log("Unread API response:", data);
14
15      if (data && typeof data.total === "number") {
16        setUnreadCount(data.total);
17        setPerConversation(data.byConversation || {});
18      } else {
19        setUnreadCount(0);
20        setPerConversation({});
21      }
22    } catch (err) {
23      console.error("Error fetching unread messages:", err);
24    }
25  };
26
27   useEffect(() => {
28     fetchUnread();
29
30     const interval = setInterval(() => {
31       fetchUnread();
32     }, 3000);
33
34     return () => clearInterval(interval);
35   }, [userId]);
36   return { unreadCount, perConversation, refreshUnread: fetchUnread };
37 }
38
    
```

useUnreadMessages.js

```

1 import { useEffect, useState } from "react";
2
3 export default function useUnreadPreview(userId) {
4   const [unreadMessages, setUnreadMessages] = useState([]);
5
6   useEffect(() => {
7     if (!userId) return;
8
9     const fetchPreviews = async () => {
10      try {
11        const res = await fetch("http://localhost:4000/messages/unread-latest/${userId}");
12        const data = await res.json();
13
14        if (Array.isArray(data) && data.length > 0) {
15          setUnreadMessages(data);
16
17          await fetch("http://localhost:4000/messages/mark-displayed", {
18            method: "POST",
19            headers: { "Content-Type": "application/json" },
20            body: JSON.stringify({ user_id: userId }),
21          });
22        } else {
23          setUnreadMessages([]);
24        }
25      } catch (err) {
26        console.error("Failed to fetch preview messages:", err);
27        setUnreadMessages([]);
28      }
29    };
30
31    fetchPreviews();
32    const interval = setInterval(fetchPreviews, 3000);
33
34    return () => clearInterval(interval);
35  }, [userId]);
36   return unreadMessages;
37 }
    
```

useUnreadPreview.js

Secvență de cod 4.74 Gestionarea mesajelor necitite și a notificărilor de tip preview

Al doilea, `useUnreadPreview()`, interoghează „/messages/unread-latest/:userId” pentru a obține cele mai recente mesaje necitite. Notificările vizuale, sub forma unei casete animate cu avatar, nume și conținutul mesajului, sunt declanșate exclusiv în `Navbar.js`, atunci când un mesaj este primit în timp real prin `WebSocket`, prin evenimentul `socket.on("new_message")`. Dacă mesajul nu aparține conversației active, se setează „showNotif” pe `true` și se populează datele în „liveNotifData”, ceea ce duce la afișarea componentei „MessageNotificationBox” și permite afișarea imediată a notificărilor fără refresh, însoțită de sunet.

În componenta `MessageThread.js`, fiecare mesaj conține un meniu de opțiuni (`MessageOptionsMenu`) cu butoane pentru ștergere. La apăsarea unui buton, este afișat un `ConfirmationModal`.

```

const handleDelete = (messageId) => {
  setConfirmData({ messageId, type: "forMe" });
  setShowConfirm(true);
};

const handleDeleteForAll = (messageId) => {
  setConfirmData({ messageId, type: "forAll" });
  setShowConfirm(true);
};
    
```

Secvență de cod 4.75 Inițierea ștergerii mesajelor – pentru mine sau pentru toți

Confirmarea declanșează apelul către backend:

```
const handleConfirmDelete = async () => {
  const { messageId, type } = confirmData;
  const endpoint =
    type === "forAll" ? "delete-for-all" : "delete";

  try {
    const res = await fetch(`http://localhost:4000/messages/${endpoint}`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ message_id: messageId, user_id: userId }),
    });

    const data = await res.json();
    if (data.success) {
      setMessages((prev) => prev.filter((msg) => msg.message_id !== messageId));
      showToast(
        type === "forAll" ? "Message deleted for everyone." : "Message deleted."
      );
    }
  } catch (err) {
    console.error("Error deleting message:", err);
  } finally {
    setConfirmData({ messageId: null, type: null });
    setShowConfirm(false);
    setOpenMenuId(null);
  }
};
```

Secvență de cod 4.76 Finalizarea ștergerii mesajelor: doar pentru utilizator sau global

Dacă ștergerea are succes, mesajul este eliminat din lista locală `setMessages()` și este afișat un toast informativ.

În backend, pentru ștergere doar pentru utilizatorul curent, se folosește endpoint-ul: POST „./messages/delete”, în care mesajul nu este șters fizic, ci este marcat ca „șters” pentru acel utilizator care este logat, folosind câmpurile `deleted_for_sender` sau `deleted_for_receiver`. Dacă și celălalt utilizator ștersese deja mesajul, atunci rândul este eliminat complet din baza de date.

```
if (isSender) {
  await pool.query(
    `UPDATE messages SET deleted_for_sender = TRUE WHERE message_id = $1`,
    [message_id]
  );
} else if (isReceiver) {
  await pool.query(
    `UPDATE messages SET deleted_for_receiver = TRUE WHERE message_id = $1`,
    [message_id]
  );
}

const shouldDelete =
  (isSender && message.deleted_for_receiver) ||
  (isReceiver && message.deleted_for_sender);

if (shouldDelete) {
  await pool.query(
    `DELETE FROM messages WHERE message_id = $1`,
    [message_id]
  );
}

res.json({ success: true, deletedCompletely: shouldDelete });
```

Secvență de cod 4.77 Ștergerea mesajelor din conversații: logică per utilizator și eliminare definitivă

Pentru ștergere globală (Delete for everyone), doar expeditorul are permisiunea, iar endpoint-ul dedicat este: POST „./messages/delete-for-all”, acesta șterge complet mesajul fără condiții suplimentare.

Funcționalitatea de inițiere a unei conversații permite utilizatorilor să caute alți utilizatori și să înceapă un chat cu oricine, chiar dacă nu au mai interacționat anterior. Căutarea se face în timp real prin componenta SearchBar.js, care trimite interogări către endpointul backend /messages/search. Acesta caută utilizatori în baza de date în funcție de numele introdus, prioritizând rezultatele după următoarele criterii: self-match, utilizatori urmăriți și apoi alți utilizatori.

```
try {
  const res = await fetch(
    `http://localhost:4000/messages/search?userId=${userId}&query=${encodeURIComponent(value)}`
  );
  const data = await res.json();

  // Adaugă last_message_preview din conversations dacă există
  const updatedResults = data.map((user) => {
    const match = existingConversations.find(c => c.user_id === user.user_id);
    return {
      ...user,
      isSelf: parseInt(user.user_id) === parseInt(userId),
      conversation_id: match?.conversation_id || null,
      last_message_preview: match?.last_message_preview || null,
    };
  });

  onSearchResults(updatedResults);
}
```

Secvență de cod 4.78 Căutarea utilizatorilor pentru mesagerie și asocierea conversațiilor existente

```
router.get("/search", async (req, res) => {
  const { userId, query } = req.query;
  const searchTerm = `${query.toLowerCase()}%`;

  if (!userId || !query) {
    return res.status(400).json({ error: "Missing parameters" });
  }

  try {
    const followedResult = await pool.query(
      `SELECT a.user_id, a.first_name, a.last_name, a.profile_picture
      FROM follows f
      JOIN accounts a ON a.user_id = f.following_id
      WHERE f.follower_id = $1
      AND (LOWER(a.first_name) LIKE $2 OR LOWER(a.last_name) LIKE $2)
      LIMIT 10,
      [userId, searchTerm]
    );

    const followedIds = followedResult.rows.map((u) => u.user_id);
    const excludeList = followedIds.length > 0 ? `AND user_id NOT IN (${followedIds.map((_, i) => `$$${i + 3}`.join(", ")))` : "";

    const values = [searchTerm, userId, ...followedIds];

    const allResult = await pool.query(
      `SELECT user_id, first_name, last_name, profile_picture
      FROM accounts
      WHERE (LOWER(first_name) LIKE $1 OR LOWER(last_name) LIKE $1)
      AND user_id != $2
      ${excludeList}
      LIMIT 10,
      values
    );

    const selfMatch = await pool.query(
      `SELECT user_id, first_name, last_name, profile_picture
      FROM accounts
      WHERE user_id = $1 AND (LOWER(first_name) LIKE $2 OR LOWER(last_name) LIKE $2)`,
      [userId, searchTerm]
    );

    const allUsers = [...selfMatch.rows, ...followedResult.rows, ...allResult.rows];
    const uniqueUsers = new Map();
    allUsers.forEach(user => uniqueUsers.set(user.user_id, user));

    res.json(Array.from(uniqueUsers.values()));
  } catch (err) {
    console.error("Search error:", err);
    res.status(500).json({ error: "Internal server error" });
  }
});
```

Secvență de cod 4.79 Endpoint pentru căutarea utilizatorilor în sistemul de mesagerie

Dacă utilizatorul selectează un rezultat din listă, aplicația apelează `handleSelectConversation()`. Dacă nu există o conversație anterioară în baza de date, se creează o „conversație temporară” în stare locală. Aceasta este necesară pentru a permite compunerea unui prim mesaj fără a crea imediat o intrare permanentă în baza de date. După trimiterea primului mesaj, se trimite o cerere POST către „/messages/conversations”, care creează efectiv conversația în baza de date. Imediat după, aplicația actualizează conversațiile locale, curăță rezultatele de căutare și asociază noua `conversation_id` în structura activă.

Pentru funcționalitatea de ștergere a unei conversații, aplicația permite utilizatorului să elimine local toate mesajele din acea conversație, fără a afecta vizibilitatea acestora pentru interlocutor. La nivel de frontend, acțiunea este declanșată din ConversationList, unde fiecare conversație are un buton cu iconița de coș (FaTrash). Când se apasă pe acest buton, se apelează funcția `confirmDelete(conversationId)`, care deschide un `ConfirmationModal`. Dacă utilizatorul confirmă, funcția `handleDeleteConfirmed()` trimite o cerere POST la backend. La nivel de backend, endpointul `„/messages/conversations/delete”` marchează toate mesajele din acea conversație ca fiind șterse pentru utilizatorul respectiv. Acest lucru asigură că utilizatorul care a șters conversația nu o va mai vedea, dar interlocutorul o poate accesa în continuare.

4.2.8 Funcționalități administrative

Administratorii pot vizualiza, adăuga, filtra și șterge categorii prin pagina dedicată `ManageCategoriesPage.js`. Toate operațiile se realizează prin interacțiuni cu backend-ul, iar interfața este sincronizată automat.

La montarea componentei, se face un GET către `„/admin/categories/getCategories”` pentru a obține toate categoriile existente, alături de numărul de postări și produse aferente fiecăreia.

```
router.get("/getCategories", async (req, res) => {
  try {
    const categoriesWithCounts = await pool.query(`
      SELECT
        c.category_id,
        c.name,
        c.description,
        COUNT(DISTINCT p.post_id) AS post_count,
        COUNT(DISTINCT i.item_id) AS product_count
      FROM categories c
      LEFT JOIN posts p ON p.category_id = c.category_id
      LEFT JOIN marketplace_items i ON i.category_id = c.category_id
      GROUP BY c.category_id, c.name, c.description
      ORDER BY c.name ASC;
    `);
    res.json(Array.isArray(categoriesWithCounts.rows) ? categoriesWithCounts.rows : []);
  } catch (err) {
    console.error("Error fetching categories:", err);
    res.status(500).json({ error: "Internal server error" });
  }
});
```

Secvență de cod 4.80 Obținerea categoriilor cu numărul de postări și produse asociate

Categoriile pot fi filtrate în timp real după nume și descriere, folosind câmpurile de căutare din interfață.

Adăugarea unei categorii se face printr-un formular care trimite un POST către `„/admin/categories/add”`, cu `name` și `description` în corpul cererii. După salvare, categoria nouă este adăugată local, fără a reîncărca pagina.

Ștergerea este permisă doar pentru categoriile care nu sunt asociate cu postări sau produse (`post_count` și `product_count` = 0).

```
<div className={styles.statsGrid}>
  {filteredCategories.map(cat => (
    <div key={cat.category_id} className={styles.statCard}>
      <h4>{cat.name}</h4>
      <p>{cat.description || "—"}</p>
      <p><strong>{cat.post_count}</strong> posts</p>
      <p><strong>{cat.product_count}</strong> products</p>
      {Number(cat.post_count) === 0 && Number(cat.product_count) === 0 && (
        <button
          onClick={() => setPendingDelete({ category_id: cat.category_id, name: cat.name })}
          className={styles.deleteButton}>
            Delete
          </button>
        )}
    </div>
  ))}
</div>
```

Secvență de cod 4.81 Afișarea categoriilor și activarea butonului de ștergere condiționată

Apăsarea butonului "Delete" deschide un ConfirmationModal, iar la confirmare se trimite un DELETE către „/admin/categories/:id”. În caz de succes, categoria este eliminată și din starea locală categories.

Postările sunt preluate de la backend printr-un endpoint dedicat „/admin/moderatePosts/all”, care returnează atât conținutul, cât și numărul de raportări agregate. Se pot aplica filtre live după numele autorului, ID-ul acestuia sau textul postării, direct din frontend, fără reîncărcarea paginii.

Pentru fiecare postare, administratorul poate accesa lista utilizatorilor care au raportat-o, printr-un click pe iconul de raportare. Acest lucru declanșează un request către „/admin/moderatePosts/reports/:postId”, care returnează numele și ID-ul celor care au trimis sesizări.

Butonul „See Comments” declanșează setarea stării activeCommentsPostId, care la rândul ei deschide componenta CommentsModal. Acest modal primește postId ca prop și este responsabil pentru afișarea comentariilor asociate postării. Evenimentul de click este izolat prin e.stopPropagation() pentru a preveni redirecționarea către pagina de profil. Modalul reutilizează componenta folosită pentru utilizatorii obișnuiți.

O postare poate fi ștearsă printr-un DELETE către „/admin/moderatePosts/delete-post/:postId”. După confirmare, sistemul elimină postarea din baza de date, incrementează câmpul deleted_posts în profilul administratorului și trimite automat un email utilizatorului care a publicat postarea, cu motivul ștergerii și conținutul acesteia

```
// Delete a post and update admin stats
router.delete('/delete-post/:postId', async (req, res) => {
  const { postId } = req.params;
  const adminId = req.query.admin_id || 1; // Admin ID, use session or token in production
  const client = await pool.connect();

  try {
    await client.query("BEGIN");

    // Get post info
    const postInfo = await client.query(
      `SELECT
        p.content,
        a.email,
        a.first_name,
        (SELECT file_url FROM post_media WHERE post_id = p.post_id LIMIT 1) AS image_url
      FROM posts p
      JOIN accounts a ON p.user_id = a.user_id
      WHERE p.post_id = $1`, [postId]);

    if (postInfo.rows.length === 0) {
      await client.query("ROLLBACK");
      return res.status(404).json({ error: "Post not found" });
    }

    const { content, email, first_name, image_url } = postInfo.rows[0];

    // Delete post likes
    await client.query("DELETE FROM likes WHERE post_id = $1", [postId]);

    // Delete post
    await client.query("DELETE FROM posts WHERE post_id = $1", [postId]);

    // Send notification email
    try {
      await transporter.sendMail({
        from: "oxanicorina@gmail.com",
        to: email,
        subject: "Your post has been removed by the admin",
        html: `
          <p>Hi ${first_name}</p>
          <p>Your post has been removed by an administrator, due to reports by other users.</p>
          <p>Content of the post:</p>
          <div style="border-left: 4px solid #ccc; margin: 10px 0; padding-left: 10px;">
            ${content}
          </div>
          <p>Image: 
          <p>Best regards, C.R. Moderation Team</p>
        `);
      console.log("Email sent successfully.");
    } catch (error) {
      console.warn("Failed to send email:", error);
    }

    res.json({ message: "Post deleted and email sent" });
  } catch (err) {
    await client.query("ROLLBACK");
    console.error("Error deleting post:", err);
    res.status(400).json({ error: "Internal server error" });
  } finally {
    client.release();
  }
});
```

Secvență de cod 4.82 Ștergerea unei postări de către administrator și notificarea utilizatorului

Produsele sunt încărcate din backend prin endpointul: GET „/admin/moderateProducts/all”. Acesta returnează lista completă a produselor împreună cu numărul de raportări pentru fiecare.

Administratorii pot aplica filtre rapide după: numele utilizatorului (filters.name), ID-ul utilizatorului (filters.userId), descrierea produsului (filters.description).

```
<div className={styles.filterBox}>
  <input
    type="text"
    placeholder="Filter by name"
    value={filters.name}
    onChange={e => setFilters({ ...filters, name: e.target.value })}
    className={styles.input}
  />
  <input
    type="text"
    placeholder="Filter by user ID"
    value={filters.userId}
    onChange={e => setFilters({ ...filters, userId: e.target.value })}
    className={styles.input}
  />
  <input
    type="text"
    placeholder="Filter by description"
    value={filters.description}
    onChange={e => setFilters({ ...filters, description: e.target.value })}
    className={styles.input}
  />
</div>
```

Secvență de cod 4.83 Interfață de filtrare pentru pagina de moderare a postărilor

Prin apăsarea pe iconița de raportare (<FiFlag />), se declanșează un request către: GET „/admin/moderateProducts/reports/:productId”. Acesta returnează lista detaliată a utilizatorilor care au raportat produsul respectiv, afișată într-un dropdown.

Pentru ștergerea unui produs, se folosește: DELETE „/admin/moderateProducts/delete-product/:productId”. După confirmarea printr-un ConfirmationModal, produsul este eliminat din baza de date, utilizatorul va primi un mail de înștiințare, iar lista se reîmprospătează automat.

Administrarea utilizatorilor este realizată prin pagina ModerateUsersPage, unde sunt afișați toți utilizatorii existenți în sistem, sortați descrescător în funcție de numărul de raportări. Lista este completată cu date relevante precum numele, emailul și poza de profil a fiecărui utilizator.

Pentru fiecare utilizator, se poate accesa lista completă a raportărilor primite, cu detalii despre motivul raportării și identitatea raportorului. Aceste informații sunt obținute printr-un apel la endpoint-ul backend „/admin/moderateUsers/reports/:userId”.

```
// Get all reports for a specific user
router.get("/reports/:userId", async (req, res) => {
  const { userId } = req.params;
  try {
    const result = await pool.query(
      `SELECT
        r.reporter_id,
        r.reason,
        acc.first_name || ' ' || acc.last_name AS reporter_name
      FROM user_reports r
      JOIN accounts acc ON r.reporter_id = acc.user_id
      WHERE r.reported_id = $1
      ORDER BY r.created_at DESC`,
      [userId]
    );
    res.json(result.rows);
  } catch (err) {
    console.error("Error fetching reports:", err);
    res.status(500).json({ message: "Internal server error" });
  }
});
```

Secvență de cod 4.84 Obținerea raportărilor asociate unui utilizator (moderare conturi)

Funcționalitatea de ștergere este disponibilă pentru fiecare utilizator raportat, declanșată printr-un ConfirmationModal. La confirmare, se trimite un request DELETE către „/admin/moderateUsers/delete-user/:userId”, care:

- Verifică existența utilizatorului în baza de date și îi extrage emailul și numele complet.
- Șterge definitiv contul din tabela accounts.
- Trimite un email de notificare prin Nodemailer, informând utilizatorul despre ștergere.
- Incrementează numărul de utilizatori șterși de către administrator în tabela admins (câmpul deleted_users).

Aplicația oferă și filtrare după ID, nume, prenume sau adresă de email, pentru a ușura identificarea utilizatorilor.

Pentru a monitoriza evoluția aplicației, administratorul are acces la o interfață dedicată, AdminStatisticsPage, unde poate vizualiza în timp real statistici referitoare la utilizatori, postări, produse, mesaje, comenzi și interacțiuni sociale. Funcționalitatea este împărțită în mai multe taburi, fiecare reprezentând o categorie de date.

Pe partea de frontend, componenta AdminStatisticsPage afișează o interfață tabulară (tabs = ["Users", "Posts", "Products", "Messages", "Orders", "App"]). La selectarea fiecărui tab, se declanșează un fetch către un endpoint corespunzător, iar rezultatele sunt afișate într-un grid de carduri cu valorile relevante.

Exemplu: când este activat tabul „Users”, se face un request către: fetch("http://localhost:4000/admin/statistics/users") și se afișează valorile total și newThisMonth.

Pentru reprezentarea grafică a evoluției, se folosește componenta StatisticsChart, care consumă datele de progres lunar de la endpointuri.

```
useEffect(() => {  
  const fetchData = async () => {  
    try {  
      const response = await fetch(`http://localhost:4000/admin/statistics/${endpoint}/progress`);  
      const data = await response.json();  
    }  
  }  
});
```

Secvență de cod 4.85 Obținerea progresului statistic pentru interfața de administrare

Graficul este generat în componenta StatisticsChart.js, folosind biblioteca react-chartjs-2 care oferă o interfață React peste Chart.js. În funcție de tipul de statistică selectat (ex: users, posts, orders, app), componenta face un fetch către un endpoint corespunzător (ex: /admin/statistics/users/progress) și afișează un grafic de tip linie.

```
// Monthly Cumulative Product Growth  
router.get("/products/progress", async (req, res) => {  
  try {  
    const result = await pool.query(generateMonthsQuery("marketplace_items", "created_at"));  
    res.json(result.rows);  
  } catch (err) {  
    console.error("Error fetching products growth stats:", err);  
    res.status(500).json({ error: "Failed to fetch products growth statistics" });  
  }  
});
```

Secvență de cod 4.86 Endpoint pentru evoluția lunară a produselor adăugate

Pentru statisticile aplicației (likes, follows), se construiesc două seturi de date în paralel, fiecare reprezentând evoluția lunară a unei interacțiuni. Valorile afișate sunt cumulative, adică reprezintă totalul înregistrărilor până la fiecare lună, nu doar incrementul lunar.

De asemenea, un modal (UserListModal) este folosit pentru a detalia lista utilizatorilor noi sau existenți, cu funcționalitate de căutare și navigare către profilul acestora.

Pe backend, fiecare tab are un endpoint dedicat în routes/admin/statistics.js, unde se fac interogări directe în PostgreSQL, un exemplu ar fi totalul utilizatorilor și utilizatori noi din luna curentă.

```
const totalUsers = await pool.query(`SELECT COUNT(*) AS total FROM accounts`);  
  
const newThisMonth = await pool.query(`  
  SELECT COUNT(*) AS newThisMonth  
  FROM accounts  
  WHERE DATE_TRUNC('month', created_at) = DATE_TRUNC('month', CURRENT_DATE)  
`);
```

Secvență de cod 4.87 Statistici utilizatori: total și înscriși în luna curentă

4.3 TABELUL RUTELOR BACKEND

Pentru a susține funcționalitățile aplicației, în backend au fost implementate o serie de rute grupate logic pe module, fiecare corespunzând unui set specific de acțiuni. [Tabelul 4.1](#) oferă o imagine de ansamblu asupra acestor rute, indicând metoda HTTP utilizată,

endpoint-ul aferent, datele primite de la frontend și o scurtă descriere a scopului fiecărei rute.

Tabel 4.1 Lista completă a endpointurilor backend RESTful

	Metoda HTTP	Endpoint	Date primite de endpoint	Descriere
1.	GET	/auth/google	Nicio dată explicită de la client; inițiază OAuth cu scope profile și email	Inițializează procesul de autentificare prin Google folosind strategia passport-google-oauth20.
2.	GET	/auth/google/callback	Redirect cu codul de autorizare Google (implicit prin OAuth)	Procesează răspunsul de la Google și redirecționează utilizatorul către frontend cu parametri de login.
3.	GET	/auth/facebook	Nicio dată explicită de la client; inițiază OAuth cu scope email	Inițializează procesul de autentificare prin Facebook folosind passport-facebook.
4.	GET	/auth/facebook/callback	Redirect cu codul de autorizare Facebook (implicit prin OAuth)	Procesează răspunsul de la Facebook și redirecționează utilizatorul către frontend cu datele obținute.
5.	GET	/auth/logout	Cookie de sesiune (implicit); nu primește date în body sau query	Deconectează utilizatorul prin req.logout() și redirecționează către pagina de login din frontend.

6.	POST	/users/login	JSON: { email, password }	Autentifică utilizatorul prin email și parolă; verifică parola criptată folosind bcrypt.
7.	POST	/users/adduser	JSON: { first_name, last_name, email, password, birth_date, bio }	Înregistrează un utilizator nou; salvează parola criptată și detalii de profil în baza de date.
8.	POST	/users/additional-info	JSON: { user_id, birth_date, country, city }	Salvează informații adiționale despre utilizator (data nașterii, țară, oraș).
9.	GET	/users/:userId	URL param: userId (integer)	Returnează datele de profil ale unui utilizator specific, inclusiv statistici.
10.	POST	/users/change-password	JSON: { user_id, oldPassword, newPassword }	Actualizează parola unui utilizator după verificarea celei vechi și trimite email de confirmare.
11.	POST	/users/update-profile	JSON: { user_id, country, city, bio }	Actualizează datele de profil: țară, oraș și biografie.
12.	POST	/users/send-reset-code	JSON: { email }	Generează și trimite pe email un cod de resetare a parolei, valabil 15 minute.
13.	POST	/users/verify-reset-code	JSON: { email, code }	Verifică dacă codul de resetare introdus este corect și încă valabil.
14.	POST	/users/reset-password	JSON: { email, newPassword }	Resetează parola utilizatorului pe baza unui cod

				valid; trimite notificare prin email.
15.	DELETE	/users/:userId	URL param: userId (integer)	Șterge definitiv un utilizator din baza de date după ID.
16.	POST	/users/report	JSON: { reporter_id, reported_id, reason }	Salvează o raportare împotriva unui utilizator; permite un raport pe zi per pereche utilizator.
17.	GET	/post/	Query param: user_id (string/integer)	Returnează toate postările unui utilizator, ordonate descrescător după dată.
18.	DELETE	/posts/:postId	URL param: postId (integer)	Șterge o postare și fișierele media asociate acesteia din Cloudinary și baza de date.
19.	PUT	/posts/:postId	URL param: postId (integer); Body: { content }	Actualizează conținutul unei postări existente.
20.	POST	/posts/:postId/report	URL param: postId (integer); Body: { user_id }	Salvează o raportare pentru postare; permite o singură raportare pentru aceeași postare.
21.	POST	/uploads/upload-profile	multipart/form-data: Body: { user_id } + file	Încarcă o nouă poză de profil în Cloudinary, o salvează în baza de date și șterge imaginea anterioară.
22.	POST	/uploads/upload-post	multipart/form-data: Body: { user_id, content, category_id } + files[] (max 10)	Creează o postare nouă și salvează fișierele media (imagini sau video) asociate.

23.	GET	/uploads/user-posts/:user_id	URL param: user_id (integer)	Returnează toate postările unui utilizator, fiecare cu fișierele media asociate (ca listă de URL-uri).
24.	POST	/shop/add-product	multipart/form-data: Body: { user_id, title, description, price, category_id } + images[]	Adaugă un produs nou în marketplace și încarcă imaginile în Cloudinary.
25.	GET	/shop/user-products/:userId	URL param: userId (integer)	Returnează produsele publicate de un utilizator, împreună cu imaginile asociate.
26.	POST	/shop/report-product/:itemId	URL param: itemId (integer); Body: { user_id, reported_user_id }	Salvează o raportare pentru un produs; permite o singură raportare per utilizator.
27.	DELETE	/shop/delete-product/:itemId	URL param: itemId (integer)	Șterge produsul, imaginile din Cloudinary, raportările, și intrările din coș/favorite.
28.	PUT	/shop/update-product	multipart/form-data: Body: { item_id, title, description, price, stock, imagesToDelete } + newImages[]	Actualizează detaliile produsului și actualizează imaginile (șterge și adaugă).
29.	POST	/shop/add-to-cart	JSON: { user_id, product_id, quantity }	Adaugă un produs în coșul utilizatorului; dacă produsul există deja, mărește cantitatea.
30.	GET	/cart/user-cart/:userId	URL param: userId (integer)	Returnează toate produsele

				din coșul unui utilizator, împreună cu datele despre vânzător și o imagine.
31.	PUT	/cart/update-quantity	JSON: { cart_id, quantity }	Actualizează cantitatea unui produs existent în coș.
32.	DELETE	/cart/delete/:cartId	URL param: cartId (integer)	Șterge un produs specific din coș, după cart_id.
33.	POST	/cart/clear	JSON: { user_id }	Golește întregul coș de cumpărături pentru utilizatorul respectiv.
34.	GET	/favorites/user-favorites/:userId	URL param: userId (integer)	Returnează toate produsele marcate ca favorite de un utilizator, incluzând detalii despre produs și vânzător.
35.	POST	/favorites/add	JSON: { user_id, item_id, seller_id }	Adaugă un produs în lista de favorite, dacă nu este deja adăugat.
36.	DELETE	/favorites/delete/:favoriteId	URL param: favoriteId (integer)	Elimină un produs din lista de favorite după ID-ul său intern.
37.	GET	/follows/check	Query params: follower_id, following_id	Verifică dacă un utilizator îl urmărește deja pe altul. Returnează isFollowing: true/false.
38.	POST	/follows/	JSON: { follower_id, following_id }	Creează o relație de tip follow între doi utilizatori și

				trimite notificare în timp real.
39.	DELETE	/follows/	JSON: { follower_id, following_id }	Șterge relația de urmărire (unfollow) între doi utilizatori.
40.	GET	/follows/followers/:userId	URL param: userId (integer)	Returnează lista utilizatorilor care îl urmăresc pe cel specificat.
41.	GET	/follows/following/:userId	URL param: userId (integer)	Returnează lista utilizatorilor urmăriți de cel specificat.
42.	GET	/messages/search	Query: userId, query	Caută utilizatori pe baza numelui, prioritizând userii urmăriți.
43.	POST	/messages/conversations	JSON: { user1_id, user2_id }	Creează (sau returnează) conversația dintre doi utilizatori.
44.	GET	/messages/:conversationId	URL param: conversationId; Query: user_id	Returnează toate mesajele dintr-o conversație, excluzând cele marcate ca șterse pentru utilizator.
45.	POST	/messages/send	JSON: { sender_id, receiver_id, content }	Trimite un mesaj și notifică în timp real destinatarul.
46.	GET	/messages/conversations/:userId	URL param: userId	Returnează lista conversațiilor utilizatorului, cu ultimul mesaj și informații despre interlocutor.
47.	POST	/messages/mark-read	JSON: { user_id, conversation_id }	Marchează toate mesajele primite ca citite și emite evenimente message_seen.

48.	GET	/messages/unread/:userId	URL param: userId	Returnează numărul de mesaje necitite pe conversație și totalul.
49.	GET	/messages/unread-latest/:userId	URL param: userId	Returnează ultimele 3 mesaje necitite și nevizualizate (was_displayed = false).
50.	POST	/messages/delete	JSON: { message_id, user_id }	Marchează un mesaj ca șters pentru expeditor sau destinatar. Dacă ambele părți au șters, îl elimină complet.
51.	POST	/messages/delete-for-all	JSON: { message_id, user_id }	Șterge definitiv un mesaj pentru toți utilizatorii (doar dacă user_id este expeditorul).
52.	POST	/messages/conversations/delete	JSON: { conversation_id, user_id }	Marchează toate mesajele dintr-o conversație ca șterse pentru utilizatorul dat.
53.	POST	/messages/mark-displayed	JSON: { user_id }	Marchează mesajele necitite ca fiind vizualizate (pentru afișare toast).
54.	POST	/likes/add	JSON: { user_id, post_id }	Adaugă un like pentru o postare. Dacă postarea aparține altui utilizator, se trimite și o notificare.
55.	POST	/likes/remove	JSON: { user_id, post_id }	Șterge like-ul unui utilizator de la o postare.
56.	GET	/likes/:post_id/:user_id	URL params: post_id, user_id (intergers)	Returnează numărul total de like-uri pentru postare și dacă

				utilizatorul respectiv a dat like.
57.	GET	/appreciation/unseen-count	Query: user_id (integer)	Returnează numărul total de notificări de apreciere necitite pentru un utilizator.
58.	POST	/appreciation/mark-as-seen	Query: user_id (integer)	Marchează toate notificările de apreciere ca fiind văzute (seen = true) pentru utilizator.
59.	GET	/appreciation/list	Query: user_id (integer)	Returnează o listă de maxim 30 notificări recente (like, follow, comment) cu detalii despre utilizatorul care a declanșat acțiunea și imaginea postării.
60.	POST	/orders/	JSON: { buyer_id, seller_id, item_id, status, payment_method, paid_amount, total_due, country, city, state, zip_code, street, address_details }	Creează o comandă nouă în baza de date și trimite un email de confirmare cumpărătorului.
61.	GET	/orders/user/:user_id	URL param: user_id (integer)	Returnează toate comenzile plasate de un anumit utilizator, ordonate descrescător după data creării.
62.	POST	/stripe/create-payment-intent	JSON: { amount } (float, în euro)	Creează un obiect PaymentIntent în Stripe și returnează

				clientSecret necesar pentru plata cu cardul. Suma este convertită în cenți.
63.	GET	/discover/posts	Query: user_id (obligatoriu), opțional: search, category, filter	Returnează o listă de postări recomandate (follow, categorie proprie, istoric, căutare sau cele mai recente). Include media, user info și număr de like- uri.
64.	GET	/discover/categories	-	Returnează lista tuturor categoriilor de postări din sistem, sortate alfabetic.
65.	GET	/discover/products	Query: user_id (obligatoriu), opțional: search, category, filter	Returnează o listă de produse sugerate sau căutate, filtrate după: urmăritori, categorii proprii, istoric de căutări, text sau preț. Include și imaginile produselor.
66.	GET	/comments/:postId	URL param: postId (integer)	Returnează toate comentariile asociate unei postări, ordonate cronologic, cu numele și poza comentatorilor.
67.	POST	/comments/:postId	URL param: postId; Body: { text, user_id }	Adaugă un comentariu la o postare. Dacă postarea aparține altui utilizator, se

				trimite notificare de tip comment.
68.	GET	/admin/:id	URL param: id (integer)	Returnează informațiile profilului unui administrator (nume, email, poză, oraș, țară, data nașterii, data creării).
69.	POST	/admin/login	JSON: { email, password }	Autentifică un administrator pe baza emailului (cu domeniu @craft.com) și parolei.
70.	GET	/admin/:adminId/actions	URL param: adminId (integer)	Returnează statistici legate de activitatea administratorului (ștergeri de useri, postări, produse).
71.	POST	/admin/upload-admin-profile	multipart/form-data: { admin_id } + Fișier	Încarcă sau actualizează poza de profil a administratorului în Cloudinary. Șterge imaginea veche dacă este cazul.
72.	POST	/admin/change-password	JSON: { admin_id, oldPassword, newPassword }	Actualizează parola administratorului după verificarea celei vechi și trimite email de confirmare.
73.	GET	/admin/statistics/users	-	Returnează numărul total de utilizatori și câți s-au înregistrat în luna curentă.
74.	GET	/admin/statistics/posts	-	Returnează numărul total de postări și câte sunt raportate.

75.	GET	/admin/statistics/products	-	Returnează numărul total de produse și câte sunt raportate.
76.	GET	/admin/statistics/messages	-	Returnează numărul total de mesaje, câte sunt citite și necitite.
77.	GET	/admin/statistics/orders	-	Returnează numărul total de comenzi, câte sunt finalizate și câte sunt în așteptare.
78.	GET	/admin/statistics/app	-	Returnează numărul total de aprecieri (likes) și urmărirea (follows) din aplicație.
79.	GET	/admin/statistics/users/progress	-	Returnează evoluția lunară cumulativă a înregistrărilor de utilizatori (ultimele 12 luni).
80.	GET	/admin/statistics/posts/progress	-	Returnează evoluția lunară cumulativă a postărilor.
81.	GET	/admin/statistics/products/progress	-	Returnează evoluția lunară cumulativă a produselor adăugate.
82.	GET	/admin/statistics/messages/progress	-	Returnează evoluția lunară cumulativă a mesajelor trimise.
83.	GET	/admin/statistics/orders/progress	-	Returnează evoluția lunară cumulativă a comenzilor plasate.
84.	GET	/admin/statistics/app/progress	-	Returnează evoluția lunară a aprecierilor și

				urmăririlor (likes & follows).
85.	GET	/admin/statistics/users/all	-	Returnează lista tuturor utilizatorilor (id, nume, email), ordonați după data înregistrării.
86.	GET	/admin/statistics/users/new	-	Returnează lista utilizatorilor înregistrați în luna curentă.
87.	GET	/admin/categories/getCategories	-	Returnează toate categoriile, alături de numărul de postări și produse asociate fiecăreia.
88.	POST	/admin/categories/add	Body: { name, description }	Adaugă o nouă categorie în baza de date. Numele este obligatoriu.
89.	DELETE	/admin/categories/:id	URL param: id (ID-ul categoriei)	Șterge categoria cu ID-ul specificat din baza de date.
90.	GET	/admin/moderatePosts/all	-	Returnează toate postările, cu informații despre user, număr de like-uri, raportări și media. Postările raportate apar primele.
91.	DELETE	/admin/moderatePosts/delete-post/:postId	URL param: postId, query param: admin_id	Șterge postarea și incrementează numărul de postări șterse de admin. Trimite și email autorului postării.
92.	GET	/admin/moderatePosts/reports/:postId	URL param: postId	Returnează o listă cu userii care au raportat

				postarea respectivă.
93.	GET	/admin/moderateProducts/all	-	Returnează toate produsele din marketplace, cu informații despre utilizator, număr de raportări și imagini. Cele mai raportate apar primele.
94.	DELETE	/admin/moderateProducts/delete-product/:itemId	URL param: itemId, query param: admin_id	Șterge produsul și actualizează statisticile adminului. Trimite notificare prin email către autorul produsului.
95.	GET	/admin/moderateProducts/reports/:itemId	URL param: itemId	Returnează o listă cu utilizatorii care au raportat produsul respectiv.
96.	GET	/admin/moderateUsers/all	-	Returnează toți utilizatorii din platformă, împreună cu numărul de raportări primite, ordonați descrescător după raportări.
97.	GET	/admin/moderateUsers/reports/:userId	URL param: userId	Returnează toate raportările pentru utilizatorul dat, incluzând motivul și numele celui care a raportat.
98.	DELETE	/admin/moderateUsers/delete-user/:userId	URL param: userId, session/query param: admin_id	Șterge contul utilizatorului, trimite notificare prin email și incrementează statisticile

				adminului (deleted_users).
99.	GET	/moderateOrders	-	Returnează toate comenzile din baza de date, ordonate descrescător după data creării.
100.	PUT	/moderateOrders/:orderId/status	URL param: orderId Body: { newStatus: "..."} }	Actualizează statusul comenzii către una dintre valorile permise (pending, shipped, completed, cancelled) și trimite email de notificare cumpărătorului.

5. UTILIZAREA APLICAȚIEI DE CĂTRE UTILIZATORI

Acest capitol descrie în detaliu modul de utilizare al aplicației „CraftCircle” din perspectiva utilizatorului final. Sunt prezentate principalele funcționalități disponibile, pașii necesari pentru interacțiunea cu interfața, precum și mecanismele de gestionare a conținutului și contului.

5.1 ÎNREGISTRARE ȘI AUTENTIFICARE

5.1.1 Înregistrare prin formular

Utilizatorul își poate crea un cont completând formularul de înregistrare. La apăsarea butonului „Sign up”, aplicația validează câmpurile (formatul emailului, parolele etc.) și afișează erori în cazul unor completări incorecte. Câmpul „Confirm Password” include un buton tip „ochi” pentru vizualizarea parolei. După o înregistrare reușită, utilizatorul este redirecționat către pagina de completare a datelor adiționale ([Figura 5.1](#)).

Dacă utilizatorul își dă seama că, de fapt, deține deja un cont pe platformă, poate fi redirecționat către pagina de autentificare prin apăsarea butonului „Log in” ([Figura 5.1](#)).

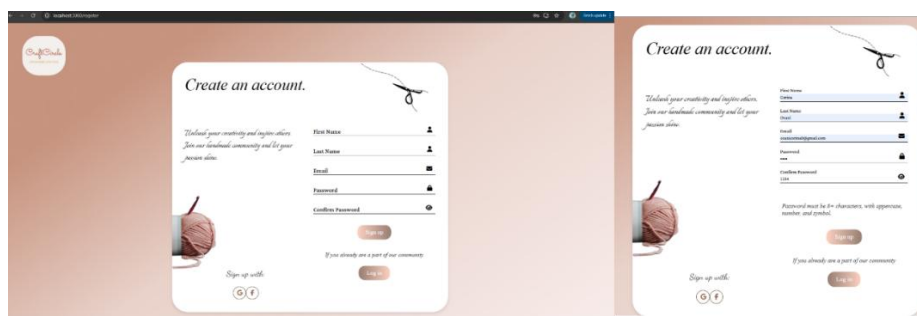


Figura 5.1 Interfața Sign up

5.1.1 Înregistrare și autentificare cu Google / Facebook

Pentru a își face un cont nou sau accesa contul existent, la nivel de interfață, utilizatorul are posibilitatea de a selecta metoda preferată de autentificare socială prin intermediul butoanelor reprezentate în imaginile de mai jos:

Sign up with:



Figura 5.2 Opțiuni de înregistrare sau autentificare cu Google și Facebook din pagin Sign up



În cazul în care parola este uitată, se oferă un buton „Reset Password”, care declanșează un flux de recuperare prin e-mail, descris în subcapitolul 4.1.6. Pentru utilizatorii care nu au încă un cont, se oferă un buton „Sign Up”, care deschide formularul de înregistrare (Figura 5.5).

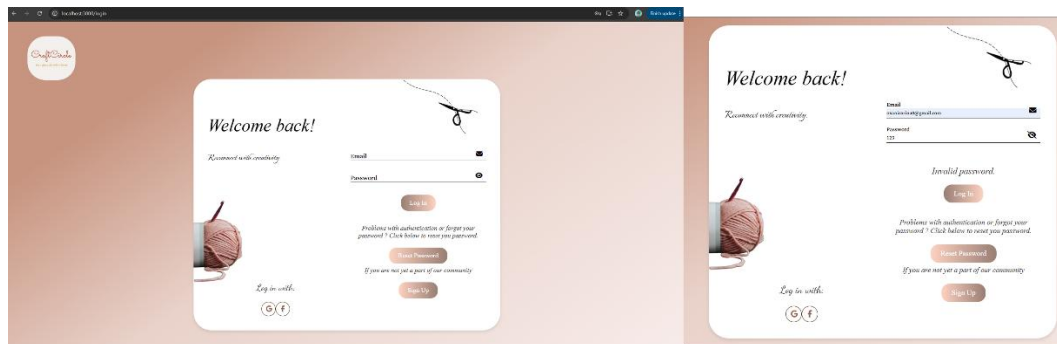


Figura 5.5 Intefăț Log In

5.1.4 Resetarea parolei

Resetarea parolei poate fi inițiată prin apăsarea butonului „Reset Password”, care deschide un prim modal unde utilizatorul trebuie să introducă adresa de e-mail asociată contului. La apăsarea butonului „Send Code”, aplicația trimite un cod de confirmare la adresa respectivă și deschide automat un al doilea modal pentru introducerea acestui cod.

După validarea codului prin butonul „Verify”, utilizatorul este direcționat către un al treilea modal, unde poate seta o nouă parolă și o poate confirma. La final, prin apăsarea butonului „Save Password”, parola este actualizată în sistem, iar utilizatorul primește un e-mail de confirmare a schimbării acesteia.

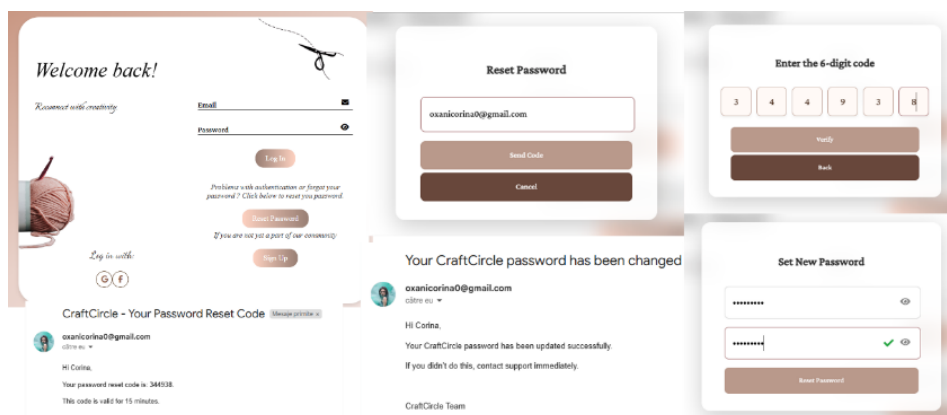


Figura 5.6 Modale Reset Password

5.2 PAGINA DE PROFIL

5.2.1 Vizualizarea profilului personal și al altor utilizatori

Pagina de profil din „CraftCircle” oferă un spațiu personalizat în care utilizatorii își pot evidenția activitatea creativă, pot adăuga postări și interacționa cu alți membri. În partea superioară se află bara de navigație, care permite accesul rapid către secțiuni precum „Discover”, „Home”, „Messages”, „Orders”, „Favorites” și coșul de cumpărături, unele fiind însoțite de badge-uri pentru notificări (Figura 5.7). Tabul „Home” din navbar redirecționează către pagina de profil a utilizatorului logat.

Sub navbar sunt afișate fotografia, numele, descrierea și localizarea utilizatorului. Dacă profilul este propriu, apare un icon de tip creion pentru schimbarea pozei de profil, care deschide opțiunile: „Choose file”, „Save” și „Revert” (Figura 5.9).

Urmează un panou cu statistici (followeri, urmăriri, postări) și un buton către shop, fiecare element fiind interactiv. Utilizatorii care își vizualizează propriul profil au și un formular de postare, unde pot adăuga descriere, categorie și fișiere media, cu previzualizare și opțiuni de ștergere (Figura 5.10).

Postările sunt afișate sub formă de carduri, cu imagine, descriere, aprecieri și comentarii. Proprietarul postării are opțiunile „Edit Post” și „Delete Post”, iar ceilalți doar „Report Post”. Postările altor utilizatori pot fi apreciate și comentate. Butonul „See or Add Comments” deschide un modal cu comentariile, iar apăsarea pe numele sau poza unui comentator redirecționează către profilul său (Figura 5.11).

La vizualizarea unui profil străin, interfața se adaptează: formularul de postare și opțiunea de editare a pozei dispar, iar în loc apare butonul „Start Follow” sau „Stop Follow”, în funcție de relația dintre utilizatori. Lângă poza de profil este afișat și un icon de raportare (steguleț), care deschide un formular pentru motivul raportării (Figura 5.8, Figura 5.12).

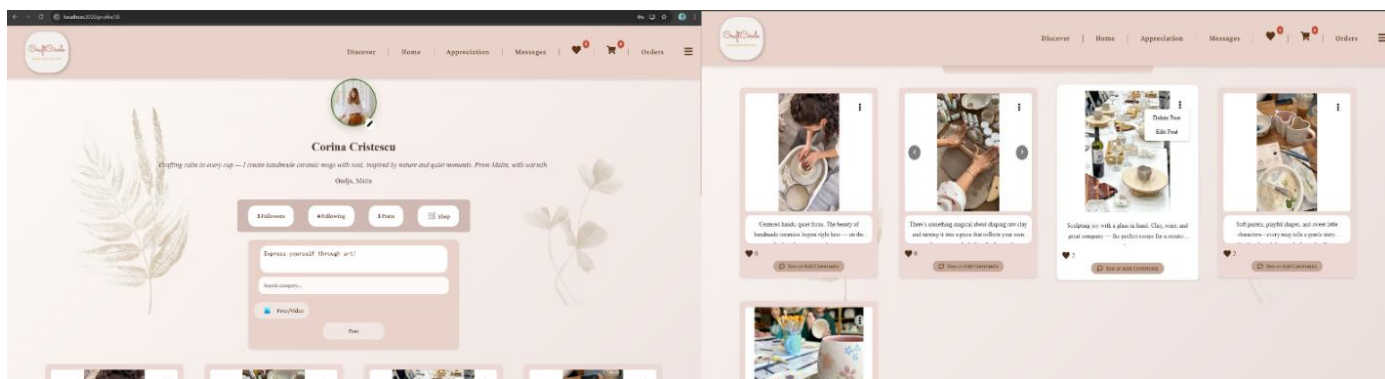


Figura 5.7 Pagina de profil - utilizator logat

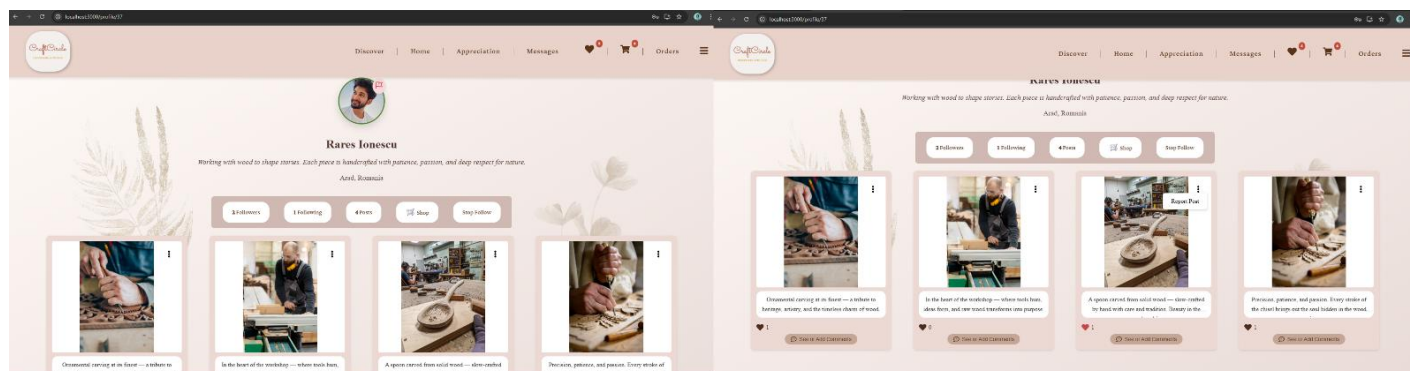


Figura 5.8 Interfața profilului unui alt utilizator (vizualizat de către utilizatorul logat)

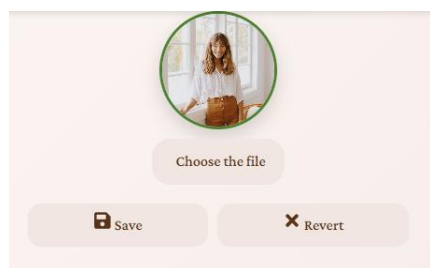


Figura 5.9 Schimbarea pozei de profil

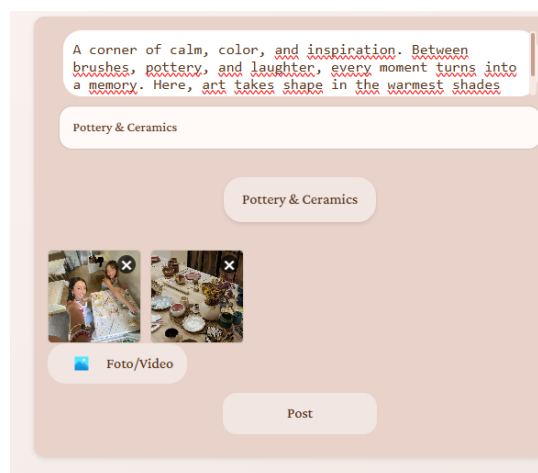


Figura 5.10 Formular de creare a unei postări

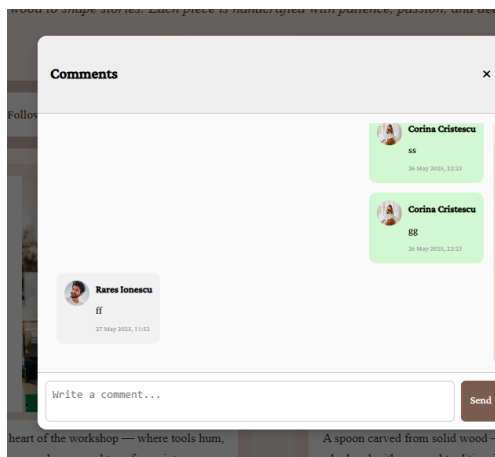


Figura 5.11 Modal comentarii aferente unei postări

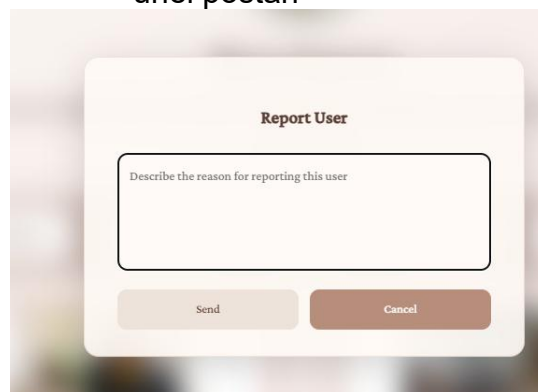


Figura 5.12 Modal raportare utilizator

5.2.2 Pagina Followers

Pagina „Followers” este o pagină dedicată asociată fiecărui profil, accesibilă prin butonul „Followers” din panoul informativ. Aceasta afișează lista completă a utilizatorilor care urmăresc profilul respectiv, fiecare într-un card individual cu nume și fotografie de profil.

Dacă utilizatorul logat nu îi urmărește deja, în dreptul fiecărui card va apărea un buton Start Follow. Prin apăsarea acestuia, utilizatorul poate începe să urmărească persoana respectivă direct din listă.

Aspectul general al paginii este similar cu cel al profilului: bara de navigație, antetul cu poza, descrierea și locația utilizatorului sunt păstrate.

La apăsarea pe un card, utilizatorul este redirecționat către pagina de profil a persoanei respective.

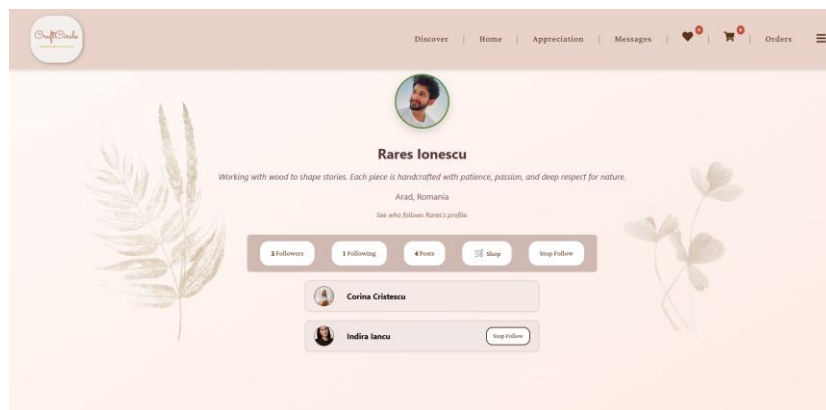


Figura 5.13 Pagina Followers: afișarea urmăritorilor

5.2.3 Secțiunea Following

Pagina „Following” este o altă pagină dedicată din cadrul profilului și poate fi accesată prin butonul „Following” din panoul informativ. Aici sunt afișați toți utilizatorii pe care profilul curent îi urmărește, prezentați în același format de carduri ca în pagina Followers.

Dacă utilizatorul logat alege să înceteze urmărirea, în dreptul unui card va apărea butonul Stop Follow. La apăsarea acestuia, relația de urmărire este întreruptă și interfața se actualizează corespunzător.

La fel ca în cazul paginii Followers, la click pe un card, utilizatorul este direcționat către profilul persoanei respective.

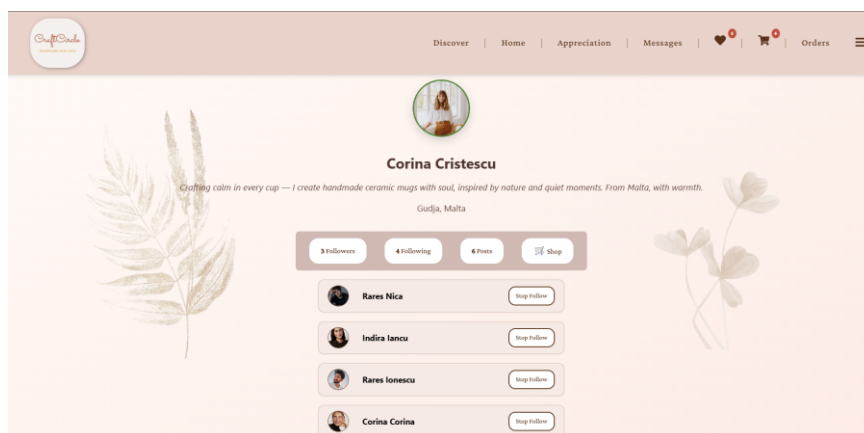


Figura 5.14 Pagina Following: afișarea utilizatorilor urmăriți

5.2.4 Pagina Shop personal

Pagina „Shop” este dedicată fiecărui profil și se accesează din zona de statistici. Afișează produsele disponibile spre vânzare, într-un format coerent cu restul profilului.

Dacă utilizatorul este pe propriul profil, sub statistici apare un formular de adăugare produs ([Figura 5.15](#)) (toate obligatorii, exceptând descrierea). După trimiterea formularului, produsul este adăugat imediat în listă.

Produsele sunt afișate ca carduri. Utilizatorul logat are, în colțul fiecărui card, un meniu cu opțiuni de „Edit Product” și „Delete Product”. La editare, câmpurile devin modificabile (titlu, descriere, preț, stoc), iar stocul poate fi setat ca in-stock sau out-of-stock. Modificările se salvează cu Save sau se anulează cu „Cancel”, fiind vizibile imediat.

Pe shop-ul altor utilizatori, formularul de adăugare dispăre, iar meniul cardului conține doar Report Product. Dacă produsul este în stoc, apar butoanele „Add to Cart” și „Add to Favorites”. Dacă nu, acestea sunt înlocuite cu eticheta „Out of Stock”.

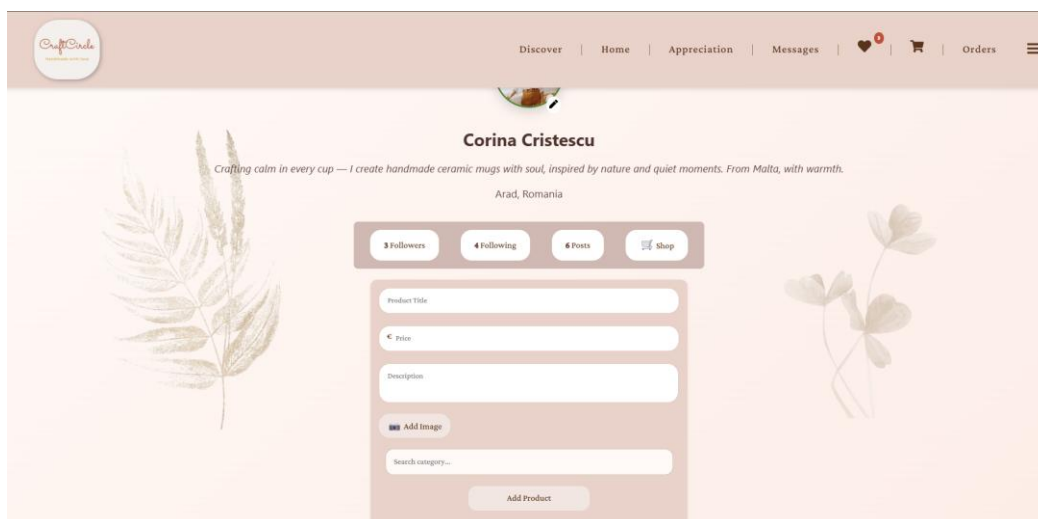
The image shows a web interface for adding a product. At the top, there's a navigation bar with links: Discover, Home, Appreciation, Messages, a heart icon, a shopping cart icon, and Orders. Below this is the user's profile header for 'Corina Cristescu', with a bio and location. A central menu has buttons for Followers, Following, Posts, and Shop. The 'Add Product' form is in the center, with fields for Product Title, Price, Description, an 'Add Image' button, and a 'Search category...' dropdown. An 'Add Product' button is at the bottom.

Figura 5.15 Formular de adăugare produs în pagina de profil personal

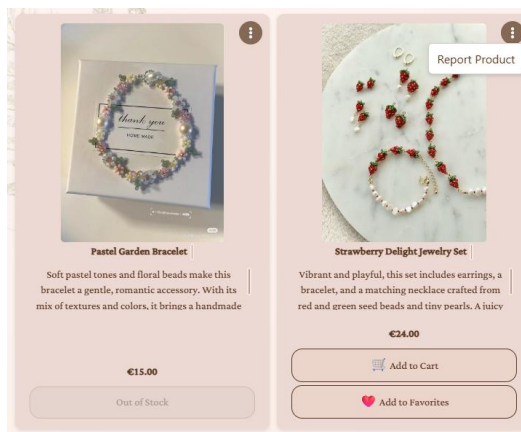


Figura 5.16 Carduri de produs vizibile pentru alți utilizatori, cu opțiuni de raportare, adăugare în coș și favorite

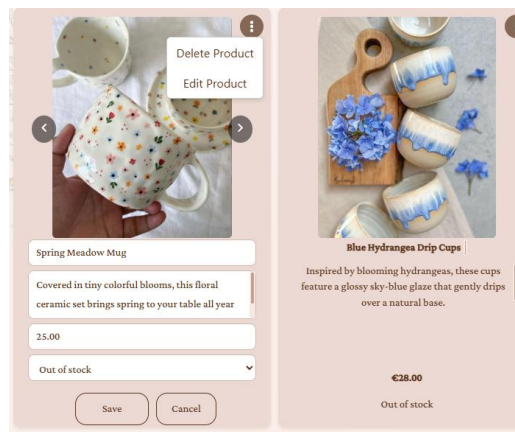


Figura 5.17 Carduri de produs proprii, cu opțiuni de editare, ștergere și actualizare a stocului

5.3 PAGINA DE APRECIERI

Pagina „Appreciation” afișează interacțiunile primite de la alți utilizatori, precum like-uri, comentarii sau follow-uri. Este accesibilă din navbar sau printr-o notificare animată care apare la o nouă apreciere ([Figura 5.19](#)).

Notificarea include avatarul, numele utilizatorului și tipul acțiunii („liked your post”, „commented on your post” sau „started following you”) și redirecționează către pagina „Appreciation”.

Aprecierea este afișată sub formă de listă, fiecare element conținând poza și numele utilizatorului, tipul interacțiunii și, dacă e cazul, o miniatură a postării ([Figura 5.18](#)). Click pe numele persoanei deschide profilul acesteia; click pe postare duce la profilul propriu, cu scroll automat și evidențiere temporară a postării.

În navbar apare și un badge roșu cu numărul de aprecieri necitite. După accesarea paginii, badge-ul dispare, iar toate aprecierile sunt marcate ca vizualizate.

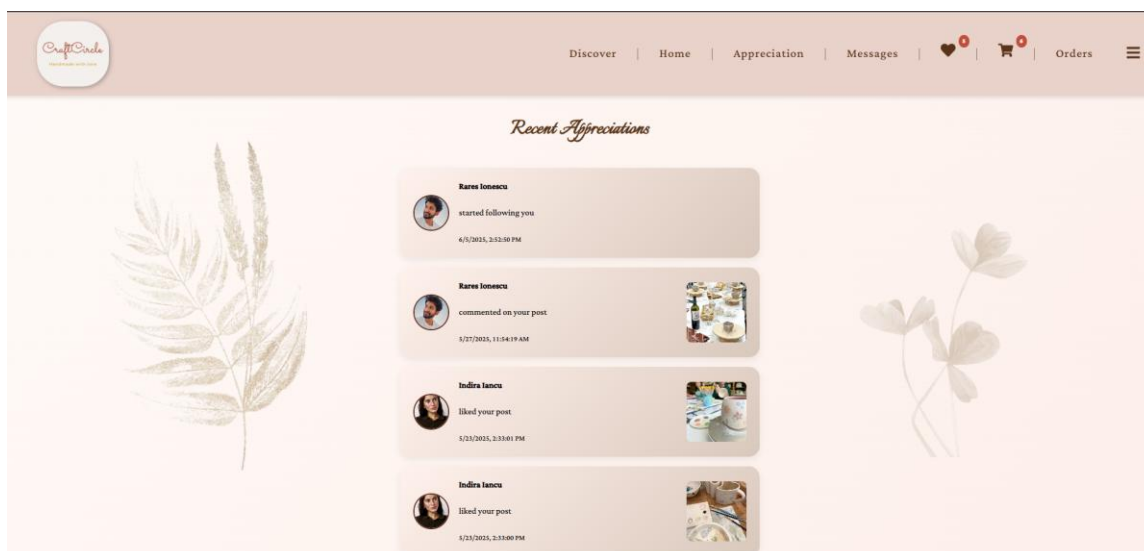


Figura 5.18 Interfață pagina de Aprecieri

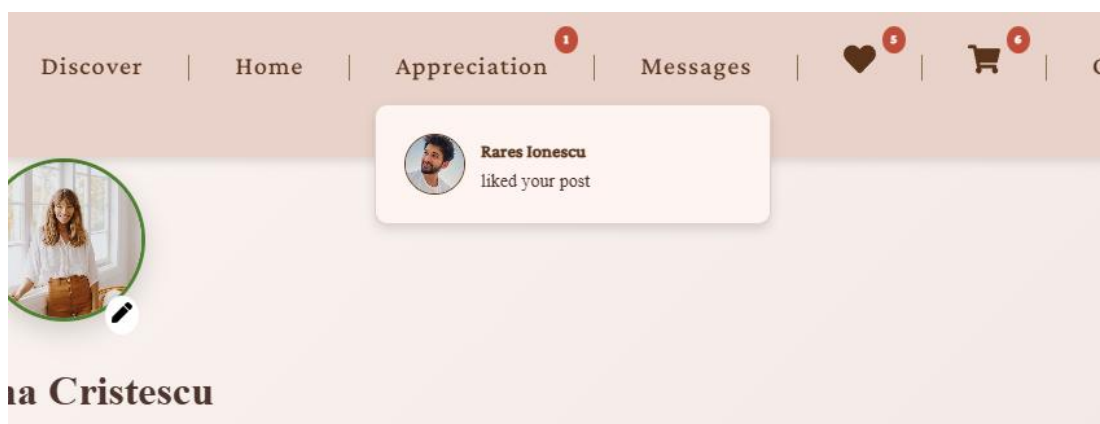


Figura 5.19 Notificare apreciere

5.4 PAGINA DE MESAJE

Pagina „Messages” permite comunicarea în timp real prin mesaje text și poate fi accesată din bara de navigație. Interfața este împărțită în două zone: în stânga – lista conversațiilor, în dreapta – conversația activă ([Figura 5.20](#)).

La selectarea unei conversații, mesajele sunt afișate: cele trimise apar în dreapta, cele primite în stânga. Fiecare mesaj propriu are un meniu cu opțiunile Delete for me și Delete for everyone, iar cele primite doar Delete for me. Acțiunea este confirmată printr-un modal, urmată de un toast cu „Message deleted” ([Figura 5.21](#)).

Mesajele se trimit prin caseta „Type a message...” folosind butonul „Send” sau tasta Enter. Mesajele proprii includ ora trimiterii și statusul „Seen”, dacă au fost citite.

Fiecare conversație din listă are o pictogramă pentru ștergere, care deschide un modal de confirmare. Ștergerea se aplică doar pentru utilizatorul logat.

Profilul interlocutorului poate fi accesat prin click pe numele sau fotografia sa. Pentru conversații noi, există o bară de căutare „Search for users” în partea de sus. Selectarea unui utilizator fără conversație existentă deschide un chat gol; după trimiterea primului mesaj, conversația este salvată și listată ([Figura 5.22](#)).

Aplicația permite și auto-mesajarea, afișând utilizatorul curent ca „Me”, pentru notițe personale.

Lista conversațiilor este derulabilă și sortată automat în funcție de cel mai recent mesaj.

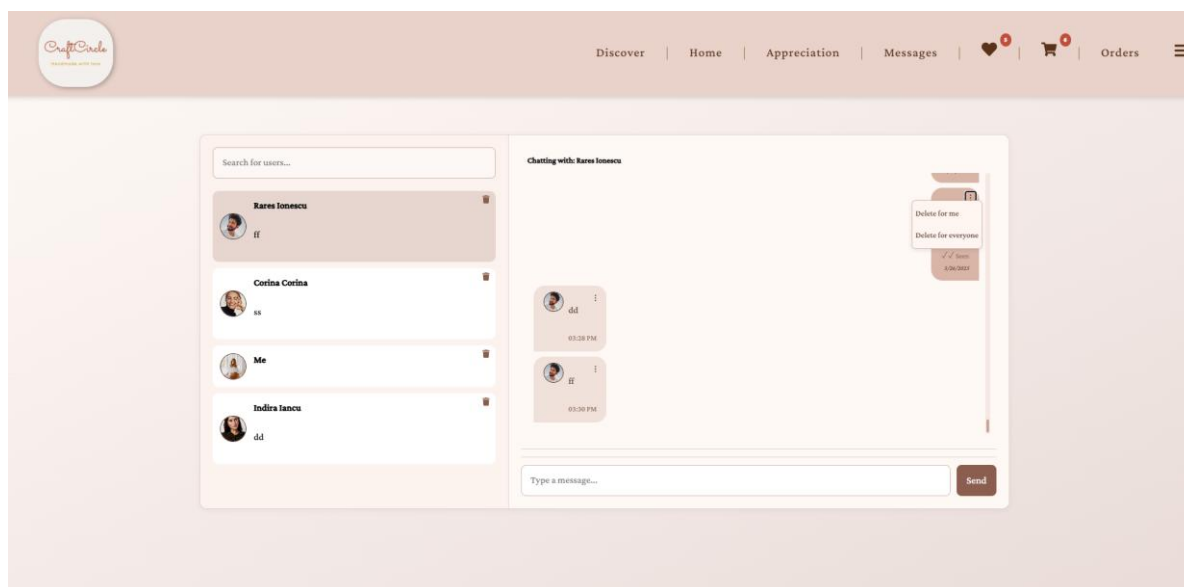


Figura 5.20 Interfață pagină mesagerie

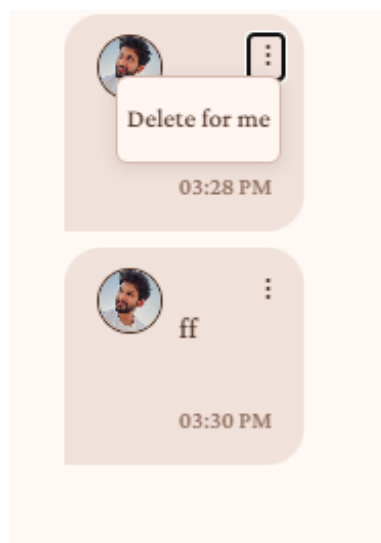


Figura 5.21 Mesaje interlocutor

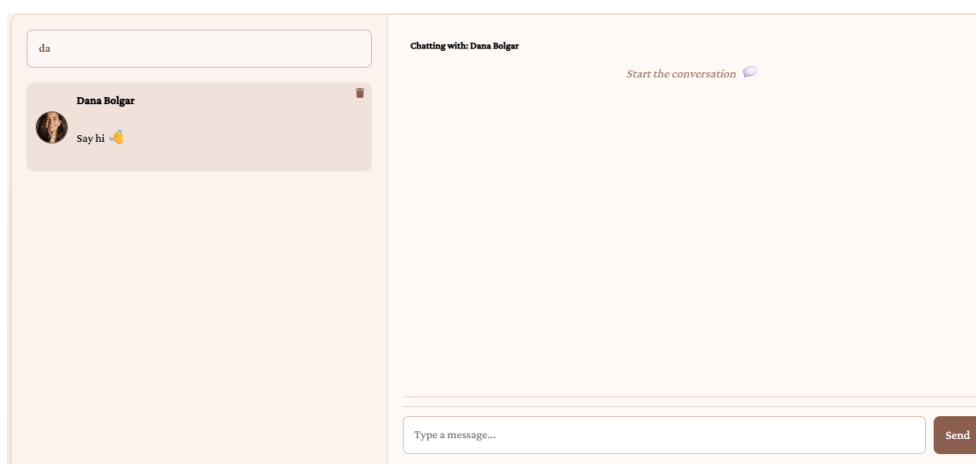


Figura 5.22 Căutarea unui utilizator și începerea unei conversații

5.5 PAGINA DE FAVORITE

Pagina „Favorites” permite salvarea produselor preferate și poate fi accesată din bara de navigație, prin pictograma în formă de inimă. Lângă aceasta apare un indicator numeric ce reflectă numărul actual de produse favorite, actualizat automat la adăugare sau eliminare.

Un produs poate fi salvat apăsând butonul „Add to Favorites” de pe cardul său. Dacă este deja salvat, apare un mesaj informativ.

În pagina „Favorites”, produsele sunt grupate după vânzător, fiecare grup afișând numele acestuia și lista produselor. Fiecare card conține o imagine reprezentativă, numele produsului, prețul și butoanele „Add to Cart” și „Remove”.

Adăugarea în coș nu afectează lista de favorite — produsul rămâne salvat până la eliminare manuală. După apăsarea butonului „Remove”, lista și indicatorul din navbar se actualizează instant.

Click pe orice parte a cardului (exceptând butoanele) redirecționează către pagina de shop a vânzătorului, unde produsul este evidențiat temporar printr-un efect vizual.

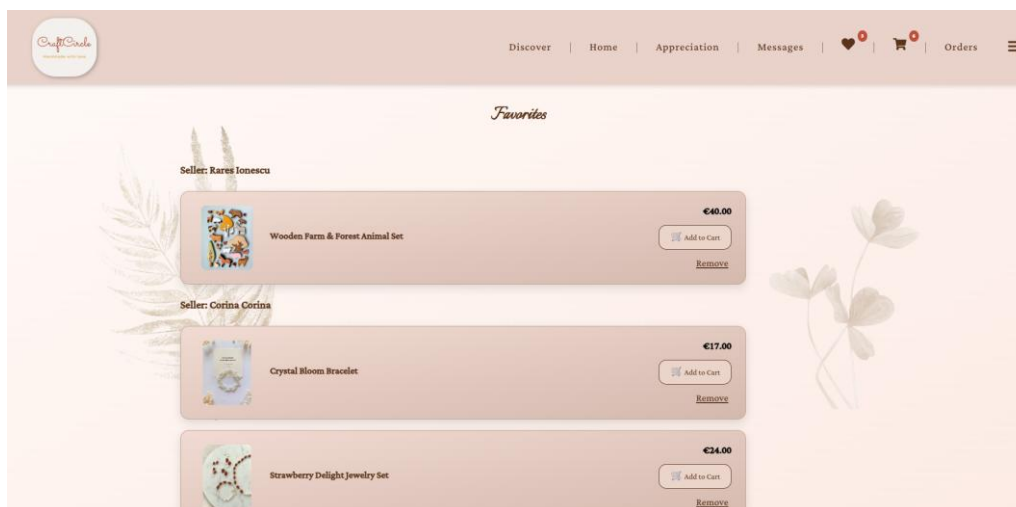


Figura 5.23 Interfața paginii de favorite

5.6 PAGINA COȘ DE CUMPĂRĂTURI

Pagina „Shopping Cart” centralizează toate produsele adăugate de utilizator în coș. Se accesează din navbar prin pictograma de coș, însoțită de un indicator numeric actualizat în timp real ([Figura 5.24](#)).

Adăugarea unui produs se face prin butonul „Add to Cart” de pe cardul produsului. Dacă este apăsat de mai multe ori, cantitatea din coș crește corespunzător, iar indicatorul se actualizează instant.

Produsele din coș sunt grupate după vânzător. Fiecare card afișează imaginea, numele, prețul unitar și cantitatea, ajustabilă cu butoanele „+” și „-”. Produsele pot fi eliminate cu butonul „Delete”, iar actualizarea este imediată.

În dreapta paginii se află secțiunea „Order Summary”, unde este afișat totalul comenzii. Apăsarea butonului „Shipping Details” deschide un modal cu informațiile de livrare: țara și orașul destinatarului, țara vânzătorului și costul transportului. Transportul este 5€ pentru livrare internă și variabil pentru livrare internațională. Dacă totalul depășește 100€, livrarea este gratuită.

După verificarea transportului, utilizatorul apasă „Checkout”. Se deschide modalul „Payment and Delivery Info”, unde se selectează metoda de plată: „Card” sau „Cash” ([Figura 5.25](#), [Figura 5.26](#)).

Pentru „Cash”, utilizatorul completează adresa și finalizează cu „Confirm Order”. Pentru „Card”, se apasă „Pay with Card”, se introduc detaliile și, după confirmare, comanda este salvată, coșul golit și utilizatorul redirecționat către pagina „Orders”.

Adresa de livrare este preluată automat din profil. Dacă este nevoie de altă adresă, aceasta trebuie modificată anterior în secțiunea „Edit Profile”.

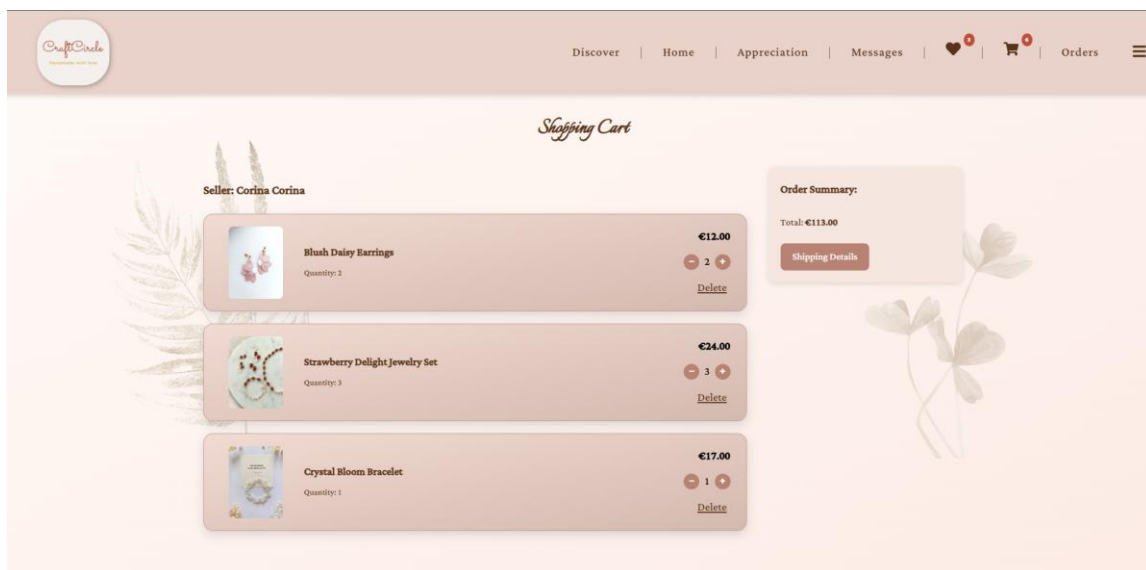


Figura 5.24 Interfața paginii „Shopping Cart”

Figura 5.25 Formular comandă plată ramburs

Figura 5.26 Formular comandă plată prin card

5.7 PAGINA COMENZI

Pagina Orders permite utilizatorilor să consulte detaliile comenzilor plasate. Aceasta poate fi accesată din bara de navigație, prin tabul Orders, sau este afișată automat prin redirectionare imediat după finalizarea unei comenzi.

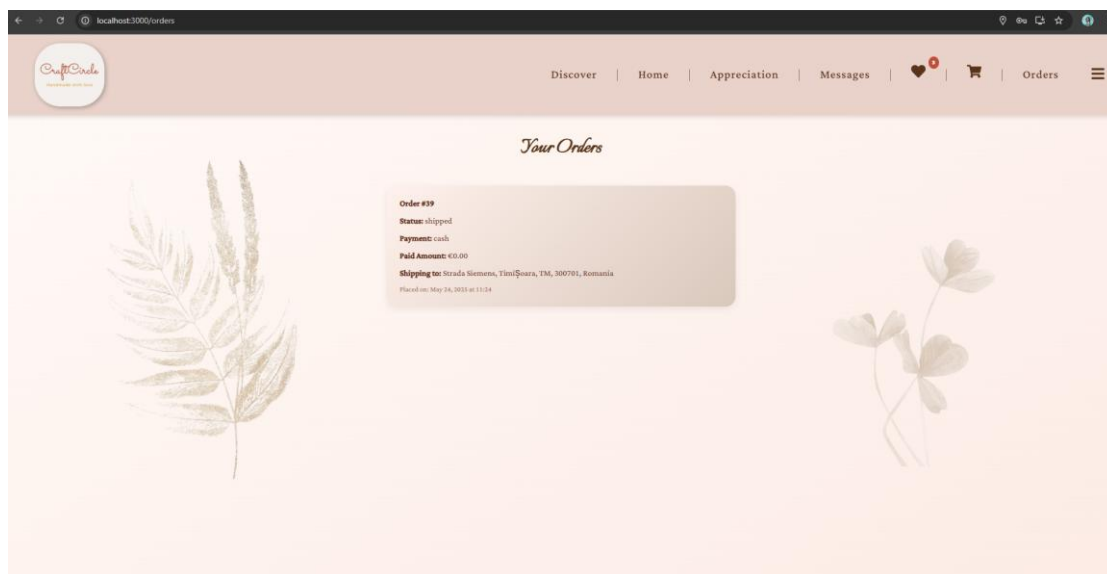


Figura 5.27 Interfață pagina comenzi

5.8 MENIUL DIN BARA DE NAVIGAȚIE

Meniul dropdown, accesibil din colțul din dreapta sus al aplicației, oferă utilizatorului acces rapid la acțiuni legate de contul său. Prin apăsarea iconiței tip meniu (≡), se deschide o listă de opțiuni, fiecare declanșând o funcționalitate specifică:

Change Password

Această opțiune deschide un modal dedicat schimbării parolei. Formularul conține trei câmpuri:

- Old Password
- New Password
- Confirm Password

După actualizarea cu succes, utilizatorul primește un email de notificare privind schimbarea parolei.

Edit Profile

Prin această opțiune, utilizatorul poate edita datele personale:

- Țara
- Orașul
- Informațiile din bio

Modalul preia automat valorile curente și permite modificarea lor. După apăsarea butonului Save, datele sunt actualizate.

Contact

Această opțiune deschide un modal informativ cu detalii de contact:

- Numărul de telefon disponibil
- Intervalul orar în care se poate apela

- Adresa de email a suportului tehnic

Este o secțiune statică, doar cu rol informativ pentru utilizator.

Delete Account

Selectarea acestei opțiuni declanșează un modal de confirmare, cu mesaj de avertisment. Utilizatorul este întrebat dacă dorește să șteargă definitiv contul și toate datele asociate.

Confirmarea duce la eliminarea definitivă a contului din sistem.

Logout

Această opțiune deschide un modal de confirmare pentru delogare. Dacă utilizatorul confirmă, este deconectat din cont și redirectionat automat către pagina de login.

Figura 5.28 Formular schimbare parolă

Figura 5.29
Formular editare date
personale

Figura 5.30 Modal de
contact

Figura 5.31 Modal confirmare
ștergere cont

Figura 5.32 Modal confirmare ieșire din
cont

6. UTILIZAREA APLICAȚIEI DE CĂTRE ADMINISTRATOR

6.1 AUTENTIFICARE ADMIN

Autentificarea administratorului în aplicația „CraftCircle” se realizează prin aceeași metodă standard de autentificare ca și în cazul utilizatorilor obișnuiți, folosind un formular de login ce solicită adresa de email și parola. La validarea corectă a acestor date, administratorul este redirecționat către pagina de profil personală.

Pentru detalii despre procesul de autentificare standard, inclusiv validările se poate consulta secțiunea [5.1.3](#).

6.2 PAGINA DE PROFIL

După autentificare, administratorul are acces la o pagină de profil informativă, accesibilă din navbar prin butonul „Dashboard”. Pagina afișează numele, emailul, orașul, țara, data nașterii și data creării contului.

În stânga cardului de profil se află poza de profil, modificabilă prin pictograma de tip creion. La apăsare apar butoanele „Choose the file”, „Save” și „Revert” pentru schimbarea imaginii.

Sub profil se află secțiunea „Admin Actions”, unde este afișat numărul total de utilizatori, postări și produse șterse de administratorul curent.

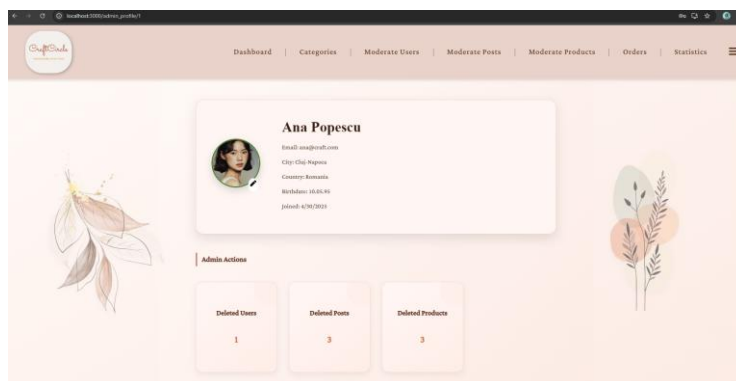


Figura 6.1 Interfață pagina profil administrator

6.3 GESTIONAREA CATEGORIILOR

Pagina „Manage Categories” permite administratorului să vizualizeze toate categoriile din platformă, alături de descriere, numărul de postări și produse asociate fiecăreia. Categoriile care nu conțin nicio postare și niciun produs pot fi șterse prin apăsarea butonului „Delete”.

Administratorul poate adăuga o nouă categorie completând numele și, opțional, o descriere, apoi apăsând Add. De asemenea, există posibilitatea de a filtra categoriile existente fie după nume, fie după descriere, folosind câmpurile dedicate de căutare.

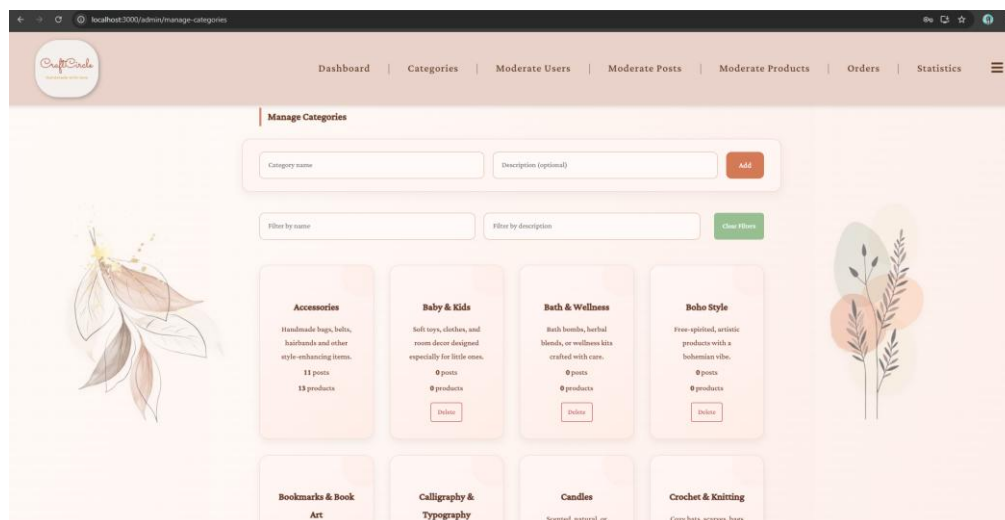


Figura 6.2 Interfață pagina „Manage Categories”

6.4 MODERAREA UTILIZATORILOR

Pagina „Moderate Users” afișează toți utilizatorii existenți în aplicație, ordonați descrescător după numărul de raportări primite. Pentru fiecare utilizator este afișat numele, emailul și numărul de raportări. Utilizatorii pot fi căutați rapid după ID, nume sau email prin bara de căutare din partea superioară a paginii.

Apăsând pe pictograma Reports, se deschide o căsuță care afișează detalii despre fiecare raport: cine l-a făcut și conținutul mesajului. Numele utilizatorului care a raportat este clickabil și redirecționează către pagina lui de profil. La fel, dacă se face click pe poza sau numele utilizatorului raportat, se ajunge în pagina de profil a acestuia.

În cazul în care se dorește ștergerea unui cont, se apasă butonul „Delete”, urmat de confirmarea acțiunii într-un modal care include numele utilizatorului. Dacă se confirmă, utilizatorul este șters, iar acesta va primi automat un email de notificare privind eliminarea contului.

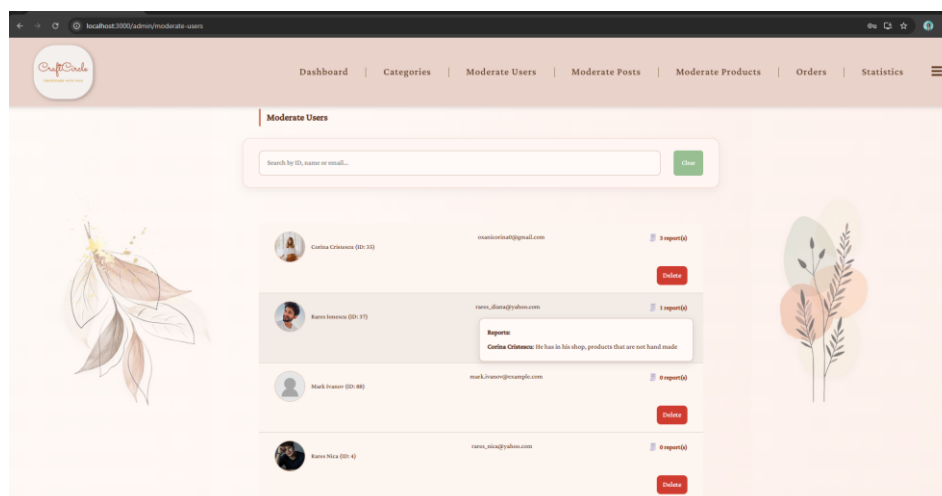


Figura 6.3 Interfață pagina „Moderate Users”

6.5 MODERAREA POSTĂRILOR

Pagina „Moderate Posts” afișează toate postările din platformă, ordonate descrescător după numărul de raportări. Fiecare postare este afișată într-un card care conține numele și ID-ul utilizatorului, descrierea, numărul de like-uri și un carusel cu imaginile postării.

Postările pot fi filtrate rapid folosind câmpurile din partea de sus, după numele utilizatorului, ID-ul acestuia sau conținutul postării.

La apăsarea pictogramei de tip steag, se afișează o căsuță cu detalii despre raportare: utilizatorul care a făcut raportarea și ID-ul său. Căsuța se închide automat prin click în afara cartonașului.

Dacă se face click pe un cartonaș de postare (în afara butoanelor), utilizatorul este redirecționat către pagina de profil a autorului postării, unde acea postare este automat evidențiată pentru a facilita verificarea.

Prin apăsarea butonului „See Comments” situat sub galeria de imagini, administratorul poate accesa un modal în care sunt afișate toate comentariile utilizatorilor la postarea respectivă.

Prin butonul „Delete Post”, administratorul poate șterge postarea respectivă. Acțiunea este confirmată printr-un modal, iar dacă se alege „Yes”, postarea este ștearsă, iar autorul este notificat prin email.

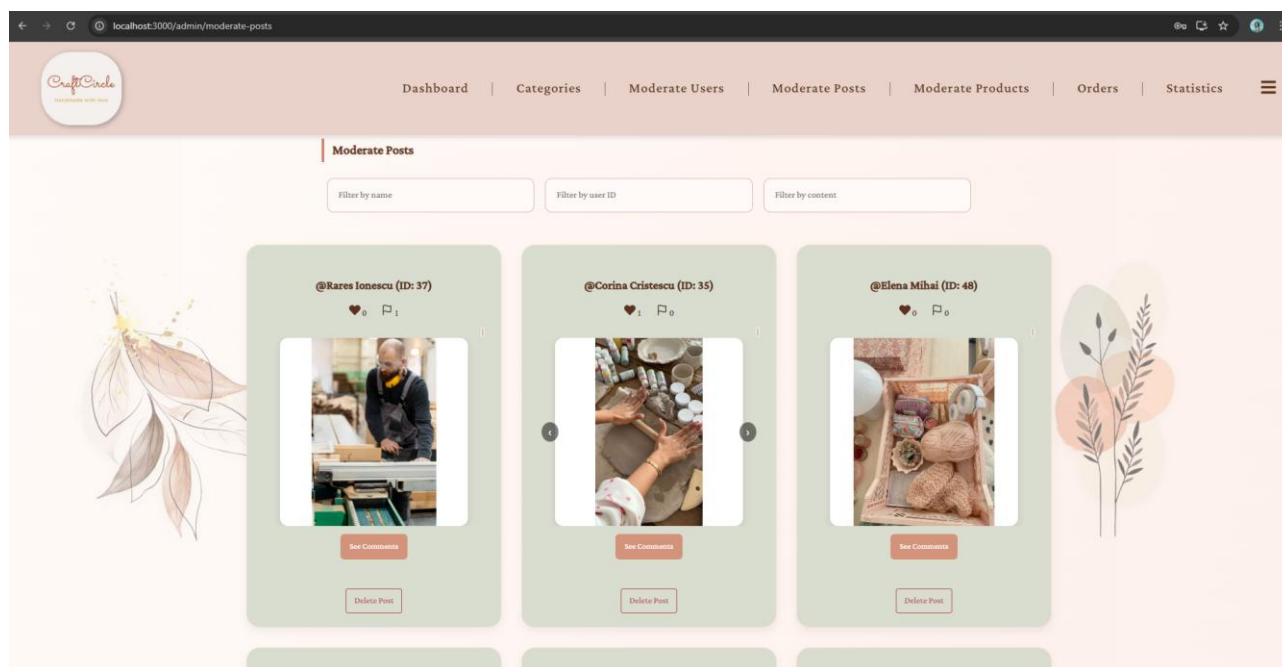


Figura 6.4 Interfață pagin „Moderate posts”

6.6 MODERAREA PRODUSELOR

Pagina „Moderate Products” afișează toate produsele din platformă sub formă de carduri, fiecare conținând numele și ID-ul utilizatorului, o descriere, precum și una sau mai multe imagini.

Produsele pot fi filtrate rapid după nume, ID-ul utilizatorului sau descriere. Dacă se face click pe pictograma steag, este afișat numele utilizatorului care a raportat acel produs. Căsuța se închide automat prin click în afara cardului.

Apăsând oriunde pe cardul produsului (în afara butoanelor), se face redirecționarea către pagina de shop a utilizatorului respectiv. Pagina se va derula automat și va evidenția produsul pe care s-a făcut click, pentru a facilita identificarea acestuia.

Produsul poate fi șters folosind butonul „Delete Product”, urmat de confirmarea acțiunii într-un modal. Dacă se confirmă, produsul este eliminat, iar utilizatorul este notificat prin email.

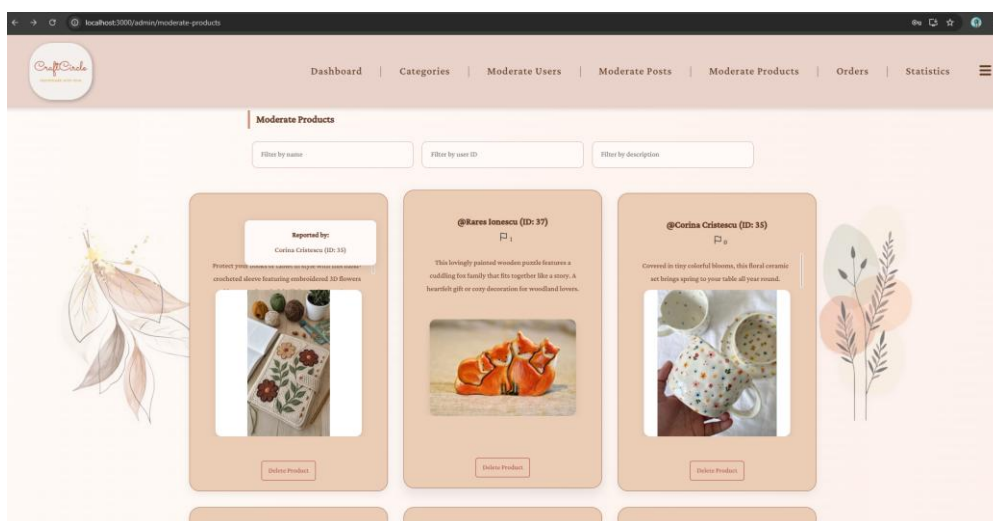


Figura 6.5 Interfață pagina „Moderate Products”

6.7 MANAGEMENTUL COMENZILOR

Pagina „Admin Orders” permite administratorului să vizualizeze și să gestioneze toate comenzile din platformă, afișate sub formă de carduri. Fiecare card include ID-ul comenzii, ID-ul utilizatorului, suma plătită și suma totală, metoda de plată (card sau ramburs), statusul comenzii, data plasării și adresa de livrare.

Statusul comenzii este editabil. Dacă se apasă pe pictograma de tip creion, se activează un meniu dropdown cu opțiunile Pending, Shipped, Cancelled și Completed, iar modificarea se salvează prin butonul Save.

În partea superioară a paginii, administratorul poate filtra comenzile după: ID-ul utilizatorului, țara de destinație, data comenzii, status și interval de preț (minim și maxim). Aceste filtre facilitează căutarea rapidă și organizarea eficientă a comenzilor.

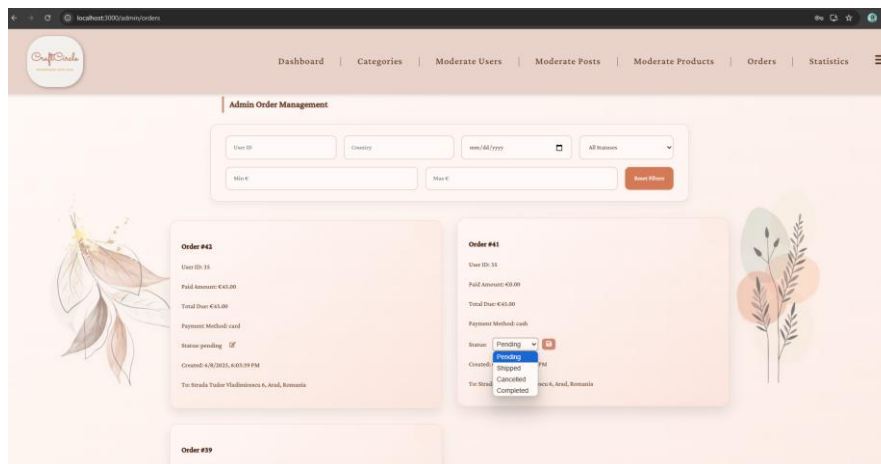


Figura 6.6 Interfață pagin „Admin Order Management”

6.8 STATISTICI GENERALE

Pagina „Statistics” are rol exclusiv informativ și oferă o privire de ansamblu asupra activității din platformă. În partea superioară sunt disponibile șase taburi: „Users”, „Posts”, „Products”, „Messages”, „Orders” și „App”.

În tabul „Users”, se afișează numărul total de utilizatori și utilizatorii noi din luna curentă. La apăsarea pe cartonașe, se deschide lista detaliată a utilizatorilor, care poate fi filtrată după nume, email sau ID. Dacă se selectează un utilizator, se face redirecționarea către profilul acestuia. Un grafic interactiv afișează evoluția numărului total de utilizatori în ultimele 12 luni.

În taburile „Posts” și „Products”, sunt afișate totalurile și numărul de postări sau produse raportate.

Tabul „Messages” arată numărul total de mesaje trimise pe platformă, iar „Orders” afișează numărul total de comenzi procesate.

În secțiunea „App”, sunt prezentate două valori globale: numărul total de like-uri și numărul total de relații de tip „follow” dintre utilizatori.



Figura 6.7 Statistici despre utilizatori în panoul de administrare

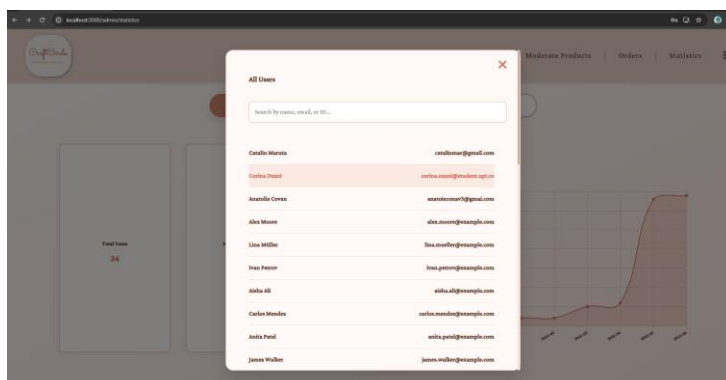


Figura 6.8 Lista utilizatorilor afișată în modalul de administrare

6.9 MENIUL DIN BARA DE NAVIGARE

În colțul din dreapta sus al Navbar-ului, administratorul are acces la un meniu dropdown cu două opțiuni: „Change Password” și „Logout”.

Funcționalitatea acestor opțiuni este identică cu cea oferită utilizatorilor standard. Aceste funcționalități sunt detaliate deja în subcapitolul [5.8](#).

7. TESTARE ȘI VALIDARE

Testarea aplicației „CraftCircle” s-a desfășurat exclusiv în mediul local (localhost), utilizând browsere moderne, în principal Google Chrome, și scenarii predefinite pentru a verifica funcționarea corectă a principalelor fluxuri de utilizare. Deși aplicația nu a fost încă publicată online, procesul de testare a vizat stabilitatea acesteia și identificarea eventualelor erori sau comportamente neprevăzute în interfață și funcționalități.

7.1 TESTARE FUNCȚIONALĂ (BLACK BOX)

Testarea funcțională a fost realizată manual, fără accesarea codului sursă, și a vizat modul în care aplicația răspunde la diferite acțiuni ale utilizatorului. Au fost testate componente precum înregistrarea și autentificarea utilizatorilor, inclusiv validarea erorilor în cazul unor câmpuri necompletate sau parole incorecte. S-au verificat de asemenea procesele de adăugare și editare a postărilor și produselor, pentru a ne asigura că datele sunt salvate corect în baza de date și reflectate în interfață.

Fluxul de adăugare a produselor în coș, modificarea cantității și plasarea comenzilor au fost testate pentru a confirma persistența informațiilor și trimiterea cu succes a detaliilor legate de plată și livrare. Funcționalitățile sociale, precum aprecierea postărilor, urmărirea altor utilizatori, adăugarea de comentarii și generarea notificărilor, au fost de asemenea verificate.

Un aspect important l-a reprezentat testarea sistemului de mesagerie și a notificărilor în timp real, implementate cu ajutorul WebSocket-urilor prin biblioteca Socket.IO. Pentru validarea acestor funcționalități, aplicația a fost deschisă simultan în două sesiuni separate de browser (una normală și una în modul incognito), folosind două conturi diferite. Prin trimiterea de mesaje și interacțiuni între cele două conturi, s-a observat afișarea instantanee a mesajelor, marcarea lor ca necitite și actualizarea badge-urilor de notificare în timp real.

Toate testările s-au realizat în browser (Google Chrome), prin navigare manuală și observarea comportamentului aplicației în timp real.

7.2 TESTARE STRUCTURALĂ (WHITE BOX)

Fiind dezvoltatorul unic al aplicației, am desfășurat și testarea structurală (White Box), analizând în detaliu codul sursă pentru a verifica funcționarea logicii aplicației. Pe partea de backend, au fost testate endpoint-urile API folosind Postman, verificând răspunsurile JSON, codurile de status HTTP și manipularea corectă a datelor în PostgreSQL. În frontend, testarea a inclus inspecția componentelor React, verificarea stării (state) și a props-urilor prin instrumentele de debug din browser.

Baza de date a fost analizată atât prin interogări SQL directe, cât și prin utilizarea interfeței grafice oferite de pgAdmin, fără a apela exclusiv la comenzi scrise. Astfel, au fost verificate operațiuni precum inserarea, actualizarea și ștergerea datelor, fie prin aplicație, fie

direct din pgAdmin. De asemenea, au fost simulate diverse scenarii de eroare, cum ar fi introducerea unor parole neconforme, completarea parțială a adresei în timpul procesului de checkout sau autentificarea cu utilizatori inexistenți, pentru a evalua reacția aplicației și corectitudinea validărilor implementate.

Testarea a fost realizată pe un sistem de operare Windows 10, utilizând browserul Google Chrome (versiunea 137.0.7151.69), Node.js (versiunea 20.12.0), PostgreSQL (versiunea 17.2) și biblioteca React (versiunea 18.2.0).

7.3 LIMITĂRI ȘI DIRECȚII VIITOARE

Având în vedere că aplicația nu a fost încă publicată online, testarea s-a realizat exclusiv în mediul local, fără implicarea unui public mai larg și fără a putea acoperi toate situațiile posibile de utilizare. Fiind un proiect realizat integral pe cont propriu, fără contribuția altor persoane în procesul de testare, este posibil ca unele probleme de utilizare sau incompatibilități să fi trecut neobservate până în acest punct.

În stadiul actual, aplicația are câteva limitări funcționale importante. De exemplu, funcționalitatea comenzilor este doar parțial implementată: utilizatorii pot vedea doar comenzile plasate de ei, însă nu și comenzile primite în propriul shop. Totodată, nu există încă o funcție pentru returnarea produselor, ceea ce reduce opțiunile utilizatorilor după finalizarea unei comenzi. Interfața nu este complet adaptată pentru ecrane mici, iar în cazul redimensionării ferestrei browserului sau al accesării aplicației de pe dispozitive cu rezoluții reduse, pot apărea unele probleme de afișare și aliniere.

Pentru a pregăti aplicația pentru o eventuală lansare, urmează să fie aduse mai multe îmbunătățiri. Printre acestea se numără publicarea aplicației pe un server de test sau producție, adaptarea interfeței pentru a funcționa corect pe ecrane de dimensiuni diferite și organizarea unei perioade de testare beta, în care să fie colectat feedback real de la utilizatori. De asemenea, este planificată extinderea sistemului de comenzi, astfel încât fiecare vânzător să poată vizualiza comenzile primite, precum și implementarea unei funcționalități dedicate retururilor.

8. CONCLUZII

Aplicația „CraftCircle” și-a atins obiectivele propuse, oferind un spațiu digital complet funcțional pentru promovarea și vânzarea produselor handmade, cu accent pe ușurința utilizării și interacțiunea autentică dintre utilizatori. Platforma a combinat funcționalități specifice unui marketplace cu elemente de rețea socială, construind un mediu prietenos pentru comunitatea creatorilor.

Am realizat integral această aplicație, de la definirea ideii și proiectarea arhitecturii, până la implementarea completă a interfeței, a logo-ului, a logicii backend și a tuturor funcționalităților necesare pentru fiecare tip de utilizator. Structura a fost concepută într-un mod modular, pentru a permite o dezvoltare clară, o organizare eficientă și o întreținere ușoară.

Platforma a fost implementată pentru două tipuri principale de utilizatori: utilizatorul standard și administratorul.

Utilizatorii standard își pot crea și personaliza profilul, având posibilitatea de a încărca o imagine de profil și de a actualiza datele personale. Autentificarea este disponibilă atât clasic, prin email și parolă, cât și prin conturi Google sau Facebook.

Aceștia pot publica postări cu imagini, descrieri și categorii, având opțiunea de a le edita sau șterge ulterior. De asemenea, interacțiunea cu postările altor utilizatori este facilitată prin funcționalități de apreciere, comentariu și raportare.

În ceea ce privește componenta de marketplace, fiecare utilizator poate adăuga produse în propriul magazin virtual, cu posibilități complete de gestionare. Procesul de achiziție este susținut de un sistem de coș de cumpărături, cu opțiunea de plasare a comenzilor și plată online prin Stripe sau ramburs.

Comunicarea directă între utilizatori este realizată în timp real printr-un sistem de mesagerie bazat pe Socket.IO. Platforma oferă și funcționalități sociale precum urmărirea altor utilizatori și o secțiune dedicată aprecierilor primite.

În paginile de descoperire, utilizatorii pot explora postări și produse cu ajutorul unor filtre avansate, în funcție de conținut, popularitate sau categorie, ceea ce contribuie la o experiență de navigare personalizată.

Administratorii beneficiază de un panou dedicat, prin care pot modera conținutul publicat, gestiona conturile utilizatorilor standard, produsele puse spre vânzare, categoriile și comenzile, precum și accesa statistici privind activitatea de pe platformă.

Aplicația a fost structurată în două părți principale:

- client/ – frontend-ul, scris în React;
- server/ – backend-ul, dezvoltat cu Node.js și Express.

Frontend-ul a inclus peste 70 de fișiere .js, organizate în directoare precum: components/ (componente vizuale reutilizabile), pages/, Admin/Pages (paginile aplicației), modals/ (ferestre modale), hooks/, utils/ (logică auxiliară) și fișiere centrale ca App.js și index.js.

O componentă importantă a aplicației a fost reprezentată de ferestrele modale, pentru care am creat peste 10 fișiere dedicate. Aceste modale au fost utilizate pentru a permite interacțiuni rapide, fără schimbarea rutei și fără încărcarea altor pagini – oferind o

experiență fluidă și modernă. Exemple relevante includ: `EditProfileModal.js` (pentru actualizarea datelor de profil), `PasswordModal.js` și `NewPasswordModal.js` (pentru schimbarea sau resetarea parolei), `DeleteAccountModal.js` (pentru ștergerea contului), `PaymentModal.js` și `ConfirmationModal.js` (pentru procesul de checkout), `ReportUserModal.js`, `CommentsModal.js` (pentru raportări și interacțiuni sociale rapide), `VerifyCodeModal.js` (pentru validarea prin cod numeric la resetare).

Am utilizat CSS Modules pentru a asigura izolarea stilurilor la nivel de componentă, evitând astfel suprascriserile accidentale și asigurând un control precis asupra fiecărui element din interfață. Fiecare `div`, buton, card, bară de filtrare sau fereastră modală va fi personalizat vizual prin clase unice și reguli CSS adaptate, contribuind la o estetică unitară și profesională a platformei.

Backend-ul a fost împărțit în:

- `config/` – fișiere pentru conectarea la baza de date, autentificare (Google, Facebook), Nodemailer și Cloudinary;
- `routes/` – rutele aplicației, organizate tematic (`auth.js`, `users.js`, `posts.js`, `orders.js`, `stripe.js`, `admin/statistics.js` etc.); În total au fost definite 100 de rute backend RESTful, grupate în mod clar și prezentate în [Tabelul 4.1](#).
- `server.js` – fișierul central de inițializare al aplicației.

Baza de date (PostgreSQL) a fost structurată în tabele relaționale, printre care: `accounts`, `posts`, `orders`, `comments`, `favorites`, `messages`, `categories`, `post_reports`, `post_search_history`, `marketplace_items` etc. Datele de test au fost introduse manual pentru a valida funcționalitățile și a simula scenarii reale. Pentru a evidenția vizual funcționalitățile de publicare, promovare și achiziționare a produselor, am utilizat imagini reprezentative preluate de pe Pinterest, exclusiv în scop demonstrativ și fără intenții comerciale. Acestea au rol ilustrativ și ajută la înțelegerea comportamentului aplicației în context real.

Securitatea aplicației a fost tratată cu atenție: toate datele confidențiale (chei Stripe, credențiale email, parole, tokenuri API, date DB) au fost stocate în fișiere `.env`, atât în frontend, cât și în backend. Aceste fișiere au fost excluse din GitHub prin `.gitignore`, împreună cu `node_modules/`, pentru a păstra un repository curat și profesionist.

În etapa de testare, aplicația a fost rulată local, iar datele au fost inspectate folosind pgAdmin și interacțiuni directe în UI.

Chiar dacă anumite funcționalități (precum vizualizarea comenzilor primite de către vânzători, retururile sau optimizarea completă pentru ecrane mai mici) nu au fost încă implementate, aplicația oferă o bază stabilă și extensibilă pentru dezvoltări viitoare. „CraftCircle” demonstrează astfel potențialul unei aplicații realizate integral, de la idee la produs funcțional.

În faza inițială de proiectare, anumite interfețe precum cele de autentificare și înregistrare au fost realizate în Figma, cu scopul de a contura o idee generală a designului site-ului și de a oferi un punct de pornire vizual pentru implementarea efectivă (Figura 8.1). Chiar dacă aspectul final a suferit modificări pe parcurs, prototipul inițial a contribuit la structurarea clară a interfeței.

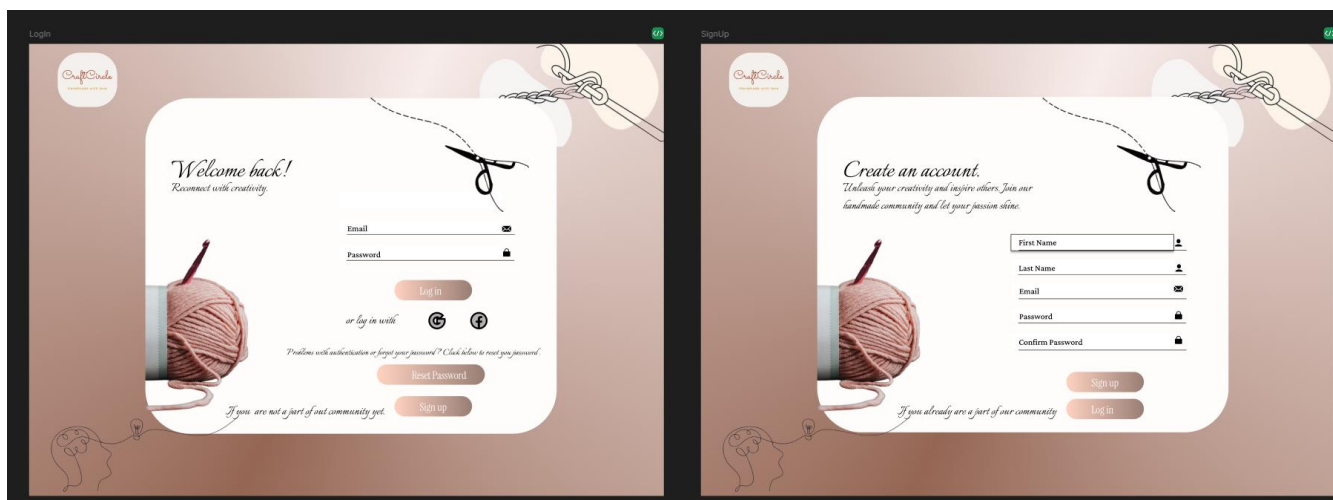


Figura 0.1 Interfețele pentru autentificare și înregistrare realizate în Figma

BIBLIOGRAFIE

- [1] Meta Open Source – Introducing React: <https://react.dev/> . Accesat la: 20 iunie 2025.
- [2] React Docs – Despre props: <https://react.dev/learn/passing-props-to-a-component>. Accesat la: 20 iunie 2025.
- [3] Babel – JSX in Depth: <https://babeljs.io/docs/en/babel-plugin-transform-react-jsx>. Accesat la: 20 iunie 2025.
- [4] React – FAQ: Virtual DOM and Internals: <https://legacy.reactjs.org/docs/faq-internals.html>. Accesat la: 20 iunie 2025.
- [5] React Docs – useState: <https://react.dev/reference/react/useState>. Accesat la: 20 iunie 2025.
- [6] React Docs – useEffect: <https://react.dev/reference/react/useEffect>. Accesat la: 20 iunie 2025.
- [7] React Docs – useRef: <https://react.dev/reference/react/useRef>. Accesat la: 20 iunie 2025.
- [8] React Router Docs – useNavigate: <https://reactrouter.com/en/main/hooks/use-navigate>. Accesat la: 20 iunie 2025.
- [9] React Router Docs – useLocation: https://api.reactrouter.com/v7/functions/react_router.useLocation.html. Accesat la: 20 iunie 2025.
- [10] MDN Web Docs – localStorage: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>. Accesat la: 20 iunie 2025.
- [11] React Toastify – <https://fkhadra.github.io/react-toastify>. Accesat la: 20 iunie 2025.
- [12] React Icons Docs: <https://react-icons.github.io/react-icons/> . Accesat la: 20 iunie 2025.
- [13] react-chartjs-2 Docs – <https://react-chartjs-2.js.org>. Accesat la: 20 iunie 2025.
- [14] Stripe React SDK – <https://stripe.com/docs/stripe-js/react>. Accesat la: 20 iunie 2025.
- [15] Socket.IO Client Docs – <https://socket.io/docs/v4/client-api/>. Accesat la: 20 iunie 2025.
- [16] CSS Modules. GitHub. Disponibil la: <https://github.com/css-modules/css-modules>. Accesat la: 20 iunie 2025.
- [17] Create React App. Adding CSS Modules. Disponibil la: <https://create-react-app.dev/docs/adding-a-css-modules-stylesheet/>. Accesat la: 20 iunie 2025.
- [18] Figma – Collaborative interface design tool: <https://www.figma.com/design/>
- [19] Axios – <https://axios-http.com/docs/intro>. Accesat la: 20 iunie 2025.
- [20] REST Countries API <https://restcountries.com/v3.1/all>. Accesat la: 20 iunie 2025.
- [21] CountriesNow Cities API – <https://countriesnow.space/api/v0.1/countries/cities> (acceptă doar cereri POST cu body JSON). Accesat la: 20 iunie 2025.
- [22] Zippopotam.us API – <https://api.zippopotam.us/>. Accesat la: 20 iunie 2025.
- [23] OpenRouteService API – <https://openrouteservice.org/dev/#/api-docs>. Accesat la: 20 iunie 2025.
- [24] Node.js Official Docs – About: <https://nodejs.org/en/about>. Accesat la: 20 iunie 2025.

- [25] Node.js Docs – Package Management: <https://nodejs.org/en/learn/getting-started/an-introduction-to-the-npm-package-manager> . Accesat la: 20 iunie 2025.
- [26] Express.js Docs – Introduction: <https://expressjs.com/en/starter/installing.html>. Accesat la: 20 iunie 2025.
- [27] dotenv – Documentație oficială: <https://www.npmjs.com/package/dotenv>. Accesat la: 20 iunie 2025.
- [28] body-parser – Documentație oficială: <https://www.npmjs.com/package/body-parser>. Accesat la: 20 iunie 2025.
- [29] cors – Documentație oficială: <https://www.npmjs.com/package/cors>. Accesat la: 20 iunie 2025.
- [30] nodemailer – Documentație oficială: <https://nodemailer.com/about/>. Accesat la: 20 iunie 2025.
- [31] multer – Documentație oficială: <https://www.npmjs.com/package/multer>. Accesat la: 20 iunie 2025.
- [32] Passport – Documentație oficială: <https://www.passportjs.org/docs/>. Accesat la: 20 iunie 2025.
- [33] node-postgres (pg) – <https://node-postgres.com/>. Accesat la: 20 iunie 2025.
- [34] Stripe Node.js SDK – <https://stripe.com/docs/api?lang=node>. Accesat la: 20 iunie 2025.
- [35] Cloudinary – Official Site: <https://cloudinary.com/>. Accesat la: 20 iunie 2025.
- [36] Cludinary usage in Node - https://cloudinary.com/documentation/node_integration. Accesat la: 20 iunie 2025.
- [37] Hashing Passwords with Bcrypt in Node.js <https://www.freecodecamp.org/news/how-to-hash-passwords-with-bcrypt-in-nodejs/>. Accesat la: 20 iunie 2025.
- [38] Socket.IO – Official Documentation: <https://socket.io/>. Accesat la: 20 iunie 2025.
- [39] Stripe – Descriere generală: <https://stripe.com>. Accesat la: 20 iunie 2025.
- [40] Stripe API Reference – Payment Intents: https://docs.stripe.com/api/payment_intents. Accesat la: 20 iunie 2025.
- [41] PostgreSQL Official Site – About: <https://www.postgresql.org/about/>. Accesat la: 20 iunie 2025.
- [42] PostgreSQL Docs – Features: <https://www.postgresql.org/docs/current/>. Accesat la: 20 iunie 2025.
- [43] PostgreSQL Docs – Referential Actions: <https://www.postgresql.org/docs/current/ddl-constraints.html#DDL-CONSTRAINTS-FK>. Accesat la: 20 iunie 2025.
- [44] node-postgres (pg) – <https://node-postgres.com/>. Accesat la: 20 iunie 2025.
- [45] pgAdmin – PostgreSQL Tools: <https://www.pgadmin.org/>. Accesat la: 20 iunie 2025.
- [46] Postman – Official Site: <https://www.postman.com/>. Accesat la: 20 iunie 2025.

Anexa 2

DECLARAȚIE DE AUTENTICITATE A
LUCRĂRII DE FINALIZARE A STUDIILOR*

Subsemnatul OXANI CORINA

legitimată cu permis vedere seria _____ nr. 000049292

CNP 8010205350020

autorul lucrării CraftCircle - Aplicație web dedicată promovării,
vânzării și schimbului de idei între pasionații de produse handmade

elaborată în vederea susținerii examenului de finalizare a studiilor de
LICENȚĂ

organizat de către Facultatea
de Automatică și Calculatoare din cadrul Universității

Politehnică Timișoara, sesiunea iulie a anului universitar

2025, coordonator Răzvan Cioară, luând în

considerare prevederile Capitolului V - Măsuri privind asigurarea originalității lucrărilor din
*Regulamentul privind organizarea și desfășurarea examenelor de licență/diplomă și disertație
în Universitatea Politehnică Timișoara*, aprobat prin HS nr.30 /21.03.2024 și cunoscând faptul
că în cazul constatării ulterioare a unor încălcări ale normelor de etică/a faptului că diploma a
fost obținută prin mijloace frauduloase, voi suporta sancțiunile legale prevăzute de Legea nr.
199/2023 – Legea Învățământului Superior, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale;
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului;
- sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor;
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen/prezentată public/publicată de licență/diplomă/disertație;
- În elaborarea lucrării am utilizat instrumente specifice inteligenței artificiale (IA) și anume _____ (denumirea) _____ (sursa), pe care le-am citat în conținutul lucrării/nu am utilizat instrumente specifice inteligenței artificiale (IA)¹.

Declar că sunt de acord ca lucrarea să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului său într-o bază de date în acest scop.

Timișoara,

Data

28.06.2025

Semnătura

Oxani Corina

¹*Declarația se completează de student, se semnează olograf de acesta și se inserează în lucrarea de finalizare a studiilor, la sfârșitul lucrării, ca parte integrantă.

² Se va păstra una dintre variante: 1 - s-a utilizat IA și se menționează sursa 2 – nu s-a utilizat IA